

Informatik
Hochschule Luzern (Schweiz)

WiFi CSI PEOPLE DETECTION

BACHELORARBEIT

vorgelegt am Departement Informatik der Hochschule Luzern (Schweiz) in Anbetracht zur
Erreichung des akademischen Grades *Bachelor in Informatics*.

von

Fabian Stettler

aus

Luzern (Schweiz)

Erklärung

Bachelorarbeit an der Hochschule Luzern - Informatik

Titel:	WiFi CSI People Detection
Studentin/Student:	Fabian Stettler
Studiengang:	Bachelor in Informatics
Abschlussjahr:	2026
Betreuungsperson:	Prof. Dr. Björn Jensen
Expertenperson:	Kai Waelti
Auftraggeberin/Auftraggeber:	Robotics Lab HSLU

Codierung/Klassifizierung der Arbeit

- ☐ Öffentlich (Normalfall)
- ☐ Vertraulich

Eidesstattliche Erklärung

Ich erkläre hiermit, dass ich die vorliegende Arbeit selbständig und ohne unerlaubte fremde Hilfe angefertigt habe, alle verwendeten Quellen, Literatur und andere Hilfsmittel angegeben habe, wörtlich oder inhaltlich entnommene Stellen als solche kenntlich gemacht habe das Vertraulichkeitsinteresse des Auftraggebers wahren und die Urheberrechtsbestimmungen der Hochschule Luzern respektieren werde.

Ort/Datum, Unterschrift _____

Verdankung

Danke an meine Familie, die mir bei der Fertigstellung der Arbeit geholfen haben. Danke auch an meinen zu jeder Zeit enthusiastischen und mit konstruktivem Feedback unterstützenden Betreuer Prof. Dr. Björn Jensen.

Fabian Stettler, 2026

*Geistiges Eigentum gemäss der Studienordnung für die Ausbildung
an der Hochschule Luzern, FH Zentralschweiz*

Inhaltsverzeichnis

1	Einleitung	1
1.1	Problem	1
1.2	Ziele der Arbeit	2
1.3	Aufgaben	3
2	Stand der Forschung	4
2.1	WiFi Technologie	4
2.2	Aktueller Stand des CSI Forschungsfeldes	8
3	Ideen und Konzepte	13
3.1	Architektur des Modell von Yan	13
3.2	CNN-Attention-Diffusion Architektur	17
3.3	Ground Truth Landmark Datengenerierung	21
3.4	CSI Datengenerierung	24
4	Methodik	26
4.1	Projektplanung	26
4.2	Risiko-Analyse	27
5	Realisierung	28
5.1	Initialisieren der Router	28
5.2	Datenerfassung	30
5.3	Generierung der 3D-Koordinaten	35
5.4	Export in Ground Truth Daten	38
5.5	Hardware Trainingsinfrastruktur	39
5.6	Live-Framework	39
5.7	Überwundene Schwierigkeiten	40
6	Validation und Evaluation	41
6.1	Resultate	41
6.2	Validation	44
7	Diskussion	46
7.1	Wahl der Machine Learning Modelle	46
7.2	Wahl des Router-Setups	46
7.3	Wahl der Ground-Truth-Generierung	47
7.4	Wahl der Trainingsinfrastruktur	47
8	Zusammenfassung und Ausblick	48
9	Anhänge und KI-Deklaration	53
9.1	KI-Deklaration	53
9.2	Anhänge	53

Abbildungsverzeichnis

2.1	Frequenzband 2.4 GHz	5
2.2	8-QAM Symbole	7
2.3	Setup Pulse Fi	9
2.4	Experiment-Setup nach Yan et al.	10
2.5	System Overview Geometry Aware Studie	12
3.1	Tokenisierung und Encoder	14
3.2	Pose Decoder	15
3.3	Refine Decoder	16
3.4	CNN-Encoder	18
3.5	Embedding und Attention	19
3.6	Pose Head	19
3.7	Diffusionsnetzwerk	20
3.8	Kamera-Koordinatentransformation	23
3.9	Ground Truth Datengenerierung	24
3.10	Router-Setup für CSI	25
4.1	Projektplanung	26
5.1	TP-Link Archer C7 v2	28
5.2	Netzwerkdiagramm	29
5.3	Matching-Window Grösse	30
5.4	Amplitude mit Person	31
5.5	Amplitude ohne Person	32
5.6	Amplitudenverteilung mit Person	33
5.7	Bewegungsmuster	34
5.8	Nachbearbeitung 3D-Koordinaten	36
6.1	Training-Loss Verlauf	42
6.2	Validation-Loss Verlauf	42
6.3	Resultate drei Frames	44
9.1	Aufgabenstellung	54
9.2	Aufgabenstellung	55
9.3	Planung	57
9.4	Risiko-Analyse	58

Tabellenverzeichnis

4.1	Wesentliche Projektrisiken	27
5.1	Varianzvergleich Person/keine Person	32
9.1	Verwendete Repositories	56
9.2	Verwendete Datensätze	59

Glossar

- Convolutional neural network** Eine Klasse von neuronalen Netzen, die häufig für die Bildanalyse verwendet wird und auf Faltungsoperationen basiert, um Merkmale aus Daten zu extrahieren.
- CSI** Channel-State-Information kann von WLAN Geräten ausgelesen werden. Ein CSI Wert wird als komplexe Zahl ausgedrückt, wobei der reale Teil dem Wert der Amplitude und der imaginäre Teil, der Phasenverschiebung entspricht.
- CTD-Modell** CTD steht für das eigens erstellte CNN-Transformer-Diffusion Modell, welches in der Arbeit vorgestellt wurde.
- Decoder** Ein Decoder macht das Gegenteil des Encoders und projiziert die Informationen aus der encodierten Dimension wiederum in eine andere Dimension oder in die gleiche Dimension wie der Output des Modells.
- Edge-Gerät** Gerät welches weniger Rechenressourcen als eine Workstation oder Laptop besitzt. Es kann an der "Edge" eingesetzt werden und kann zum Beispiel ein Raspberry PI.
- Encoder** Ein Encoder encodiert Informationen in einen speziellen Raum. Ein Encoder für einen Transformer encodiert die Informationen in eine mit dem Transformer kompatible Dimension, sodass der Transformer diese korrekt verarbeiten kann.
- Ground Truth** Referenzdaten mit bekannten korrekten Zielwerten, die zum Trainieren und Evaluieren eines Modells verwendet werden.
- LIDAR** LIDAR liefert Distanzwerte, indem sie messen, wie lange es dauert von Laser-Signalausendung bis das Signal vom Sensor auf dem LIDAR-Gerät wieder empfangen wird. Ein LIDAR Gerät kann so ein Bild einer Umgebung produzieren, entweder in 2D oder 3D (kugelförmig).
- mAP** mean Average Precision ist eine Evaluationsmetrik zur Bewertung der Leistung von Objekterkennungs- und Klassifikationsmodellen. Sie beschreibt die durchschnittliche Präzision über mehrere Recall-Schwellen hinweg.
- MediaPipe** Framework von Google zur Echtzeit-Verarbeitung von Bild- und Videodaten, unter anderem für Pose-Tracking.
- MIMO** Multiple Input Multiple Output bezeichnet ein Übertragungsverfahren, bei dem mehrere Sende- und Empfangsantennen gleichzeitig genutzt werden, um Daten parallel zu übertragen und den Datendurchsatz sowie die Signalqualität zu verbessern.
- MIMO-Kanalmatrix** Matrix, die den Übertragungskanal zwischen allen Sende- und Empfangsantennen eines MIMO-Systems beschreibt. Jeder Eintrag (i, j) enthält die komplexe CSI-Messung zwischen Sendeantenne j und Empfangsantenne i für einen bestimmten Subchannel und Zeitpunkt.

MPJPE Die euklidische Distanz zwischen 3D Koordinaten der Ground Truth Daten und vorhergesagten 3D Koordinaten. Dieser Wert wird dann pro Gelenk berechnet und danach über alle Gelenke gemittelt, was dann einen einzigen Wert produziert.

MSE MSE steht für Mean-Squared-Error. Diese Loss-Funktion bestraft grössere Fehler quadratisch und wird zur Regressionsoptimierung verwendet.

NTP Network Time Protocol ist ein Netzwerkprotokoll zur Synchronisation der Uhrzeit zwischen Computern und Geräten.

OpenWRT Linux-basiertes Open-Source-Betriebssystem für Router, das erweiterte Funktionen und Anpassungsmöglichkeiten bietet, einschließlich der Möglichkeit, CSI-Daten auszulesen.

QAM Quadratur-Amplitudenmodulation ist ein digitales Modulationsverfahren, bei dem Information über unterschiedliche Amplituden und Phasen von zwei orthogonalen Trägersignalen übertragen wird.

RSSI Received Signal Strength Indicator ist ein Indikator wie stark ein empfangenes Signal auf einem WLAN Client empfangen wird.

SiLU (Sigmoid Linear Unit) SiLU ist eine Aktivierungsfunktion. Sie wird in neuronalen Netzwerken verwendet und kombiniert lineare und nicht-lineare Eigenschaften, wodurch sie oft stabilere Trainingsdynamiken ermöglicht.

STE STE steht für Spatial Temporal Embedding und bezeichnet eine Repräsentation, die räumliche und zeitliche Informationen gemeinsam in einem Embedding abbildet.

Transformer Eine Modellarchitektur für sequenzielle Daten, die auf Self-Attention basiert und Abhängigkeiten zwischen Eingabeelementen effizient modelliert.

WiFi-Sensing Verwendung von WLAN-Signalen zur Erkennung von Personen, Bewegungen oder anderen Umweltveränderungen.

Akronyme

AP Access-Point.

CNN Convolutional Neural Network.

CSI Channel-State-Information.

CTD CNN-Transformer-Diffusion.

DETR Detection Transformer.

DWT Discrete Wavelet Transform.

GT Ground-Truth.

GUI Graphical User Interface.

LIDAR Light Detection and Ranging.

mAP mean average precision.

MIMO Multiple Input Multiple Output.

MLP Multi-Layer Perceptron.

MPJPE Mean Per Joint Precision Error.

MSE Mean Squared Error.

NTP Network Time Protocol.

RSSI Received Signal Strength Indicator.

SiLU Sigmoid Linear Unit.

STE Spatial Temporal Embedding.

TFTP Trivial File Transfer Protocol.

UDP User Datagram Protocol.

1. Einleitung

1.1 Problem

Die Personendetektion und Lokalisierung sind zentrale Bausteine moderner Sicherheits-, Assistenz- und Automatisierungssysteme. In öffentlichen Gebäuden, Pflegeeinrichtungen, Industrieanlagen oder smarten Wohnumgebungen müssen Personen zuverlässig erkannt, ihre Positionen bestimmt und Bewegungen im Raum verstanden werden. Dies ist relevant, um etwa Zutritte zu kontrollieren, Unfälle zu vermeiden, Abläufe zu automatisieren oder Menschen in sensiblen Bereichen zu schützen.

Kamerasysteme in Kombination mit Machine Learning haben in diesem Zusammenhang viele Probleme gelöst. Modelle wie Mask-RCNN erreichten bereits 2017 eine solide mittlere Genauigkeit (mAP), während neuere Transformer-basierte Ansätze wie ViTDet die Genauigkeit nochmals deutlich steigern konnten [1, 2]. Auch die YOLO-Familie wurde kontinuierlich verbessert: Während frühe Versionen bei mehreren Personen noch Schwierigkeiten hatten, erkennen moderne Varianten Personen sehr schnell und funktionieren auch auf Edge-Geräten wie Smartphones [3]. Für die Gesellschaft ist dies relevant, weil solche Systeme in Videoüberwachung, Zutrittskontrolle, Verkehrsanalyse und Assistenzsystemen eingesetzt werden können. Dennoch bleiben Grenzen bestehen: Kameras erfassen visuelle Daten, sind lichtabhängig und werfen besonders in privaten oder sensiblen Bereichen Datenschutzfragen auf.

LIDAR-Systeme lösen wiederum andere Probleme. Sie ermöglichen eine sehr präzise Distanzmessung und funktionieren auch ohne sichtbares Licht [4]. Dadurch eignen sie sich besonders für Navigationsaufgaben, die Erfassung von Raumgeometrien oder die Detektion von Hindernissen und Personen in Echtzeit. Für die Gesellschaft sind sie relevant, weil sie Sicherheit und Automatisierung unterstützen, etwa in Fahrzeugen, Robotik oder industriellen Umgebungen. Allerdings sind LIDAR-Sensoren kostenintensiver; 3D-Lidar-Systeme liegen preislich deutlich über einfachen Sensorsystemen [5].

Damit zeigt sich: Kamera- und LIDAR-basierte Systeme haben die Personendetektion und Lokalisierung bereits stark vorangebracht. Gleichzeitig bestehen weiterhin Anwendungsfälle, in denen visuelle oder aktive Sensorsysteme an Grenzen stossen. Genau an dieser Stelle setzt WiFi-Sensing als alternative Technologie an.

1.1.1 WiFi-basierte Personendetektion

Die WiFi-basierte Personendetektion ist aus folgenden Gründen sehr interessant. Die Durchdringung der Gesellschaft mit WiFi-Routern ist sehr weit fortgeschritten. Zudem sind WiFi-Router auch sehr erschwinglich und sind bereits ab 13.5 Franken zu erhalten [6]. Neue Entwicklungen in diesem Bereich haben deshalb eine grössere Chance, um die ganze Gesellschaft zu durchdringen und bei jeder sinnvollen Gelegenheit eingesetzt zu werden. Zudem generieren sie, wie LIDAR-Sensoren, keine persönlich sensiblen Daten. Es ist also einfacher, solche Systeme einzusetzen, da weniger rechtliche Hürden zu überwinden sind. Personendetektierung mittels WiFi funktioniert darüber hinaus auch in kompletter Dunkelheit.

Es wurde bereits gezeigt, dass Personendetektion auf Basis von WiFi-Signalen grundsätzlich möglich ist. Basierend auf der Auswertung der Received Signal Strength Indicator-Signale können

Vorhersagen über das Vorhandensein von Personen gemacht werden. Es wurde bereits im Jahre 2007 gezeigt, dass eine Personendetektion in einem Raum auf Basis von RSSI möglich ist [7]. Received Signal Strength Indicator ist ein Indikator dafür, wie stark ein empfangenes Signal auf einem WLAN-Client ankommt. Auch ist es möglich, mittels WiFi-Signalen Personen durch Wände zu detektieren. [8]. Es wird also keine line of sight benötigt, um Personen zu detektieren; diese können auch hinter Möbeln oder anderen Gegenständen sein. Auch haben Studien gezeigt, dass die Messung des Pulses von Personen mittels WiFi-Signalen realisierbar ist.

Diese Kombination aus technischer Machbarkeit, geringer Infrastruktur und Datenschutzvorteilen macht WiFi-Sensing für konkrete Anwendungsfälle besonders attraktiv.

1.1.2 Konkrete Anwendungsfälle

Nachfolgend einige Beispiele, in denen die Personendetektion mit Kameras problematisch ist und in denen WiFi-Sensing eine vielversprechende Alternative darstellt[9]:

1. **Hochsicherheits-Rechenzentren:** In diesen Bereichen muss zwar lückenlos überwacht werden, ob sich unbefugte Personen aufhalten, jedoch dürfen keine Kameras eingesetzt werden. Der Grund: Videomaterial könnte sensible Details über die Server-Architektur, spezifische Verkabelungen oder physische Sicherheitslücken (z. B. Schlosstypen) verraten. WiFi-Sensing detektiert hier die reine Präsenz, ohne Geheimnisse „sichtbar“ zu machen.
2. **Forschungs- und Entwicklungslabore:** In der Industrie (z. B. Automobil- oder Halbleiterentwicklung) herrscht oft ein striktes Fotoverbot, um Wirtschaftsspionage zu verhindern. Dennoch ist eine Personenerkennung für die Arbeitssicherheit (z. B. Abschaltung von Lasern oder Robotern bei Annäherung) essenziell.
3. **Bereiche des privaten Rückzugs (Gesundheit und Pflege):** In Krankenhäusern, Pflegeeinrichtungen oder privaten Badezimmern ist die Überwachung von Patienten zur Sturzerkennung lebenswichtig. Kameras werden hier jedoch als massiver Eingriff in die Intimsphäre empfunden und sind rechtlich schwer umsetzbar. WiFi-Signale bieten hier einen Schutzraum, der Sicherheit ohne Beobachtungsdruck garantiert.
4. **Justizvollzugsanstalten und forensische Kliniken:** In Zellen oder Sanitärbereichen ist eine ständige visuelle Überwachung oft rechtlich eingeschränkt. WiFi-Sensing kann hier helfen, Suizidversuche oder unzulässige Interaktionen diskret zu detektieren, ohne die Menschenwürde durch Videobilder zu verletzen.
5. **Religiöse und spirituelle Stätten:** An Orten des Gebets oder der Meditation werden Kameras oft als störend oder respektlos empfunden. Um dennoch Besucherströme zu messen oder Heizung und Licht effizient zu steuern, eignet sich die unsichtbare WiFi-Detektion ideal.

1.2 Ziele der Arbeit

1. Die Lokalisierung von einer Person aus WiFi-Signalen
2. Die Lokalisierung soll dabei Koordinaten von Körperstellen vorhersagen
3. Die Lokalisierung soll in 3D erfolgen

4. Die Personenlokalisierung soll für einen bestimmten Raum erfolgen

1.3 Aufgaben

1. Wissen aneignen und Konzept erstellen

- Grundlegendes Verständnis der WiFi-Technologie und der Channel State Information (CSI) erlangen
- Literaturrecherche zu aktuellen Ansätzen der Personendetektion mit WiFi-Signalen durchführen
- Ansatz und Konzept für die Personendetektion mit WiFi-Signalen definieren

2. Trainingsdaten generieren

- Einrichten des WiFi-Setups
- Erstellen eines Skripts um Daten zu sammeln
- Erstellen eines Skripts um Trainingsdaten zu generieren

3. Machine Learning Modell trainieren

- Auswahl geeigneter Plattform für das Training

4. Machine Learning Modell evaluieren

- Evaluierung des Modells auf einem Testdatensatz
- Vergleich der Ergebnisse mit anderen Ansätzen

2. Stand der Forschung

2.1 WiFi Technologie

In diesem Kapitel wird eine Einführung in die WiFi Technologie gegeben, um die Grundlagen zu verstehen, welche für die Personendetektion mit WiFi-Signalen relevant sind. Es wird dabei auf die verschiedenen WiFi-Standards eingegangen, wie Daten über WiFi gesendet und empfangen werden. Zudem wird auf einige interessante Studien im CSI (Channel State Information) Forschungsfeld eingegangen, um einen Überblick über den aktuellen Stand der Forschung zu geben.

2.1.1 WiFi Standards

WiFi-Systeme senden und empfangen Signale auf Basis von elektromagnetischen Wellen. Dabei gibt es verschiedene Standards, welche auf unterschiedlichen Frequenzbändern senden. Dies bedeutet nichts anderes als dass die elektromagnetischen Wellen, welche von einem Router gesendet werden, innerhalb dieses Frequenzbandes liegen. Ein Router, welcher auf dem 2.4 GHz-Band sendet, produziert also elektromagnetische Wellen mit einer Frequenz zwischen 2,400 GHz und 2,4835 GHz. Die Frequenz gibt dabei einfach an, wie viele Schwingungen pro Sekunde die elektromagnetischen Wellen haben. Je höher die Frequenz, desto mehr Schwingungen pro Sekunde. Eine Schwingung ist dabei eine komplette Welle, also von einem Peak zum nächsten Peak. Je mehr Schwingungen pro Sekunde, desto mehr Informationen können theoretisch übertragen werden. Deshalb können mit einem 5 GHz-Router theoretisch mehr Informationen übertragen werden als mit einem 2.4 GHz-Router, da die Frequenz höher ist und somit mehr Schwingungen pro Sekunde möglich sind. Dabei gibt es aber auch andere Standards wie z.B. 5 GHz oder 6 GHz, welche auf anderen Frequenzbändern senden und somit auch andere Frequenzen haben [10].

Innerhalb dieser Frequenzbänder gibt es dann aber auch noch verschiedene Kanäle, welche von einem Router genutzt werden können, um Daten zu senden. Bei einem 2.4 GHz-Router gibt es 14 vom Router nutzbare Kanäle. In der Praxis werden oft nur die Kanäle 1, 6 und 14 genutzt, da sich diese nicht überlappen. Wenn andere Kanäle genutzt werden, überlappen sich die elektromagnetischen Wellen von anderen Routern, sodass das Signal gestört wird und die Datenübertragung nicht mehr zuverlässig funktioniert. Ein Router entscheidet sich beim Booten und nach einem Scanvorgang der Umgebung für einen Kanal. Alle seine verbundenen Geräte nutzen dann nur diesen Kanal, um Daten zu senden und zu empfangen.

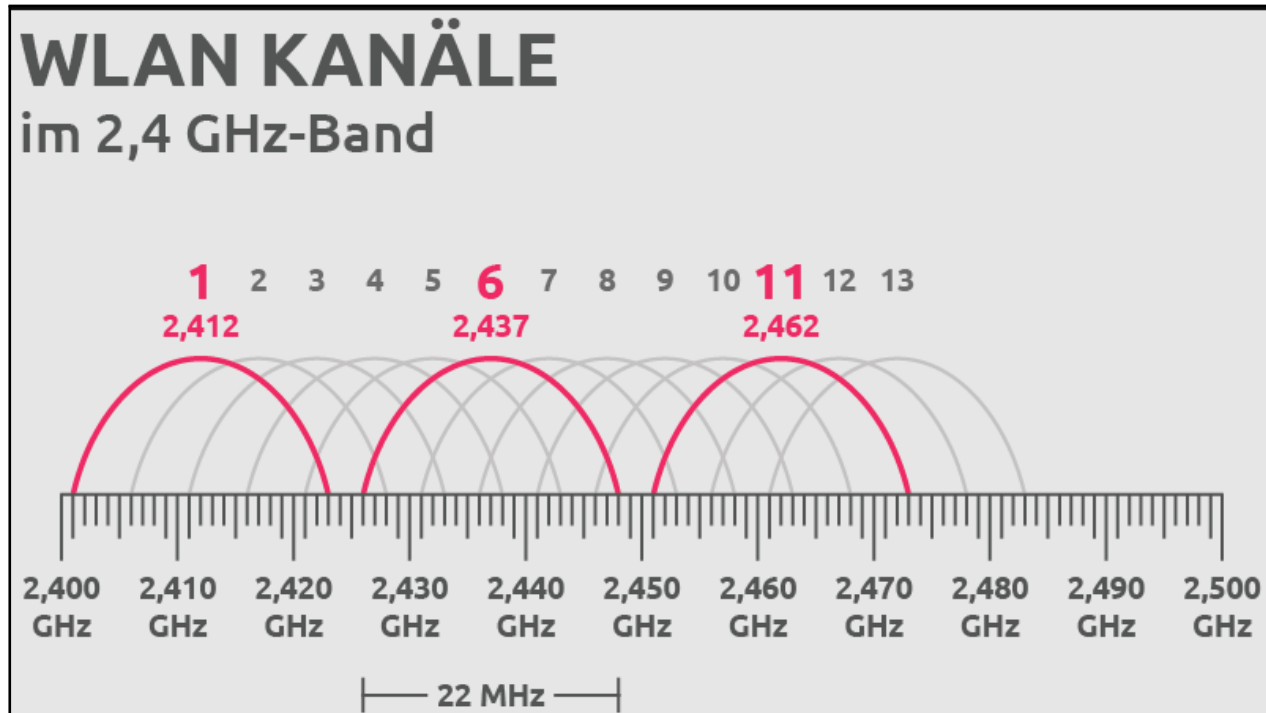


Abbildung 2.1: Die Grafik zeigt das Frequenzband eines 2,4 GHz-Routers, welches von 2,400 GHz bis 2,4835 GHz reicht. Innerhalb dieses Frequenzbandes gibt es 14 verschiedene Kanäle, welche von einem Router genutzt werden können, um Daten zu senden. In der Praxis werden oft nur die Kanäle 1, 6 und 14 genutzt, da sich diese nicht überlappen.

Innerhalb dieser Kanäle gibt es dann zusätzlich noch Subchannels, welche von einem Router genutzt werden können, um Daten zu senden. Je nach Standard und Kanalbreite existieren zwischen 32 und 56 Subchannels, welche von einem Router genutzt werden, um Daten zu senden und zu empfangen [10, 11]. Die Subchannels sind dabei einfach eine weitere Unterteilung des Kanals in Unterfrequenzen. Um die Daten von einem Subchannel zu senden oder empfangen, wird nur die elektromagnetische Welle mit der Frequenz dieses spezifischen Subchannels genutzt. Dabei werden parallel mehrere Subchannels genutzt um die Datenrate zu erhöhen. Pro Subchannel wird dabei ein Symbol gesendet, welches dann vom Empfänger gemessen und rekonstruiert wird. Je mehr Subchannels genutzt werden, desto mehr Symbole können parallel gesendet werden und desto höher ist die Datenrate [12]. Ein Symbol ist dabei die kleinste Form der Information. Ein Symbol kann dabei ein oder mehrere Bits repräsentieren. Je nachdem wie gut die Verbindung zwischen Sender und Empfänger ist, können mehr oder weniger Bits pro Symbol übertragen werden. Deshalb ist es wichtig, dass Sender und Empfänger die Qualität der Verbindung messen und dann entscheiden, wie viele Bits pro Symbol übertragen werden sollen. Dies ist der Grund, warum die Geschwindigkeit der Datenübertragung bei grosser Entfernung zu einem Router oder bei vielen Hindernissen zwischen Sender und Empfänger sinkt, da die Qualität der Verbindung schlechter wird und somit weniger Bits pro Symbol übertragen werden können.

2.1.2 Symbolversendung und Symbolreplikation

QAM und OFDM

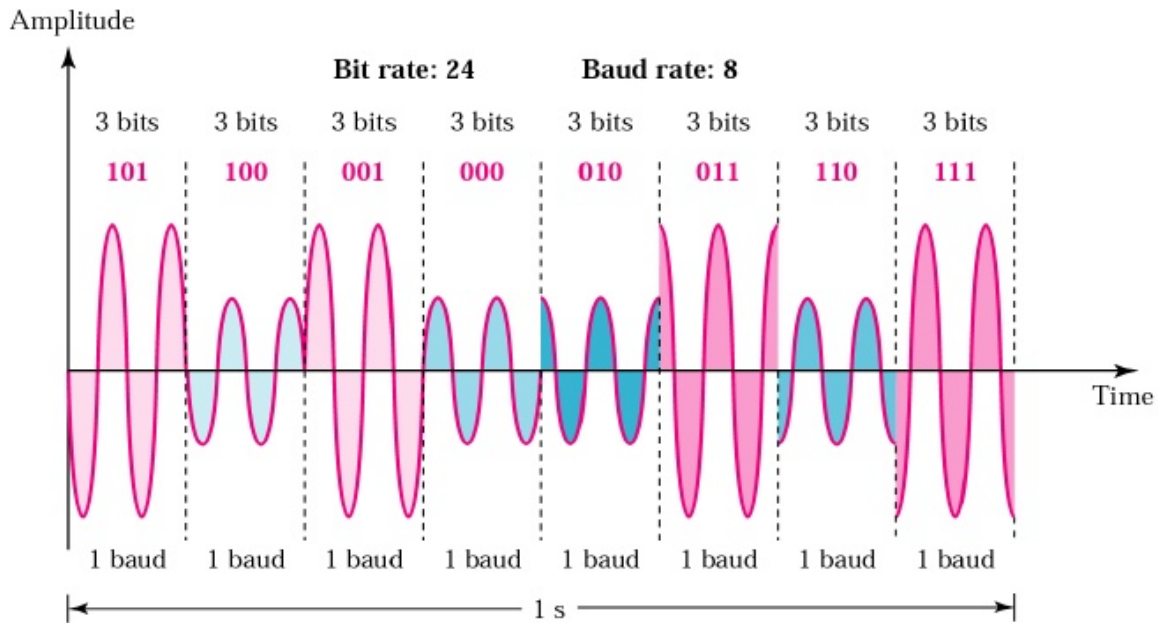
Die Quadraturamplitudenmodulation (QAM) ermöglicht es, mehrere Bits in einem Symbol zu kodieren, indem Amplitude und Phase der Trägerwelle gemeinsam verändert werden. Ein Symbol ist dabei ein definierter Zustandswert der Modulation und repräsentiert je nach Modulationsordnung mehrere Bits. Orthogonal Frequency Division Multiplexing (OFDM) ist ein Übertragungsverfahren, bei dem das Signal auf viele orthogonale Subcarrier aufgeteilt wird, sodass diese parallel und ohne gegenseitige Störung übertragen werden können. Auf jedem Subcarrier wird dabei ein QAM-Symbol übertragen.

Aus den Signalen aller Subcarrier bildet der Sender ein gemeinsames Zeitsignal. Dieses Zeitsignal ist das OFDM-Signal und stellt die physikalische Welle dar, die über die Antenne ausgesendet wird. Ein einzelnes OFDM-Signal kann also viele Daten parallel transportieren, und zwar die einzelnen Bit-Werte der QAM-Symbole.

Im folgenden Beispiel wird zur Veranschaulichung 8-QAM betrachtet. Dabei stehen acht mögliche Symbole zur Verfügung, also genau 2^3 Symbolzustände für drei Bits pro Symbol. In realen Wi-Fi-Systemen werden je nach Modulationsverfahren auch 16-QAM, 64-QAM oder 256-QAM eingesetzt, um mehr Bits pro Symbol zu übertragen. Während eines OFDM-Symbols werden die QAM-Symbole aller Subcarrier gleichzeitig gesendet und vom Empfänger anschließend getrennt ausgewertet.

Im Empfänger wird das empfangene OFDM-Signal nach der Demodulation wieder in seine Subcarrier-Komponenten zerlegt. Aus Betrag und Phase dieser komplexen Werte lassen sich die ursprünglichen QAM-Symbole pro Subcarrier rekonstruieren. Die gemessene Amplitude und Phase der Kanalantwort pro Subcarrier bilden die Grundlage für CSI-Messungen, die im weiteren Verlauf dieser Arbeit für die Personendetektion genutzt werden.

Figure 5.15 Time domain for an 8-QAM signal



McGraw-Hill

©The McGraw-Hill Companies, Inc., 2004

Abbildung 2.2: Die Grafik zeigt die 8 verschiedenen Symbole, welche bei 8-QAM gesendet werden können. Jedes Symbol repräsentiert eine Kombination von 3 Bits. In der Grafik sieht man wie die Symbole sich in Amplitude und Phase unterscheiden.

Symbolversendung

Das Produzieren eines Symbols durch den Senderrouter erfolgt durch das Anlegen einer periodisch wechselnden Spannung auf der Antenne. Dadurch wird ein zeitlich veränderliches elektromagnetisches Feld erzeugt, dessen Form durch die gewählte Modulation festgelegt wird. Das Symbol ist hier im zeitlichen Sinn als OFDM-Symbol zu verstehen. Ein OFDM-Symbol entspricht dabei einem endlichen Zeitintervall, in welchem auf jedem Subcarrier ein QAM-Symbol gesendet wird. Die Dauer eines Symbols wird mit T_s bezeichnet. Innerhalb eines solchen OFDM-Symbols können mehrere Schwingungen der Trägerwelle stattfinden, bevor das nächste Symbol übertragen wird.

Für die spätere Analyse von CSI ist besonders relevant, dass sich bereits kleine Änderungen in der Umgebung auf die ausgesendete Welle auswirken. Wird ein Symbol durch Reflexionen, Absorption oder Abschattung verändert, kann der Empfänger diese Abweichung in Amplitude und Phase messen und daraus Rückschlüsse auf den Übertragungskanal ziehen.

Demodulation und Kanalverzerrung

Die exakte Messung von Amplitude und Phase ist eine Herausforderung, da äußere Einflüsse wie Hindernisse, Reflexionen oder andere Funkgeräte die elektromagnetischen Wellen verzerren. Um diese Effekte zu kompensieren, enthält jedes WLAN-Paket zu Beginn eine Präambel mit einem bekannten Referenzsignal.

Der Empfänger nutzt die kohärente Demodulation, also die Multiplikation des Eingangssignals mit einem lokalen Referenz-Oszillator, um das ankommende Signal in den Basisband-Bereich zu verschieben und messbar zu machen. Nach der Demodulation liegt das Signal als komplexer I/Q-Wert vor. Aus Betrag und Phase dieser komplexen Darstellung lassen sich Amplitude und Phasenlage ableiten.

Im ersten Schritt der Kanalschätzung vergleicht der Empfänger das demodulierte Referenzsignal mit einer bekannten Referenz und bestimmt daraus für jeden Subcarrier einen Korrekturfaktor H . Dieser Faktor beschreibt die spezifische Dämpfung und Phasenverschiebung des Kanals. Anschließend kann der nachfolgende Payload entzerrt werden:

$$Y_{\text{bereinigt}} = \frac{Y_{\text{empfangen}}}{H}$$

Durch diese Kanalverzerrung werden Störungen mathematisch herausgerechnet, sodass die ursprünglichen Symbolinformationen näherungsweise rekonstruiert werden können. Für diese Arbeit ist dieser Schritt wichtig, weil die geschätzte Kanalantwort pro Subcarrier die Grundlage für die CSI-Messung bildet.

2.2 Aktueller Stand des CSI Forschungsfeldes

Im Folgenden werden drei Forschungsgebiete im CSI Forschungsfeld anhand von jeweils einer Studie vorgestellt. Mit CSI ist dabei Channel-State-Information gemeint. Also die Phase und Amplitude Information, welche auf einem Empfängergerät ausgelesen werden kann. Das Auslesen und systematische Analysieren dieser Informationen ermöglicht es, Rückschlüsse auf die Umgebung zu ziehen, in der sich die elektromagnetischen Wellen bewegen. Die nachfolgenden Studien bzw. Forschungsbereiche wurden ausgewählt, da sie in den letzten Jahren starke Fortschritte gemacht haben und interessante Umsetzungsmöglichkeiten für Anwendungen besitzen. Zuerst wird ein Ansatz vorgestellt, welcher nicht direkt den Use Case meiner Arbeit abdeckt. Danach werden zwei Ansätze vorgestellt, welche sich mit der Personendetektion beschäftigen.

2.2.1 PulseFi - A Low Cost Robust Machine Learning System for Accurate Cardiopulmonary and Apnea Monitoring Using Channel State Information

Ein besonders anspruchsvolles Feld der Wi-Fi-basierten Sensorik ist die Erfassung vitaler Parameter wie Herzfrequenz, Atemfrequenz und Schlafapnoe. Da Herzschläge im Vergleich zur Atmung nur extrem geringfügige mechanische Deformationen am Brustkorb verursachen, ist das Signal-Rausch-Verhältnis bei herkömmlichen Wi-Fi-Messungen oft sehr gering. Zudem setzen viele bestehende Systeme auf teure Mehrantennen-Hardware und Phaseninformationen, die auf günstigen Ein-Antennen-Geräten kaum zuverlässig auslesbar sind.

Kocheta et al. (2025) präsentieren mit *PulseFi* ein System, das diese Einschränkungen adressiert und kontaktloses Monitoring mit günstiger Standardhardware ermöglicht [13]. Im Gegensatz zu vielen bisherigen Ansätzen verzichtet PulseFi bewusst auf Phaseninformationen und nutzt ausschliesslich die Amplitude der CSI-Signale. Damit genügt eine einzelne Antenne pro Gerät, was den Einsatz von ESP32- oder Raspberry-Pi-Hardware ermöglicht. Die verarbeiteten Signale werden anschliessend in ein kompaktes LSTM-Netzwerk eingespeist, das Herzfrequenz und Atemfrequenz regrediert sowie Apnoe-Ereignisse klassifiziert.

Die Signalverarbeitung erfolgt dabei in mehreren Schritten:

1. **Amplitudenextraktion:** Aus den komplexen CSI-Werten wird der Betrag berechnet, da nur die Amplitude der Signale für die Vitalzeichen verwendet wird.
2. **Rauschunterdrückung:** Die stationäre Gleichspannungskomponente wird entfernt, um dynamische Vitalzeichen von Hardware- und Umgebungsrauschen zu trennen.
3. **Bandpassfilterung:** Ein Butterworth-Filter isoliert die für Herzfrequenz (0,8 bis 2,17 Hz) bzw. Atemfrequenz (0,1 bis 0,5 Hz) typischen Frequenzbänder. Es wird also der Amplitudenwert für jeden Subchannel über die Zeit geplottet und dann nur die Signalverläufe betrachtet, welche die oben erwähnten Frequenzen aufweisen. Alle anderen Frequenzen werden herausgefiltert, da sie nicht relevant für die Vitalzeichen sind.
4. **Glättung und Segmentierung:** Ein Savitzky-Golay-Filter reduziert Restrauschen, anschliessend werden überlappende Zeitfenster gebildet und normalisiert, bevor sie dem LSTM-Modell zugeführt werden.

Das LSTM besteht aus drei aufeinanderfolgenden Blöcken mit 64, 32 und 16 Neuronen und liefert entweder eine kontinuierliche Schätzung der Vitalparameter oder – im Apnoe-Fall – eine binäre Klassifikation über eine Sigmoid-Aktivierung.

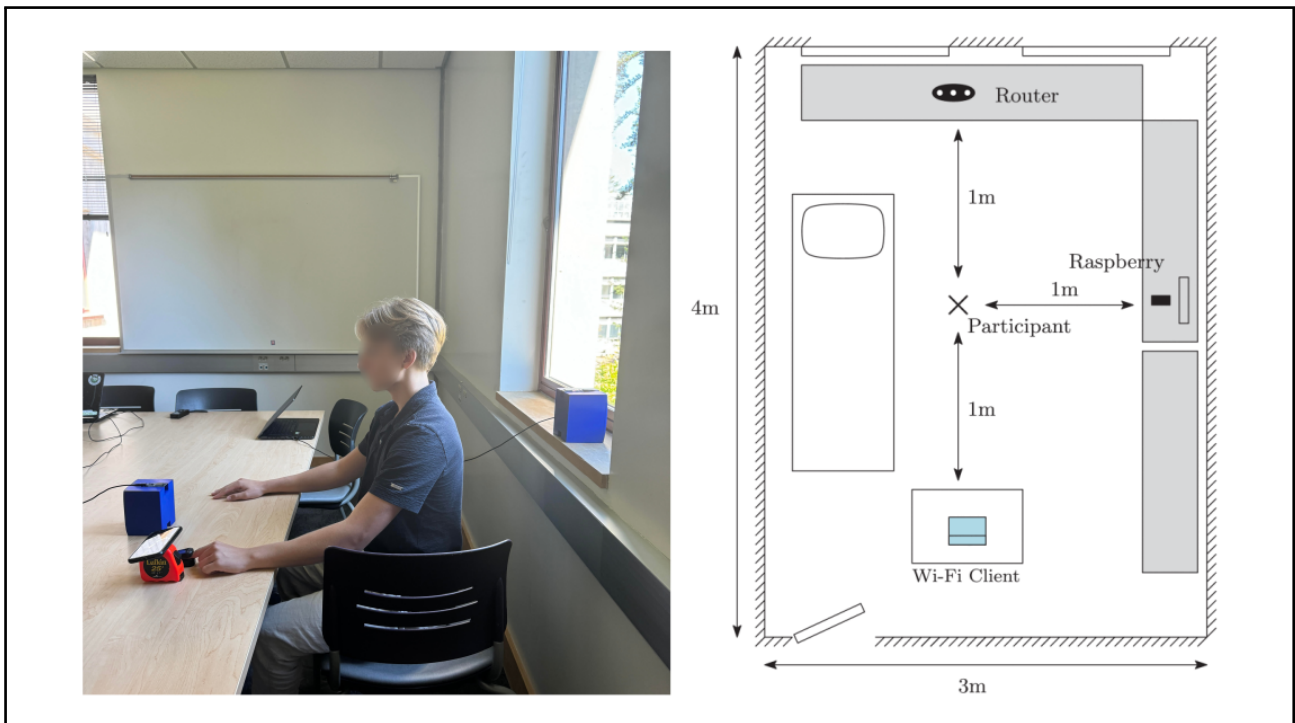


Abbildung 2.3: Setup der Experiment Area von PulseFi. Es wurde dabei in Wi-Fi-Client, eine Antenne mit einem Raspberry Pi und einen Router in der beschriebenen Anordnung genutzt.

Zur Evaluation nutzen Kocheta et al. zwei Datensätze: einen lokal mit ESP32-Geräten erhobenen Datensatz mit sieben Probanden sowie den E-Health-Datensatz mit 118 Probanden und 17 Positionen bzw. Aktivitäten, der als einer der umfangreichsten Datensätze dieser Art gilt. Auf dem ESP32-Datensatz erreicht PulseFi bei 5-Sekunden-Fenstern einen mittleren absoluten Fehler von 0,50 bpm bei der Herzfrequenz und 0,09 Atemzügen/min bei 20-Sekunden-Fenstern für die Atemfrequenz. Auf dem E-Health-Datensatz sinkt der Herzfrequenzfehler bei 30-Sekunden-Fenstern auf 0,17 bpm. Für die Apnoe-Erkennung erzielt das System 99,4 % Genauigkeit bei 95,7 % Sensitivität. Damit zeigt PulseFi, dass präzises Vitalzeichen-Monitoring auch ohne teure Mehrantennen-Setups und ohne Phaseninformation möglich ist. Der Use Case unterscheidet sich damit klar von der Pose-Schätzung, liefert aber ein eindrückliches Beispiel dafür, welche Feinstrukturen sich aus CSI-Amplituden extrahieren lassen.

2.2.2 Person-in-WiFi 3D - End-to-End Multi-Person 3D Pose Estimation with Wi-Fi

2024 haben Yan et al. eine Studie veröffentlicht, welche zeigt, dass die Transformer Architektur für die Personenlokalisierung im Raum auf Basis von WiFi-Signalen sehr gut geeignet ist [14, 15]. Sie haben dabei die Channel State Information (CSI) Signale von WiFi-Routern ausgewertet, um die Position von Personen in einem Raum vorherzusagen. Es werden dabei mehrere Personen in einem Raum erkannt und für jede dieser Personen wird ein Set an Landmarks vorhergesagt. Mit einem Landmark ist ein Gelenk einer Person bzw. eine Körperstelle gemeint, z.B. die Schulter, der Ellbogen oder das Knie. Yan et al. haben dabei ein Setup mit einem Transmitter und drei Receiver Geräten verwendet. Auf drei Geräten, werden also die CSI-Signale ausgelesen und dann für das Trainieren das Transformer Modell gebraucht. Der Transmitter sendet dabei kontinuierlich Pakete an alle Receiver. In dieser Studie werden 30 Subchannel und eine Frequenz von 5.64GHz verwendet um die Pakete zu versenden. Sie nutzen dabei auch eine Azure Kinect Kamera, welche die Ground Truth Daten der Landmarks generiert. Der Detektionsbereich beschränkt sich dabei auf ein Rechteck von 4m auf 3.5m.

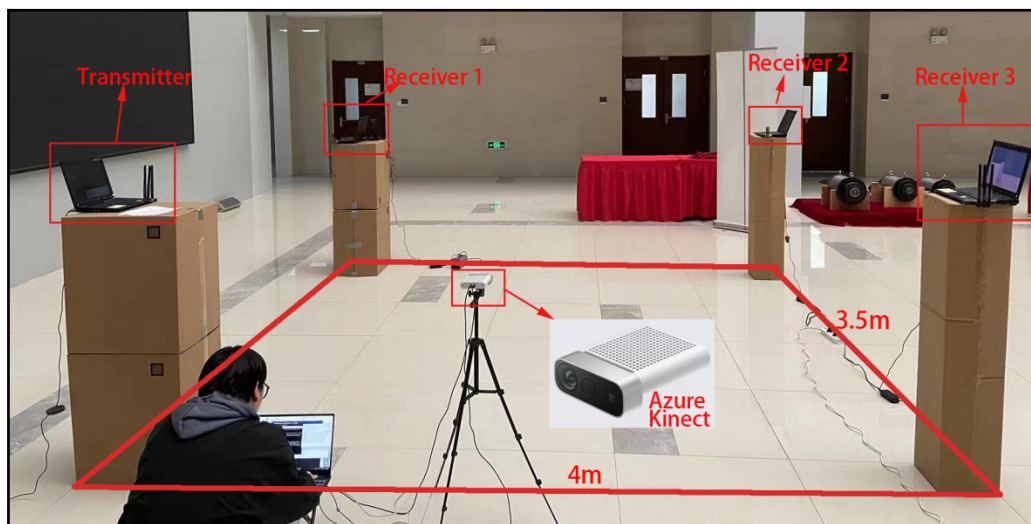


Abbildung 2.4: Setup der Experiment Area von Yan et al. mit einem Transmitter, drei Receiver-Geräten und einer Azure Kinect Kamera zur Generierung der Ground-Truth-Daten. Quelle: [14]

Das Transformer Modell basiert dabei auf einem Encoder, welcher die CSI-Signale in den

latentem Raum überführt und zwei Decoder, welche schlussendlich die Landmarks vorhersagen. [14]. Yan et al. haben gezeigt, dass sie bessere Accuracy Werte erreichen, als andere Ansätze, welche nicht die Transformer Architektur verwenden haben [16], [17]. Die Spezialität dieser Studie ist, dass mit der verwendeten Architektur mehrere Personen lokalisiert werden können. Die verwendete Architektur wird in diesem Kapitel nur kurz erklärt, da später noch genauer auf diese eingegangen wird. Yan verwenden einen Encoder mit lernbaren STE-Embeddings. Zusätzlich verwenden sie zwei Decoder Blöcke, welche nacheinander eingesetzt werden. Einen Pose Decoder, welcher grobe Posen schätzt und einen Refine Decoder, welcher diese dann verfeinert. Als Loss Funktion wird ein Hungarian Loss eingesetzt, welcher jeder Ground Truth Pose den bestmöglichen Kandidaten zuweist. Yan zeigen also

2.2.3 Breaking Coordinate Overfitting: Geometry-Aware WiFi Sensing for Cross-Layout 3D Pose Estimation

Ein zentrales Problem vieler WiFi-basierter 3D-Pose-Schätzungsverfahren ist die schlechte Generalisierung auf neue Umgebungen. Modelle wie Person-in-WiFi 3D erreichen zwar hohe Genauigkeit, solange Training und Test im selben Raum mit identischem Geräte-Setup stattfinden, brechen aber stark ein, sobald sich die Position der Transmitter- und Receiver-Geräte ändert [14].

Jia et al. (2026) identifizieren hierfür die Ursache in einem Phänomen, das sie *Coordinate Overfitting* nennen [18]. Bisherige Ansätze trainieren ein Modell, das CSI-Signale direkt auf 3D-Skelettkoordinaten aus einem kamerabasierten Referenzsystem abbildet, welches Jia et al. ändern.

Zur Lösung dieses Problems stellen Jia et al. das Framework *PerceptAlign* vor. Es besteht aus zwei Hauptkomponenten:

1. **Lightweight Coordinate Unification:** Die Kamera- und Routerpositionen werden in ein gemeinsames 3D-Koordinatensystem überführt. Dadurch liegen die WiFi-Sensorik und die visuelle Ground Truth nicht in getrennten Bezugssystemen vor, sondern in derselben räumlichen Referenz.
2. **Geometry-Conditioned Learning:** Die kalibrierten Transceiver-Koordinaten werden als hochdimensionale räumliche Embeddings kodiert und mit den CSI-Features fusioniert. Das Modell erhält damit explizit die Gerätegeometrie als Bedingung und muss diese nicht mehr aus den Signalen erraten. So wird erzwungen, dass der Netzwerkanteil für menschliche Bewegung von layout-spezifischen Artefakten getrennt wird.

Für die Datenerfassung verwenden Jia et al. ein Setup mit einem Transmitter und drei Receivern auf Basis von Intel-5300-Netzwerkkarten bei 5,2 GHz und 57 Subcarriern. Die Ground-Truth-Posen werden mit EasyMocap aus Intel RealSense D435 RGB-D-Kameras generiert. Die Signalverarbeitung erfolgt dabei in mehreren Schritten:

1. **Preprocessing:** Messen von Phasen- und Amplitudeninformationen sowie Berechnen der Doppler-Frequenzverschiebung; anschliessend wird der Datenstrom zeitlich segmentiert und mit den Videoframes synchronisiert. Die Doppler-Frequenz wird aus der zeitlichen Änderung der CSI-Phase berechnet. Aus jedem Segment werden die Magnitude, die absolute Phase sowie die Doppler-Frequenzverschiebung extrahiert und zu einer Merkmalsmatrix konkateniert. Diese Matrix wird für jeden Receiver auf eine feste Größe von 224×224 Elementen skaliert und identisch auf drei Kanäle dupliziert, sodass pro Empfänger ein

standardisierter $3 \times 224 \times 224$ Tensor als Eingang für die nachfolgende Feature-Extraktion entsteht. Aus der Doppler-Frequenz wird die kontinuierliche Veränderung der Bewegungsgeschwindigkeit und -richtung einzelner Körperteile relativ zu den Antennen gewonnen, was dem Modell die für die 3D-Rekonstruktion essenziellen dynamischen Bewegungsmuster liefert.

2. **Feature-Extraktion:** Ein geteilter ResNet-34-Encoder verarbeitet die CSI-Tensoren aller Receiver und erzeugt pro Frame und Empfänger einen Feature-Vektor.
3. **Geometrie-Fusion:** Die kalibrierten Receiver-Positionen werden über sinusoidale Positionskodierung und ein MLP in räumliche Embeddings überführt, mit den CSI-Features sowie lernbaren Zeit- und Receiver-Embeddings zu Tokens zusammengeführt.
4. **Pose-Vorhersage:** Ein spatio-temporaler Transformer-Encoder mit sechs Layern verarbeitet die Tokens, ein leichtgewichtiger MLP-Decoder regrediert die 3D-Gelenkkoordinaten. Als Loss-Funktion wird MSE zwischen vorhergesagten und Ground-Truth-Koordinaten verwendet.

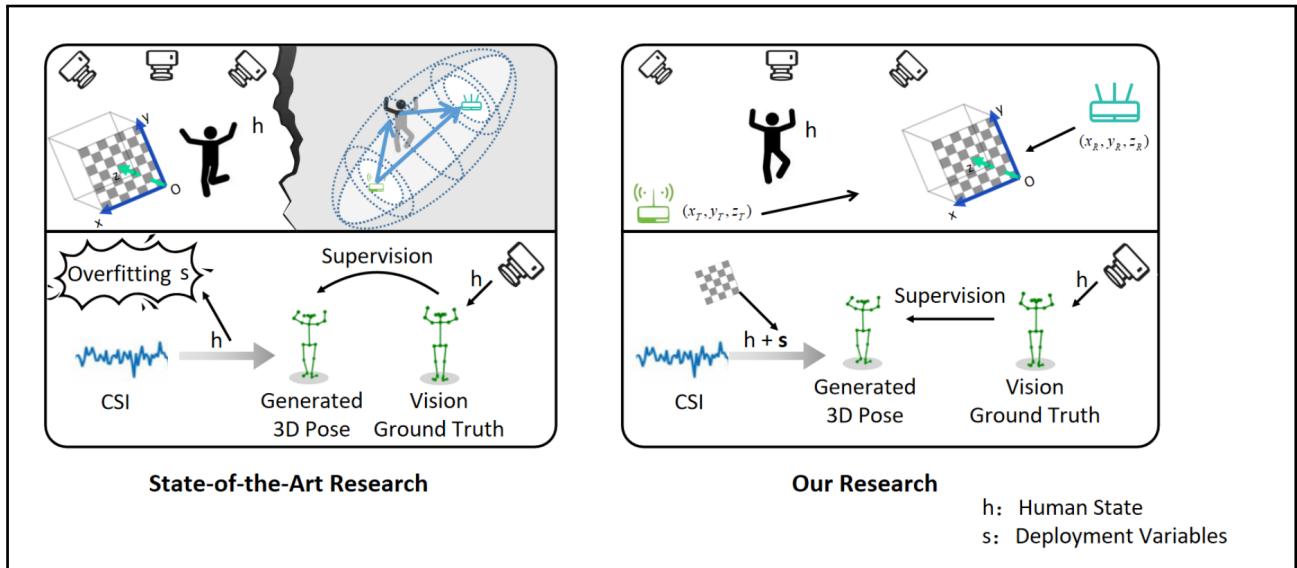


Abbildung 2.5: Links den State-of-the-Art-Ansatz ohne Routerkoordinaten und mit Overfitting auf die Routerpositionen. Rechts der Geometry-Aware-Ansatz mit Routerkoordinate und globalem Koordinatensystem.

Zur Evaluation stellen Jia et al. zudem den bislang grössten Cross-Domain-Datensatz für WiFi-basierte 3D-Pose-Schätzung bereit: 21 Probanden, fünf Szenen, 18 Aktionen und sieben unterschiedliche Geräte-Layouts mit insgesamt über 7,2 Millionen synchronisierten Frames. Im direkten Vergleich mit Person-in-WiFi 3D zeigt PerceptAlign deutliche Vorteile. Entscheidend ist dabei die Cross-Layout-Generalisierung: Während Person-in-WiFi 3D bei geänderter Geräteaufstellung einen MPJPE von 649,3 mm erreicht und praktisch unbrauchbar wird, hält PerceptAlign den Fehler bei 170,2 mm und reduziert den Cross-Domain-Fehler insgesamt um mehr als 60 %. Damit adressiert diese Studie direkt die Frage, wie WiFi-basierte Pose-Schätzung auf neue Räume übertragbar gemacht werden kann.

3. Ideen und Konzepte

3.1 Architektur des Modell von Yan

Als erste Architektur für die Arbeit soll diejenige aus der Studie von Yan verwendet werden. Das Code-Repository von Yan ist öffentlich und konnte deshalb direkt verwendet werden. Auch wenn die Ground-Truth Daten ebenfalls öffentlich sind, konnten diese aus Kompatibilitätsgründen nicht verwendet werden, da eine unterschiedliche Anzahl an Subcarrier verwendet wurden. Zudem veröffentlichten Yan auch keine trainierten Gewichte. Deshalb wurde lediglich die Architektur des Modells von Yan übernommen. Die Architektur welche auch das Lokalisieren von mehreren Personen ermöglicht wird in den folgenden Abschnitten erläutert und erklärt.

3.1.1 Tokenisierung und Embedding

Um Daten für einen Transformer kompatibel zu machen, müssen diese zuerst tokenisiert werden. Ein Token eines Large-Language-Transformers (LLM) ist dabei ein Wort bzw. ein Teil eines Wortes, welches in eine höhere mathematische Dimension transformiert wird. Bei einem Visiontransformer, welcher ursprünglich für die Bilderkennung mittels Transformerarchitektur genutzt wurde, entspricht ein Token einem kleinen Teil des ganzen Bildes. Yan haben in ihrer Studie ebenfalls die Architektur eines Vision-Transformers verwendet. Ein Token in dieser Architektur entspricht dabei den Phasen- und Amplitudeninformationen eines einzigen CSI-Pakets. Ein Paket, wie es in Yan verwendet wurde, hat dabei 30 Subchannels mit je einem Phasen- und einem Amplitudenwert. Für einen einzigen Durchlauf dieser Tokens muss eine definierte Anzahl vorhanden sein. Nur wenn genau diese definierte Anzahl an Tokens vorhanden sind, lässt sich ein korrekter Forward-Pass mit Resultat produzieren. Ein Token besteht hierbei aus Anzahl Transmitter, Anzahl Receiver, Anzahl Antennen und Anzahl Zeitschritte. Im Setup werden also 1 Transmitter, 3 Receiver, 3 Antennen pro Receiver und 20 Zeitschritte verwendet. Die Anzahl Zeitschritte bezieht sich hier auf die Anzahl ganzer CSI-Pakete, welche für einen Durchlauf benötigt werden. Dies ergibt insgesamt $(1 \times 3 \times 3 \times 20)$ 180 Tokens, mit einer jeweiligen Dimension $\in 1 \times 60$. Ein Token ist also ein Vektor mit einer Dimension $\in 1 \times 60$. Die Dimension des Vektors setzt sich aus zweimal $\in 1 \times 30$ zusammen, welche dann in denselben Vektor konkateniert werden. Das Setup von Yan et al. besitzt 30 verschiedene Subchannels, was zu 30 Amplituden und 30 Phasenwerten führt.

Ein fully-connected Layer wird dann dazu genutzt, um alle 180 Tokens in eine höhere Dimension von $\in 1 \times 256$ zu transformieren. Nach dieser Transformation werden zu jedem Vektor lernbare, sogenannte Temporal-Spatial Embeddings“ hinzuaddiert. Die Addition dieser Embeddings soll sicherstellen, dass eine korrekte Repräsentation von Ort und zeitlicher Einordnung der einzelnen Tokens dem Vektor hinzuaddiert werden kann. Ein Vektor bzw. CSI-Paket, welches zeitlich vor den anderen kommt, sollte also einen anderen Embedding-Vektor hinzuaddiert bekommen als spätere Pakete. Auch bei einem Vektor, welcher von Router 1, Antenne 1 produziert worden ist, erhält einen anderen Embedding Vektor als Pakete von anderen Routern und Antennen. Ein Embedding hat also die Funktion, die zeitlichen und örtlichen Informationen dem Vektor hinzuzufügen. Die Dimension des Inputs in den Hauptteil des Transformers ist also $\in 180 \times 256$

3.1.2 Encoder

In einem Encoder eines Transformers werden Gewichte gelernt, welche die Wichtigkeit zwischen den Elementen des Eingabe-Vektors $\in 1 \times 256$ modelliert. Dadurch kann das Modell lernen, wie die einzelnen Subchannelinformationen zueinander in Beziehung stehen. Der Encoder besteht in dieser Architektur aus 6 Schichten mit jeweils einem Multihead-Self-Attention und einem Feed-forward Block. Dies sind fundamentale Komponenten der Transformer Architektur [19]. Nach den 6 Schichten des Encoders besitzt die Architektur einen ersten Head, welcher ein erstes Set an Positionen ausgibt P_{init} mit Wahrscheinlichkeiten S_{init} . Dafür wird ein Multi-Layer Perceptron benutzt, welches 180 3D Landmark Sets produziert. Ein Landmark besteht dabei aus 14 Körperstellen. Der Output ist also $P_{init} \in 180 \times (3 \times 14)$ und $S_{init} \in 180 \times 1$. Der Output dieser Heads fließt dann in die Loss-Funktion ein.

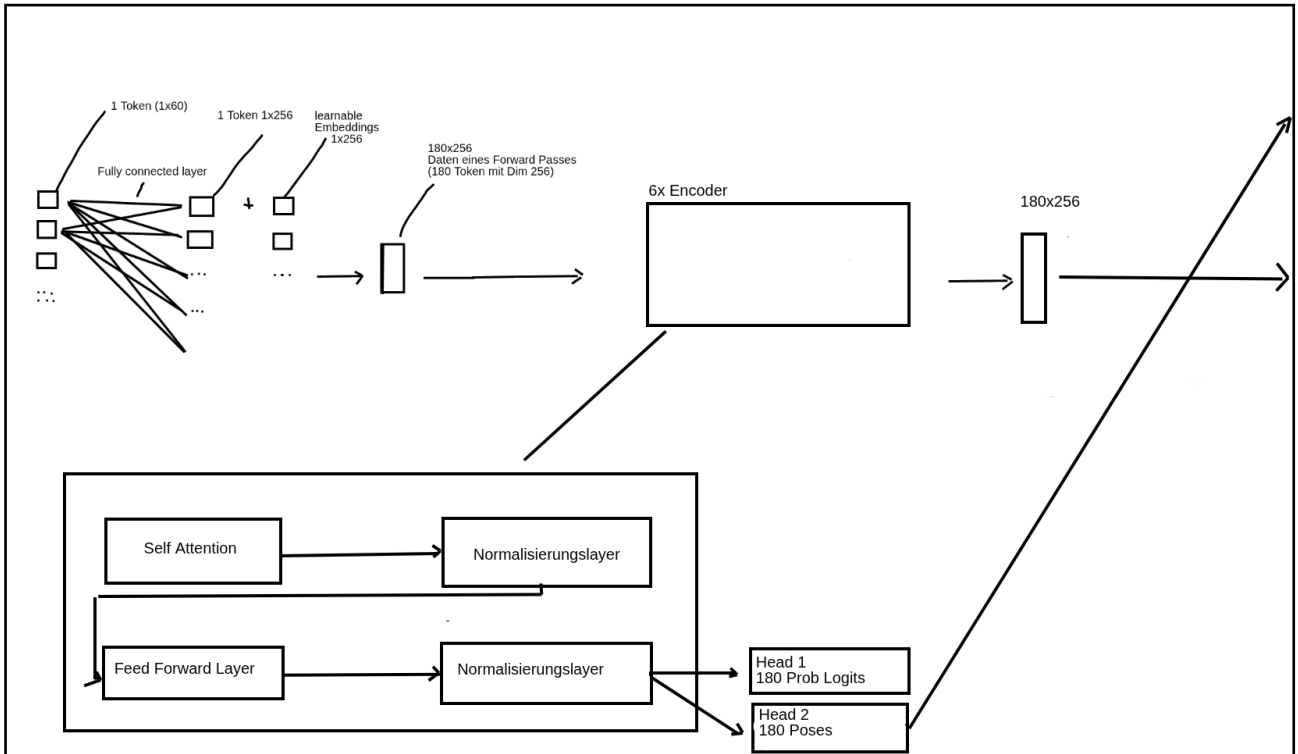


Abbildung 3.1: Skizze der Tokenisierung und des Encoders. Links oben sieht man die Projektion in den Embedding-Raum. Die 6 Encoder-Blöcke werden nochmals genauer als “Zoom in” dargestellt. Es werden jeweils die Dimensionen der einzelnen Vektoren bzw. Tensoren beschrieben. Die Pfeile am rechten oberen Bildrand sind so zu verstehen, dass sie direkt in Abbildung 3.2 einfließen und dort als Inputs wieder in den Fluss aufgenommen werden.

3.1.3 Pose Decoder

Im ersten Decoder der Architektur wird mithilfe der im Encoder produzierten Koordinaten und den Tokens die Positionen weiter verbessert. Dabei werden 3 DETR Blöcke nach [15] sowie [20] verwendet. Hierbei fließen die 100 besten Posen aus dem Encoder und die Tokens in den DETR Block. Pro Schicht wird dabei eine offset Prediction produziert und zu den Koordinaten der vorherigen Schicht hinzuaddiert. Dies ermöglicht die Restriktierung einer einzelnen Schicht, sodass der Output einer einzelnen Schicht die Pose nicht alleine extrem stark verschieben kann, sondern dass dies iterativ über viele Schichten hinweg passiert. Dies lässt sich wie

folgt darstellen:

$$P_d = P_{d-1} + \Delta P_d$$

wobei P_d die neue Pose, P_{d-1} die vorherige Pose und ΔP_d die von einer Schicht produzierte Verschiebung ist.

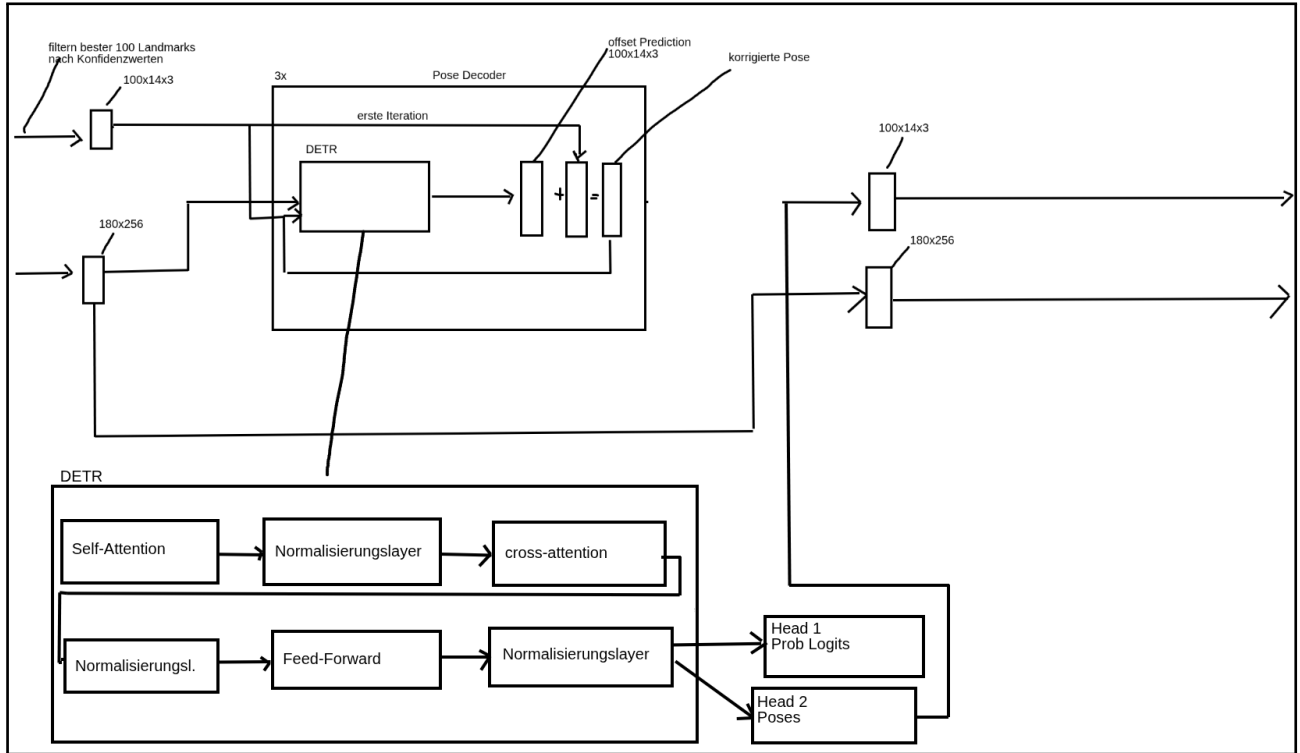


Abbildung 3.2: Skizze des Pose Decoders. Beschreibt die inkrementelle Verbesserung der 3D-Posen. Ebenfalls wird der Inhalt des DETR Blockes angedeutet und die beiden Output Heads beschrieben.

Die einzelnen Layer des Pose Decoders haben wie der Encoder zwei Heads, welche Positionen und Wahrscheinlichkeiten zu diesem Zeitpunkt produzieren. Diese fließen dann ebenfalls in die Loss-Funktion ein.

3.1.4 Refine Decoder

Der Refine Decoder dient dazu, die vom Pose Decoder bereits vorhergesagten Gelenkpositionen schrittweise weiter zu verbessern. Das zugrunde liegende Prinzip ist eine iterative Verfeinerung: Jede Schicht erzeugt nicht direkt neue absolute Koordinaten, sondern berechnet einen relativen Versatz für die Gelenkpositionen. Dieser Versatz wird anschliessend auf die Ausgabe der vorherigen Schicht addiert, wodurch die Pose in mehreren kleinen Schritten präziser wird.

Mathematisch lässt sich dieser Ablauf für die Gelenkkoordinaten wie folgt schreiben:

$$J_d = J_{d-1} + \Delta J_d$$

wobei J_d die verfeinerte Gelenkposition der aktuellen Schicht, J_{d-1} die Position der vorherigen Schicht und $\Delta J_d = (\Delta x, \Delta y, \Delta z)$ den von der aktuellen Schicht geschätzten Offset bezeichnet. Durch dieses schrittweise Vorgehen kann das Modell kleine Fehler gezielt korrigieren, ohne

dass eine einzelne Schicht die gesamte Pose stark verschiebt. Dadurch wird die Stabilität der Vorhersage erhöht und die finale Koordinatenschätzung präziser.

In dieser Architektur besteht der Refine Decoder aus drei aufeinanderfolgenden Schichten, die nach dem gleichen Grundprinzip aufgebaut sind wie die Basisblöcke des DETR-Decoders. Im Unterschied zum Pose Decoder werden jedoch nur die vielversprechendsten Posen weiterverarbeitet, damit sich das Modell auf die präzisere Nachkorrektur der wahrscheinlichsten Personen konzentrieren kann. Der Refine Decoder konzentriert sich also nur noch auf Positionen und klassifiziert nicht mehr.

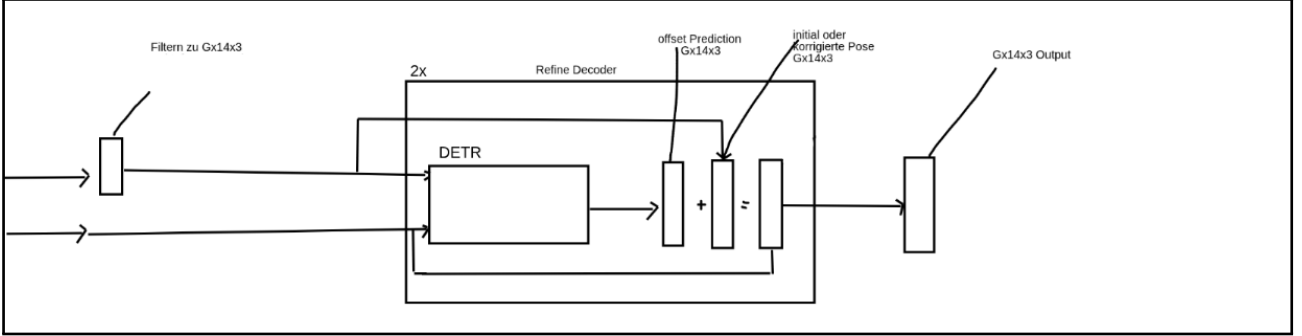


Abbildung 3.3: Skizze des Refine Decoders und des finalen Outputs.

Hier haben ebenfalls alle Layer einen Head, von welchem dann die Informationen dann in die Loss-Funktion fließen.

3.1.5 Loss Funktion der Yan Architektur

Die Loss-Funktion setzt sich aus zwei grundsätzlichen Komponenten zusammen, einer Klassifikations- und einer Regressionskomponente. Ziel ist es einerseits zu bestimmen, ob eine vorhergesagte Pose tatsächlich zu einer Person gehört L_{cls} , und andererseits die zugehörigen 3D-Gelenkkoordinaten möglichst präzise zu schätzen L_{kpt} ($L_{tot} = L_{cls} + L_{kpt}$).

Die Loss-Komponenten werden in allen Blöcken generiert: nach dem Encoder, nach jedem Pose-Decoder-Layer und nach jedem Refine-Decoder-Layer. Im Refine-Decoder-Layer wird allerdings nur noch zu L_{kpt} beigetragen. Da die Layer mehr Posen vorhersagen als in der Ground Truth vorhanden sind, braucht es ein Matching, um aus der Menge der produzierten Posen die am besten zur GT passenden auszuwählen. Dafür wird ein Hungarian-Matching verwendet, welches für alle vorhergesagten und alle tatsächlichen Posen eine Kostenmatrix berechnet und dann diejenige Zuordnung mit den geringsten Gesamtkosten auswählt. Nur diese ausgewählten Posen fließen dann in die Loss Funktion ein.

L_{cls} (Focal Loss): Für ein positives Beispiel ergibt sich:

$$L_{cls} = -\alpha(1 - \hat{y})^\gamma \log(\hat{y}) \quad \text{falls } y = 1$$

und für ein negatives Beispiel:

$$L_{cls} = -\alpha\hat{y}^\gamma \log(1 - \hat{y})$$

Die Klassifikationskomponente basiert auf der Focal Loss. Diese Verlustfunktion wurde entwickelt, um das Ungleichgewicht zwischen einfachen und schwierigen Trainingsbeispielen besser

zu behandeln. In diesem Kontext bedeutet das: Es gibt viele Vorhersagen, die keine Person darstellen, und nur wenige, die tatsächlich eine echte Person repräsentieren. Eine normale Kreuzentropie würde viele der einfachen Negativbeispiele zu stark gewichten. Die Focal Loss reduziert deshalb den Einfluss bereits gut erkannter Beispiele und fokussiert das Training auf schwierige, unsichere oder fehlerhafte Vorhersagen. Hierbei ist y das binäre Ground-Truth-Label, also $y = 1$ für eine Person und $y = 0$ für keine Person. Der Wert $\hat{y}$ bezeichnet den vom Modell vorhergesagten Konfidenzwert für die Klasse "Person". Der Parameter α dient als Gewichtung zwischen positiven und negativen Beispielen und kann helfen, ein Klassenungleichgewicht auszugleichen. Der Parameter γ ist der sogenannte Focusing-Parameter. Er steuert, wie stark einfache Beispiele abgeschwächt werden: Je grösser γ ist, desto weniger Einfluss haben bereits korrekt klassifizierte Beispiele auf den Loss.

$$L_{kpt} = \text{mean} \left(\|\hat{P} - P_{gt}\|^2 \right)$$

L_{kpt} beschreibt dann die Loss Funktion, welche bei der Koordinatenausgabe verwendet wird. Es wurde klassisch MSE (Mean-Squared Error) verwendet.

3.2 CNN-Attention-Diffusion Architektur

Neben dem Ansatz von Yan wurde eine weitere, eigene Architektur entwickelt. Diese setzt sich aus einem CNN-Encoder, Attention-Layern und einem Diffusions-Decoder zusammen. Der CNN-Encoder projiziert die WLAN-Signale in den latenten Raum $\in (1 \times 192)$. Danach werden lernbare temporale und räumliche Embeddings addiert und die Daten in einen Attention-Block eingespeist. Der Attention-Block besteht dabei aus 4 Layern mit je einem Multi-Head-Self-Attention-Modul und einem Feed-Forward-Netzwerk. Danach gibt es einen ersten Head, welcher zwei Ergebnisse produziert: einen absoluten Hüftpunkt und relative Koordinatenverschiebungen zu diesem Hüftpunkt. Daraus setzt sich dann die absolute Pose bzw. die 3D-Koordinaten zusammen. Die relativen Offsets werden dann als Vektor $\in$ im Netzwerk weiterverwendet. Diese fließen dann zusammen mit dem Output des Attention-Blocks und Diffusions-internen Time-Embeddings in das Diffusionsnetzwerk. Das Diffusionsnetzwerk besteht dann aus drei Fully-Connected-Layern und verbessert die relative Verschiebung iterativ. Die finale Pose wird dann mithilfe der absoluten Rootkoordinate aus dem Transformer und dem neuen relativen Offset aus dem Diffusionsteil berechnet.

Die grundlegende Idee dahinter ist, dass das CNN die relevanten Informationen der CSI-Daten pro Zeitschritt bzw. pro CSI-Paket lernt. Der Attention-Layer soll dann die Bedeutung und Beziehung zwischen den verschiedenen Zeitschritten lernen. Der Diffusions-Layer soll schlussendlich die Koordinaten mittels 10 Diffusionsschritten nochmals korrigieren und nachverbessern. Die Inspiration des Diffusions-Layers stammt dabei aus der folgenden Arbeit, welche auf Basis von CSI-Signalen eine Bildszene rekonstruieren wollte und dabei ein Diffusionsnetzwerk verwendet hat [21].

3.2.1 Tokenisierung und CNN-Encoder

Wie bei der Yan-Architektur müssen auch bei meiner Architektur die Informationen zunächst tokenisiert werden. Die Dimensionen eines Eingabetensors pro Batch betragen dabei $\in \mathbb{R}^{(21 \times 3 \times 3 \times 56 \times 2)}$, also Timesteps, Receiver, Antennen, Subcarrier sowie die Anzahl der Werte pro Channel (Amplitude und Phase). Die Verarbeitung erfolgt pro Zeitschritt parallel, sodass jeder der 21 Ti-

mesteps unabhängig durch den CNN-Encoder geführt wird. Ziel des Encoders ist es, die hochdimensionalen CSI-Informationen in eine kompakte Embedding-Repräsentation zu projizieren. Nach der Verarbeitung besitzt jeder Timestep eine Embedding-Dimension von $\in \mathbb{R}^{(1 \times 192)}$.

Der CNN-Encoder besteht aus insgesamt vier aufeinanderfolgenden Convolution-Blöcken mit ansteigender Kanalanzahl:

$$(2 \rightarrow 32) \rightarrow (32 \rightarrow 64) \rightarrow (64 \rightarrow 128) \rightarrow (128 \rightarrow 192)$$

Jeder Convolution-Block hat dieselbe innere Struktur. Zuerst wird eine 2D-Faltung angewendet. Anschließend folgt eine GroupNorm mit 8 Gruppen zur Stabilisierung des Trainings. Dabei wird jeweils über 8 Subchannels normalisiert. Als Aktivierungsfunktion wird SiLU (Sigmoid Linear Unit) verwendet. Zur Regularisierung wird innerhalb jedes Blocks ein Dropout eingesetzt, um Overfitting zu reduzieren. Danach folgt eine weitere 2D-Faltung, erneut mit GroupNorm. Innerhalb jedes Blocks wird eine Residualverbindung verwendet. So bleibt der Gradientenfluss stabil und tiefere Stacks sind leichter zu trainieren. Nach vier solcher Convolution-Blöcke wird Adaptive Average Pooling angewendet. Dabei wird pro (Receiver + Antenne) * Subcarrier Matrix ein Durchschnittswert erzeugt. Die finale Projektion erfolgt über einen linearen Layer:

$$\text{LayerNorm}(192) \rightarrow \text{Linear}(192, 192) \rightarrow \text{SiLU}$$

Dadurch entsteht pro Timestep ein finales CSI-Embedding der Dimension

$$\in \mathbb{R}^{192}$$

welches anschließend dem Transformer-Encoder übergeben wird.

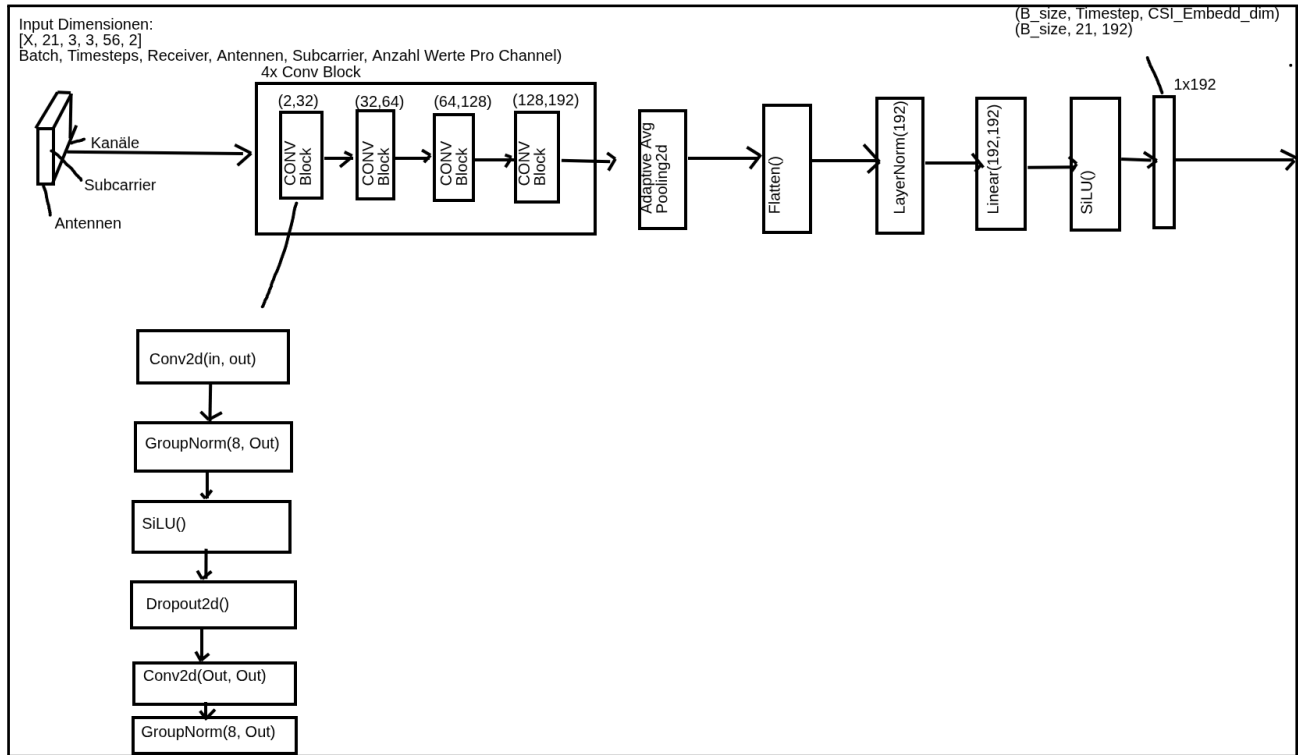


Abbildung 3.4: Architektur des CNN-Encoders

Danach werden lernbare zeitliche Embeddings den Tokens hinzugefügt. Im Attention-Block werden die Tokens dann mittels Self-Attention und Feed-Forward-Layern verarbeitet. Dies

sind Standard-Transformer-Blöcke [19]. Im Multi-Head-Self-Attention sollen die einzelnen CSI-Pakete Informationen über die zeitliche Dimension austauschen. Der Feed-Forward-Block dient dazu, die durch die Attention aggregierten CSI-Informationen pro Token nichtlinear weiter zu transformieren. Dadurch kann das Modell die relevanten Muster in den zeitlichen Signalanteilen präziser herausarbeiten.

3.2.2 Embedding und Attention Block

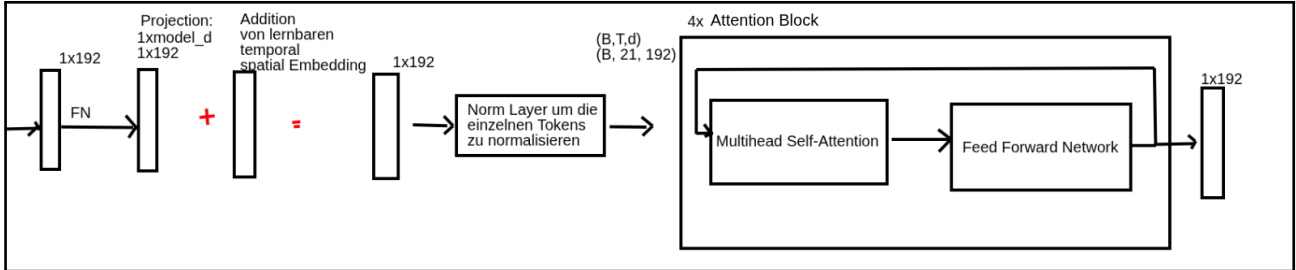


Abbildung 3.5: Embedding- und Attention-Block

Nach dem Attention-Block gibt es einen ersten Head, welcher die produzierte Pose ausgibt. Es werden vor dem Head der zeitlich letzte Token mit einem über alle Zeitschritte gemittelten Token konkateniert. Danach gibt es zwei Fully-Connected-Layer, welche dann einen absoluten Hüftpunkt und Koordinatenoffsets produzieren. Daraus lässt sich dann die absolute Pose bilden. Der absolute Hüftpunkt ist dabei der Mittelpunkt zwischen dem rechten und linken Hüftknochen. Die relativen Offsets fließen dann weiter in den Diffusions Refiner.

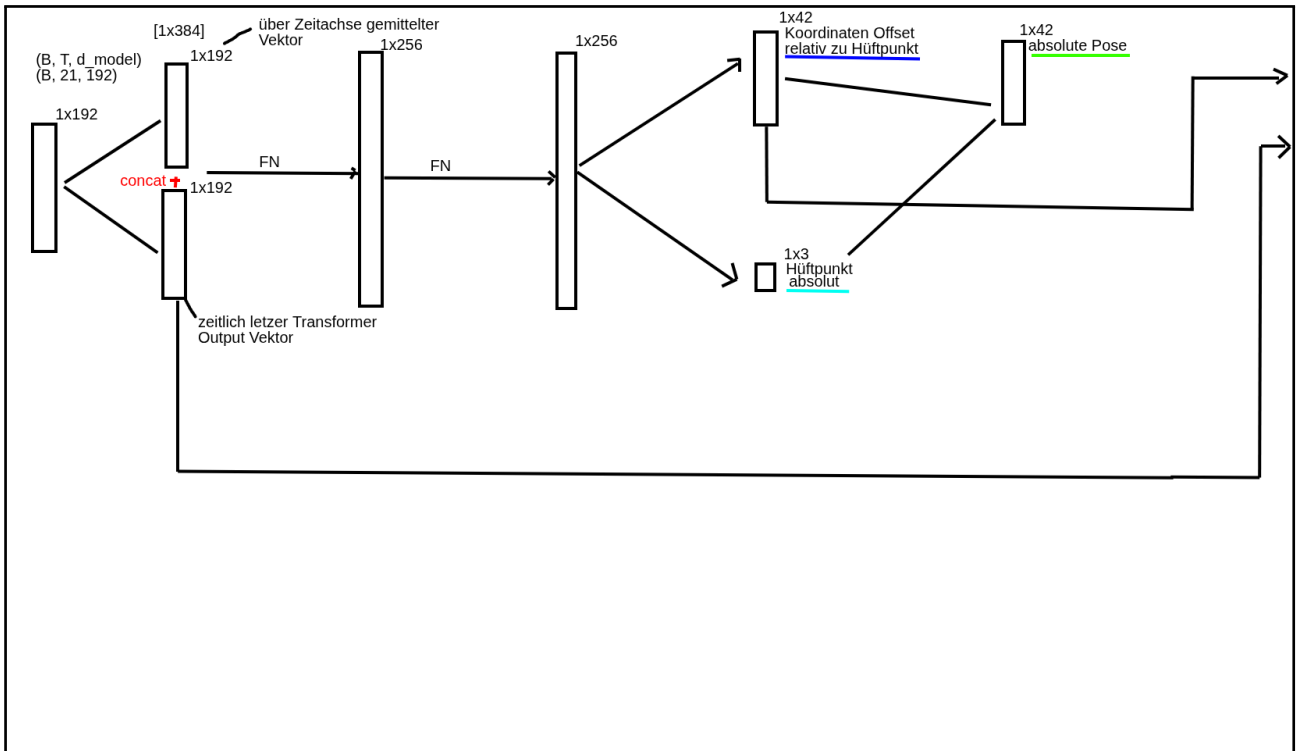


Abbildung 3.6: Pose-Head

Der Diffusions-Refiner erhält anschließend drei unterschiedliche Eingaben. Zum einen die vom Attention-Head vorhergesagte relative Pose relativ zum Hüftmittelpunkt mit der Dimension $\in \mathbb{R}^{42}$, zum anderen den globalen Transformer-Output mit der Dimension $\in \mathbb{R}^{384}$ sowie ein lernbares Diffusions-Timestep-Embedding der Dimension $\in \mathbb{R}^{64}$.

Alle drei Repräsentationen werden zunächst konkateniert, wodurch ein gemeinsamer Feature-Vektor der Dimension

$$42 + 384 + 64 = 490$$

entsteht. Dieser Vektor dient als konditionierte Eingabe für das Diffusionsnetzwerk.

3.2.3 Diffusions-Refiner

Die zusätzlichen Diffusions-Timestep-Informationen ermöglichen es dem Netzwerk zu erkennen, in welcher Iteration des Diffusionsprozesses sich der aktuelle Durchlauf befindet. Dadurch kann das Modell abhängig vom aktuellen Rauschlevel unterschiedliche Korrekturen vornehmen. Frühe Diffusionsschritte arbeiten dabei auf stark verrauschten Posen, während spätere Schritte nur noch feine geometrische Anpassungen durchführen.

Das Refiner-Netzwerk selbst besteht aus mehreren vollständig verbundenen Feed-Forward-Schichten (FN/MLP), welche die Eingaberepräsentation sukzessive in einen Korrekturoffset der Dimension $\in \mathbb{R}^{42}$ projizieren. Dieser Offset beschreibt die vorhergesagte Änderung der relativen Gelenkkoordinaten bezogen auf den Hüftmittelpunkt. Die neue Pose wird anschließend iterativ gemäß

$$\text{relative}_{t+1} = \text{relative}_t - \text{correction} \cdot \text{scale}(t)$$

aktualisiert. Dabei bestimmt $\text{scale}(t)$ die Stärke der Korrektur abhängig vom aktuellen Diffusionsschritt t .

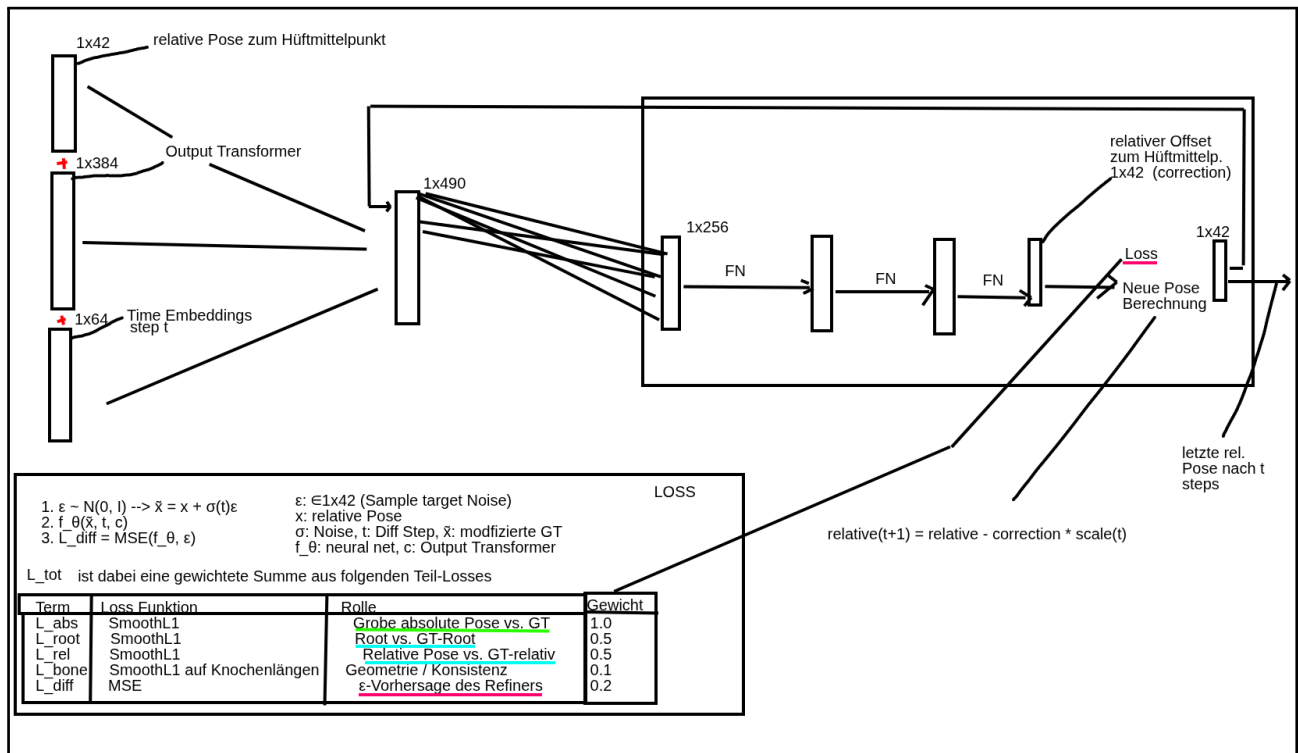


Abbildung 3.7: Diffusionsnetzwerk

3.2.4 Loss Funktion und Rauschen

Die finale Optimierung der CNN-Attention-Diffusion-Architektur kombiniert die Vorhersage der absoluten Pose, der relativen Pose, der Hüftposition, der Knochenlängen und der Diffusionsvorhersage. Für den Diffusions-Refiner wird während des Trainings zusätzlich künstliches Gaußsches Rauschen auf die Zielpose addiert:

$$\epsilon \sim \mathcal{N}(0, I)$$

$$\tilde{x} = x + \sigma(t)\epsilon$$

Das Netzwerk lernt anschließend, dieses verrauschte Signal beziehungsweise den notwendigen Korrekturoffset vorherzusagen:

$$f_{\theta}(\tilde{x}, t, c)$$

Wobei f die Funktion des Diffusionsnetzwerks darstellt. t ist der Timestep (1-10) und c die konditionierten Informationen aus dem Attention-Block. $\tilde{x}$ ist dabei die verrauschte Pose, welche als Input in das Diffusionsnetzwerk gegeben wird. σ produziert dabei abhängig vom Zeitschritt t die Stärke des Rauschens, sodass in frühen Schritten mehr Rauschen hinzugefügt wird als in späteren Schritten. Zur Optimierung wird eine gewichtete Kombination mehrerer Verlustfunktionen verwendet:

$$L_{\text{tot}} = \lambda_{\text{abs}}L_{\text{abs}} + \lambda_{\text{root}}L_{\text{root}} + \lambda_{\text{rel}}L_{\text{rel}} + \lambda_{\text{bone}}L_{\text{bone}} + \lambda_{\text{diff}}L_{\text{diff}}$$

Dabei beschreibt:

- L_{abs} : Absolute Pose gegen Ground Truth
- L_{root} : Fehler der Hüftposition
- L_{rel} : Relative Gelenkpositionen
- L_{bone} : Konsistenz der Knochenlängen (Loss nach Transformer-Output)
- L_{diff} : MSE-Verlust der Diffusionsvorhersage

Die Kombination dieser Teilverluste ermöglicht es dem Modell, sowohl globale Körperpositionen als auch lokale Gelenkgeometrien stabil und anatomisch konsistent zu verfeinern.

3.3 Ground Truth Landmark Datengenerierung

Um ein Modell zu entwickeln, welches Landmarks von Personen detektiert, müssen zuerst Trainingsdaten mit Ground-Truth-Informationen erstellt werden. Es wird also eine Möglichkeit benötigt, um 3D Koordinaten zu allen Körperstellen von Personen zu generieren, welche sich in einem Raum befinden. Dabei wird pro Frame und Person ein Set an Koordinaten generiert, welche die Körperstellen der Person repräsentieren. In der in Kapitel 2.3.1 beschriebenen Studie von Yan et al. wurde dafür eine Azure Kinect Kamera verwendet, welche die 3D-Koordinaten der Körperstellen direkt generiert. Bei dieser Arbeit steht jedoch keine Azure Kinect Kamera zur Verfügung, weshalb eine andere Methode zur Generierung der Ground Truth Daten gefunden werden muss. Es wird eine Intel RealSense Kamera verwendet, welche Tiefeninformationen und RGB-Bilder generieren kann. Diese Informationen werden in einem ersten Schritt simultan

generiert und gespeichert. Die Intel RealSense Kamera produziert die Tiefendaten auf Basis von zwei Kamerasensoren und dem Disparitätsprinzip. Es werden in beiden Bildern gleiche Punkte gesucht und deren Disparität, also die Distanz zwischen den Punkten in beiden Bildern, gemessen. Zudem kennt man auch die Baseline, also die Distanz zwischen den beiden Kamerasensoren. Mit diesen Informationen kann die Tiefeninformation generiert werden. Wobei Z die Tiefe, d die Disparität, B die Baseline und f die Brennweite der Kamera ist.

$$Z = \frac{f \cdot B}{d}$$

Danach wird ein Mediapipe Pose Modell auf die RGB-Bilder angewendet, um die 2D Koordinaten der Körperstellen zu generieren. Die Tiefeninformation kann dann auf Basis der 2D Koordinaten aus den Tiefeninformationen der Intel RealSense Kamera extrahiert werden. Zu diesem Zeitpunkt liegen also die 2D Bildkoordinaten und die Tiefeninformation pro 2D Koordinate vor. Diese Informationen können nun kombiniert werden, um die 3D Koordinaten der Körperstellen zu generieren. Um diese Koordinatentransformation von 2D auf 3D durchzuführen, werden noch die intrinsischen Parameter der Intel RealSense Kamera benötigt. Diese intrinsischen Parameter umfassen die Brennweite der Kamera, den Principal Point und den depth scale factor der Kamera. Die Brennweite der Kamera gibt die Distanz zwischen dem Bildsensor und der Linse an. Der Principal Point gibt die 2D-Bildkoordinaten an, welche die optische Achse der Kamera schneidet. In anderen Worten, es ist der Punkt im Bild, welcher durch eine senkrechte Linie durch die Linse getroffen wird. Dieser Punkt ist, aufgrund von Fertigungstoleranzen, nicht immer genau in der Mitte des Bildes. Deshalb wird er gebraucht um die 3D Koordinaten korrekt zu generieren. Der depth scale factor gibt an, wie die Tiefeninformationen der Kamera in Meter umgerechnet werden kann. Er ist notwendig, da die Tiefeninformationen der Kamera in einem anderen Format vorliegen, als die 2D Koordinaten der Körperstellen. In Figure 3.1 wird eine Visualisierung der Brennweite (focal length) und des Principal Points der Kamera gezeigt, welche für die Koordinatentransformation von 2D auf 3D notwendig sind. Die Pinhole plane ist dabei die Ebene, auf welcher die Kameralinse sich befindet und die Image plane ist die Ebene, auf welcher sich der Bildsensor befindet.

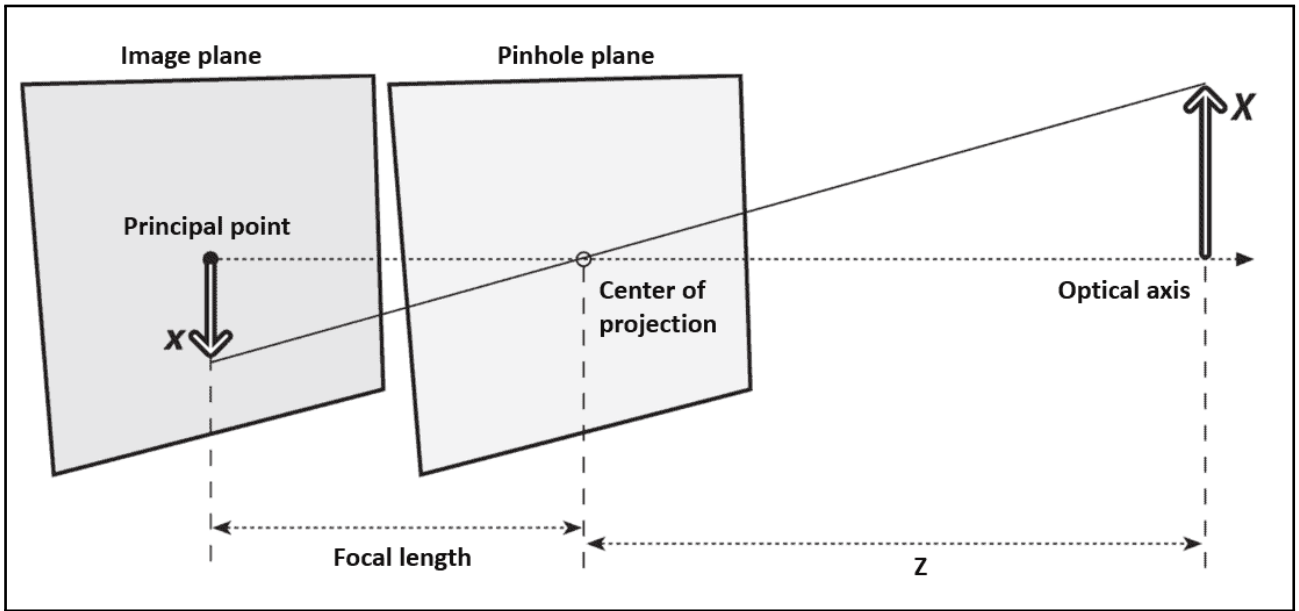


Abbildung 3.8: Visualisierung des Principal Points und der Brennweite der Kamera für die Koordinatentransformation von 2D auf 3D. Quelle: [22]

Mathematisch ausgedrückt, wird die Tiefeninformation (d) mit den 2D Koordinaten (x, y) kombiniert, um die 3D Koordinaten (X, Y, Z) zu generieren. (d) besteht dabei aus einem rohen Tiefenwert (d_{raw}), welcher von der Tiefenkamera generiert wird, und einem depth scale factor (d_{const}), welcher die Umrechnung in Meter ermöglicht und ein konstanter Wert ist. f_x und f_y sind die Brennweiten der Kamera in x- und y-Richtung und nicht die reine Brennweite als Distanz zwischen der Linse und dem Bildsensor. c_x und c_y sind die Koordinaten des Principal Points. Die Werte werden mit folgenden Formeln berechnet:

$$X = \frac{(x - c_x) \cdot (d_{raw} \cdot d_{const})}{f_x}$$

$$Y = \frac{(y - c_y) \cdot (d_{raw} \cdot d_{const})}{f_y}$$

$$Z = d_{raw} \cdot d_{const}$$

(f_x, f_y) , die Brennweiten der Kamera in Pixeln, werden mathematisch folgendermassen berechnet:

$$f_x = \frac{F \cdot W}{S_x}$$

$$f_y = \frac{F \cdot H}{S_y}$$

wobei F die Brennweite der Kamera in Metern ist, W und H die Anzahl an Pixeln in x- und y-Richtung bzw. die Auflösung der Kamera sind. S_x und S_y sind die Breite und Höhe des Bildsensors der Kamera in Metern. f_x und f_y können mit der pyrealsense2 library direkt ausgelesen werden und müssen nicht manuell berechnet werden. In Figure 3.9 wird eine Visualisierung der 3D-Koordinaten der Landmarks einer Person für einen einzigen Frame gezeigt.

3D skeleton | frame 1400 / 1691 | fixed axes (m)

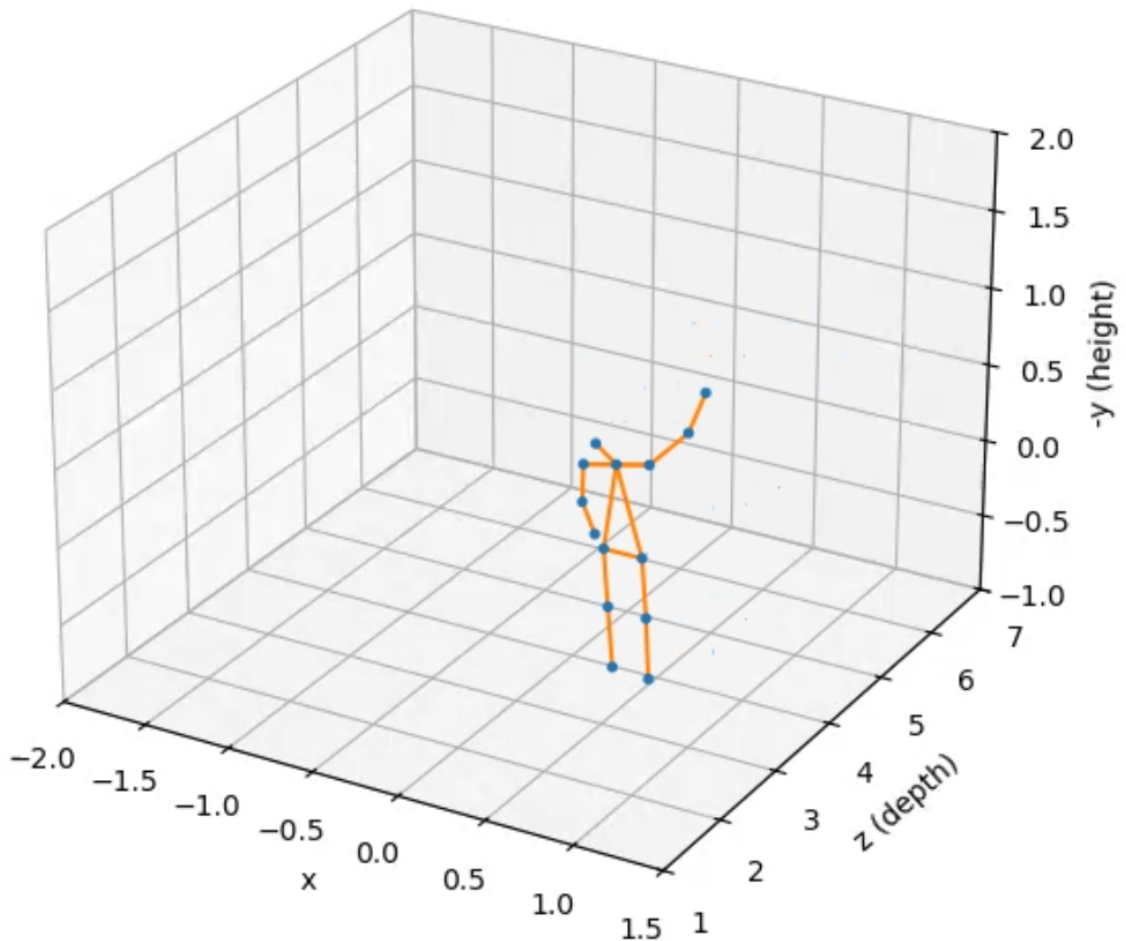


Abbildung 3.9: Visualisierung der Ground Truth Datengenerierung mittels Intel RealSense Kamera mit Tiefeninformationen. Die Grafik zeigt ein Set an 3D Koordinaten der Körperstellen einer Person, welche sich in einem Raum befindet. Beschreibung oben zeigt die konkrete Aufnahme: raising_hands5 und Framenummer: 1400

3.4 CSI Datengenerierung

Die Generierung der CSI-Daten erfolgt mit einem Transmitter-Router und drei Receiver-Routern. Alle diese Router sind vom Typ TP-Link Archer AC1750 und unterstützen die CSI-Auslesung

dank ihrer Atheros-Chipsätze. Der Transmitter-Router sendet kontinuierlich Pakete an die drei Receiver-Router, welche die CSI-Signale (Amplitude und Phase) auslesen. Die Receiver-Router befinden sich dabei im WiFi des Transmitter-Routers und sind gleichzeitig per Ethernet-Kabel und Switch mit einem Laptop verbunden. Auf den Receivern läuft dabei die Atheros Software, welche die CSI-Signale ausliest und dann an den Laptop auf einem vorkonfigurierten Port sendet. Der Laptop empfängt dann die CSI-Signale von allen drei Receivern gleichzeitig und speichert diese in .dat Files.

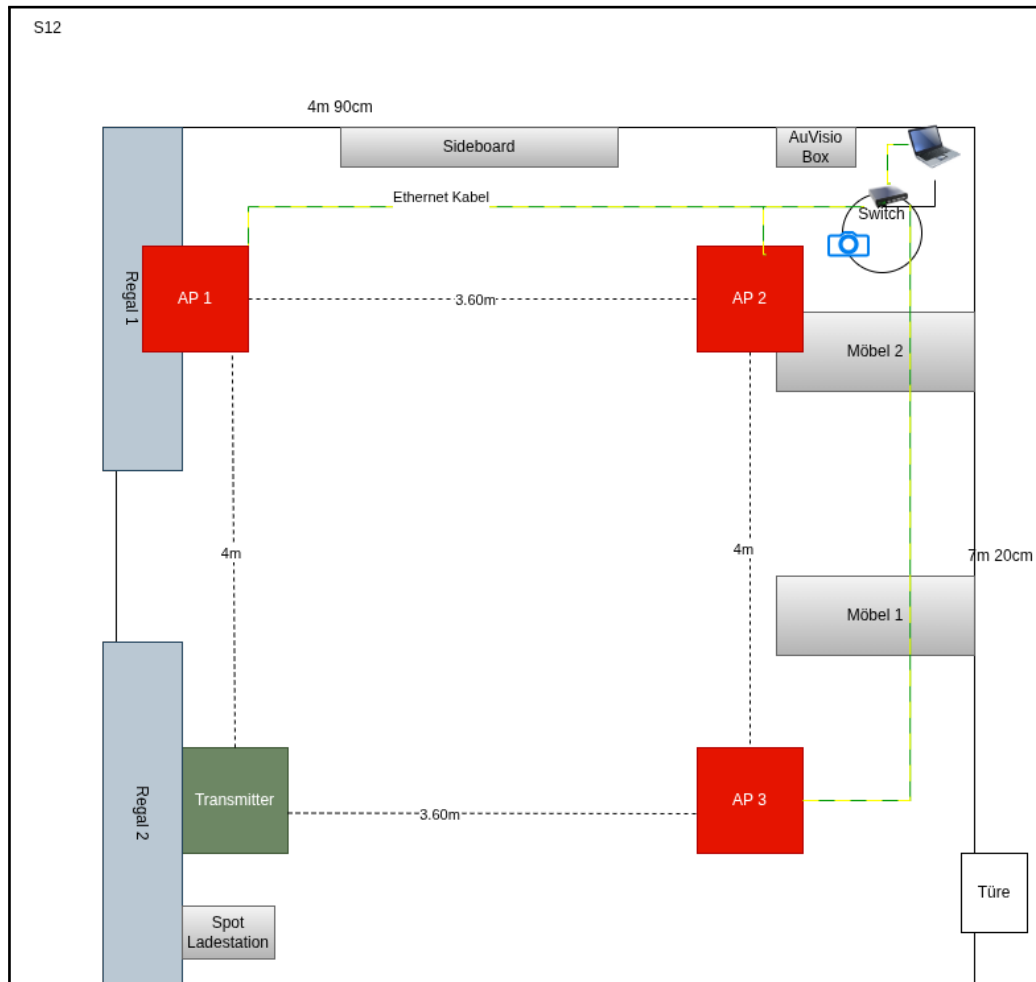


Abbildung 3.10: Visualisierung des Router-Setups für die CSI-Datengenerierung. Auf der Grafik sieht man den Aufbau des Raumes mit Möbeln und die Positionen der Router. Zusätzlich sind auch die Dimensionen des Experimentbereiches eingezeichnet sowie die Kamera, den Laptop und die Ethernet-Kabel.

4. Methodik

4.1 Projektplanung

Da es sich bei diesem Projekt um eine Einzelarbeit handelt und alle Schritte gut planbar sind, wurde mit einer klassisch, sequenziellen Vorgehensweise vorgegangen. In der Timeline sind die einzelnen Arbeitsschritte ersichtlich. In der Planung wurden 4 Meilensteine erstellt. 1 „Arbeit schreiben“, 2 „Realisierung PoC“, 3 „Evaluierung und Präsentation Endprodukt“ und 4 „Zwischenpräsentation erstellen“. „Arbeit schreiben“ beinhaltet dabei die Einreichung der Aufgabenstellung, das Schreiben der Arbeit und das Schreiben des ersten Kapitels, welches separat und direkt nach der Literaturrecherche eingeplant wurde. Die Realisierung des PoC's beinhaltet die Datenakquise für das Modell, das Erstellen und Trainieren des Modells. Die Evaluierung und Präsentation des Endproduktes beinhaltet genau das, was der Titel suggeriert. Die Ergebnisse sollen bewertet und so aufbereitet werden, dass sie in der Arbeit und bei der Endpräsentation anschaulich gezeigt werden können. Die Präsentation der Zwischenpräsentation wurde als eigener Meilenstein behandelt, da diese ebenfalls gewichtig mit 20 Prozent in die Endnote einfließt.

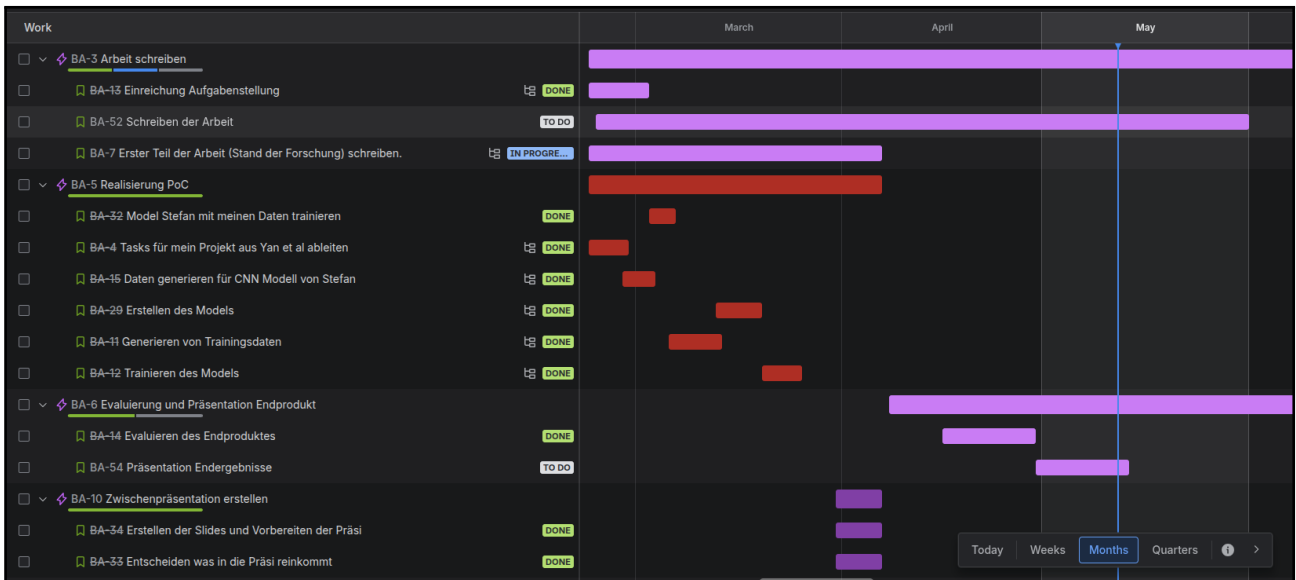


Abbildung 4.1: Planung BAA

Grundsätzlich wurde die Abfolge der Arbeitspakete, welche geplant wurden, gut eingehalten. Der Plan konnte in der Realität nicht ganz eingehalten werden und einige Meilensteine mussten etwas nach hinten verschoben werden. Die Realisierung des POCs wurde erst Ende April erreicht und nicht bereits Anfang April. Der davon abhängige Meilenstein „Evaluierung und Präsentation des Endproduktes“ hat sich deshalb ebenfalls nach hinten verschoben. Grundsätzlich war dies kein grosses Problem, da genügend Puffer eingeplant worden war. Der Meilenstein „Evaluierung und Präsentation der Ergebnisse“, der von der Realisierung des POCs abhängig ist, wurde bewusst mit einem etwas grösseren Zeitfenster geplant. Der Meilenstein „Arbeit schreiben“ wurde wie geplant parallel zu den anderen Meilensteinen bearbeitet.

Die Endpräsentation und deren Erstellung wurde bewusst nicht in diese Planung aufgenommen, da sie erst nach der Abgabe der Arbeit erfolgen wird.

4.2 Risiko-Analyse

Bei der Risiko-Analyse vor der Umsetzung wurden drei Risiken eruiert.

Risiko	Beschreibung	Schwere	Wahrscheinlichkeit
Keine Landmark-Koordinaten	Bei der Erfassung der Trainingsdaten können keine oder ungenügende Koordinaten für die Landmarks generiert werden.	Kritisch	4
Matching Kamera/CSI	Das Matching von Kamera- und CSI-Daten funktioniert nicht zuverlässig.	Kritisch	3
Modellkonvergenz	Das Modell konvergiert nicht.	Mittel	2

Tabelle 4.1: Wesentliche Projektrisiken

Das Risiko, dass bei der Erfassung der Trainingsdaten keine qualitativen 3D-GT-Koordinaten erstellt werden können, wurde aufgrund der Wahrscheinlichkeit und des Schweregrads auf die Umsetzung der Arbeit prioritär und frühzeitig behandelt. Es stellte sich heraus, dass es tatsächlich ein Problem darstellte, stabile und qualitative Koordinaten zu erstellen. Deshalb wurde, wie in 5.3.1 erläutert, viel Zeit in die Nachbearbeitung der rohen Koordinaten investiert und das Risiko dementsprechend entschärft. Die Umgehungsstrategie, bei der eine Azure Kinect gekauft würde, musste so nicht eingesetzt werden. Des Weiteren wurde viel Zeit in die Timestamp-Generierung investiert, welche beim Risiko Matching Kamera/CSI identifiziert wurde. Es wurde direkt ein Timestamp der CSI Pakete beim Auslesen auf den Routern produziert. Zudem wurde ein NTP-Skript erstellt, um beim Beginn einer Aufnahme-Session die Uhren der Router mit der Uhr des Laptops zu synchronisieren. Das letzte Risiko, dass das Modell nicht konvergiert, hatte eine leicht kleinere Eintretenswahrscheinlichkeit. Das Modell hat die Trainingsdaten gut gelernt, weshalb kein neues Modell mit anderer Architektur als Ersatz eingesetzt werden musste.

5. Realisierung

5.1 Initialisieren der Router

Bei der Wahl der Router musste darauf geachtet werden, dass die Router Atheros Chipsätze enthalten. Der Grund dafür ist, dass nur Router mit Atheros-Chipsätzen CSI-Signale auslesen können. Am Robotics Lab der HSLU wurde bereits eine entsprechende Vorgängerarbeit zu CSI-Sensing durchgeführt. Im Rahmen dieser Arbeit hat sich gezeigt, dass der TP-Link AC1750 Archer C7 v2 mit einem Atheros-Chipsatz ausgestattet ist und sich damit für das Auslesen der CSI-Signale eignet. Deshalb konnten zwei Router aus der Vorgängerarbeit weiterverwendet werden und mussten nicht neu beschafft werden [23]. Die dazugehörige Software wurde ebenfalls in die Codebasis dieser Arbeit übernommen.



Abbildung 5.1: TP-Link AC1750 Archer C7 v2 mit Atheros Chipsatz ausgestattet und der Möglichkeit CSI Signale auslesen zu können. Er besitzt drei Antennen und 4x Ethernet Gigabit Ports

Auch die bereits vorhandenen Skripte für das Auslesen der CSI-Daten sowie für das Senden der WLAN-Nachrichten vom Transmitter konnten direkt wiederverwendet werden. Darüber hinaus wurde auch das Shell-Skript übernommen, das die CSI-Daten von einem Receiver-Router an ein Drittgerät weiterleitet. Lediglich beim Parsen der CSI-Nachrichten waren kleinere Anpassungen notwendig. Zusätzlich wurde jeder CSI-Nachricht ein globaler Timestamp des jeweiligen Receiver-Routers hinzugefügt, um das spätere Matching mit den Kameradaten zu vereinfachen. Da für das Setup insgesamt vier Router benötigt wurden, mussten zwei zusätzliche Geräte beschafft werden. Dabei wurden dieselben Routermodelle wie in der Vorgängerarbeit verwendet, um die Kompatibilität sicherzustellen.

Um auf den Receiver-Routern die CSI-Signale auslesen zu können, musste die Firmware der Geräte auf ein OpenWrt-Image geflasht werden [24]. OpenWrt ist dafür die zwingende Voraussetzung, da die Standard-TP-Link-Firmware keinen Root-Zugriff auf das Betriebssystem bietet, den Zugriff auf die notwendigen Treiber blockiert und keinen direkten Zugriff auf den WLAN-Chip erlaubt [25]. OpenWrt schafft hingegen mit einem speziell modifizierten ath9k-Treiber den direkten, hardwarenahen Zugriff auf die internen Register des Chips, sodass die rohen CSI-Matrizen abgefangen und in den User-Space übergeben werden können [26].

Das Flashen der Firmware erfolgte manuell per TFTP, da sich die OpenWrt-Version nicht über die GUI der Standard-Firmware installieren liess. Dabei wurde der Reset-Knopf gedrückt gehalten, wodurch ein routerseitiger Listener gestartet wurde, der die OpenWrt-Firmware empfing und installierte. Verwendet wurde OpenWrt in der Version 19.07.

Nachdem die OpenWrt-Firmware auf die Router geflasht worden war, konnte das Netzwerk für die Receiver-Router konfiguriert werden. Auf dem Laptop wurde dafür dem entsprechenden Interface eine statische IP-Adresse zugewiesen, damit die Receiver-Router die UDP-Pakete mit den CSI-Signalen an den Laptop senden konnten. Anschliessend konnten die Shell-Skripte beziehungsweise Atheros-Skripte per scp auf die Receiver-Geräte übertragen werden. Danach musste nur noch die IP-Adresse im Shell-Skript an die zuvor festgelegte Laptop-IP angepasst werden.

Im nächsten Schritt wurden die drei Receiver-Router in das WLAN-Netzwerk des Transmitters eingebunden, damit sie überhaupt WiFi-Pakete empfangen und auswerten konnten. Dies erfolgte über die GUI auf den Receiver-Routern. Die Receiver-Router sind danach ganz normale Clients im Netz des Transmitters, wie andere Geräte auch. Der Aufbau des Router-Setups entsprach dabei der in Kapitel 3.2.2 und in Bild 3.3 beschriebenen Konfiguration. Nachfolgend noch ein Netzwerkdiagramm, welches die Kommunikation zwischen den Geräten zeigt und die Übersicht aus Figure 3.10 ergänzt.

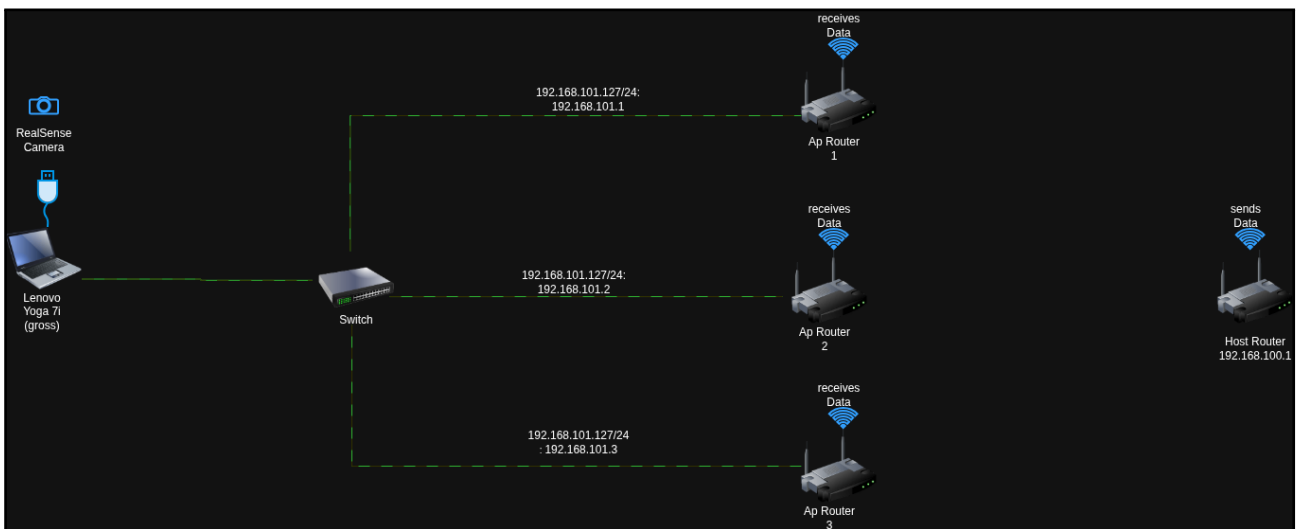


Abbildung 5.2: Netzwerkdiagramm der Router und des Laptops. Es zeigt die Kommunikation zwischen den Geräten, insbesondere die Übertragung der CSI-Daten von den Receiver-Routern zum Laptop und die verwendeten IP-Adressen.

5.2 Datenerfassung

5.2.1 Strukturelle Analyse der Daten

Um sicherzustellen, dass es keinen Bias in den empfangenen Daten gibt, wurden die Daten strukturell analysiert. Dabei wurden 9 Aufnahmen mit gleichem Setup wie bei der Aufnahme der Trainingsdaten durchgeführt. Um einen eventuellen leistungsschwachen Router zu identifizieren, wurden die Grösse der Matching-Windows über alle drei Router verglichen. Ein Matching-Window ist dabei das Zeitfenster für das Verbinden eines Frame pro Router, in welchem sich alle 7 CSI Pakete befinden. Ist das durchschnittliche Matching-Window klein, bedeutet dies ein leistungsstarker Router, da viele Pakete rund um einen Frame empfangen wurden. Ein grösseres Matching-Window bedeutet hingegen, dass weniger Pakete empfangen wurden und somit der Router leistungsschwächer ist. Wenn die Matching-Windows über alle drei Router verglichen werden, kann festgehalten werden, dass über alle 9 runs eine stabile Matching-Window Grösse erreicht werden kann. Das Setup ist also stabil und die Flussrate der Pakete ist über alle drei Router ähnlich. Das dies bei dieser Arbeit der Fall war, lässt sich aus Figure 5.2 entnehmen.

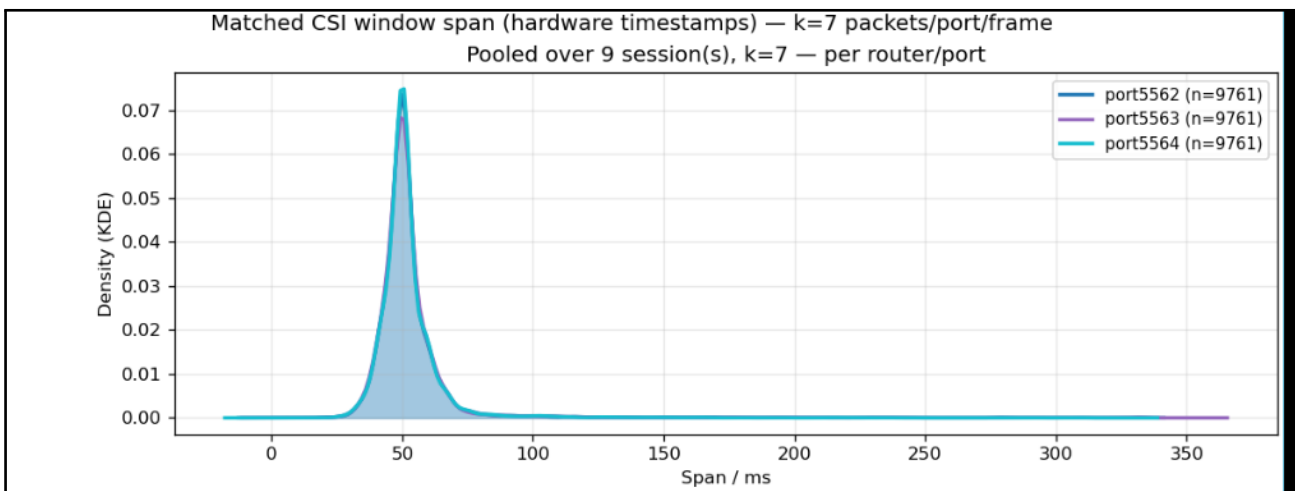


Abbildung 5.3: Grafik zeigt eine Verteilung der Matching-Window Grösse über alle drei Router hinweg. Die drei Verteilungen liegen übereinander und zeigen, dass die Flussrate der Router ähnlich sind. Die x-Achse zeigt die Grösse des Matching-Windows in Milisekunden und die y-Achse zeigt die Dichte von Matching-Windows mit dieser Grösse.

5.2.2 Statistische Analyse von CSI Daten

Die Vermutung, dass Personenerkennung mittels CSI-Daten Potenzial hat, wurde in dieser Studie bereits dargelegt [27]. Xiao et al. haben in einer Untersuchung festgestellt, dass sich die Varianz des Amplitudenwertes über die Zeit zwischen einer Umgebung mit und ohne Person unterscheidet. Die Varianz der Amplitude ist dabei mit einer Person im Raum grösser als ohne Person. Dies zeigt, dass es charakteristische Unterschiede im Amplitudenwert des CSI-Signals gibt, und motiviert den Einsatz von Machine-Learning-Methoden und -Modellen. Wenn die grosse Anzahl an Daten mittels eines geeigneten Machine-Learning-Modells verarbeitet werden kann, sollte es möglich sein, nützliche Aussagen bzw. Vorhersagen zu generieren.

In dieser Arbeit wurde ebenfalls ein statistischer Vergleich zwischen Amplitudenwerten aus einer Umgebung mit und ohne Personen gemacht. Es wurden dabei 10 Testaufnahmen gemacht,

von denen 5 eine Person beinhalten und 5 einen leeren Raum zeigen. Die beiden nachfolgenden Bilder zeigen zwei separate Aufnahmen und sind eine Auswahl aus den 10 Aufnahmen (Aufnahme 5 und 6). Die beiden Grafiken zeigen den Amplitudenverlauf über die Zeit. Beim Vergleich von Figure 5.3 (Person im Raum) und Figure 5.4 (keine Person im Raum), kann man bereits mit blosssem Auge erkennen, dass sich die beiden Diagramme unterscheiden. 5.3 hat grössere und mehr Peaks in den Amplitudenwerten als 5.4.

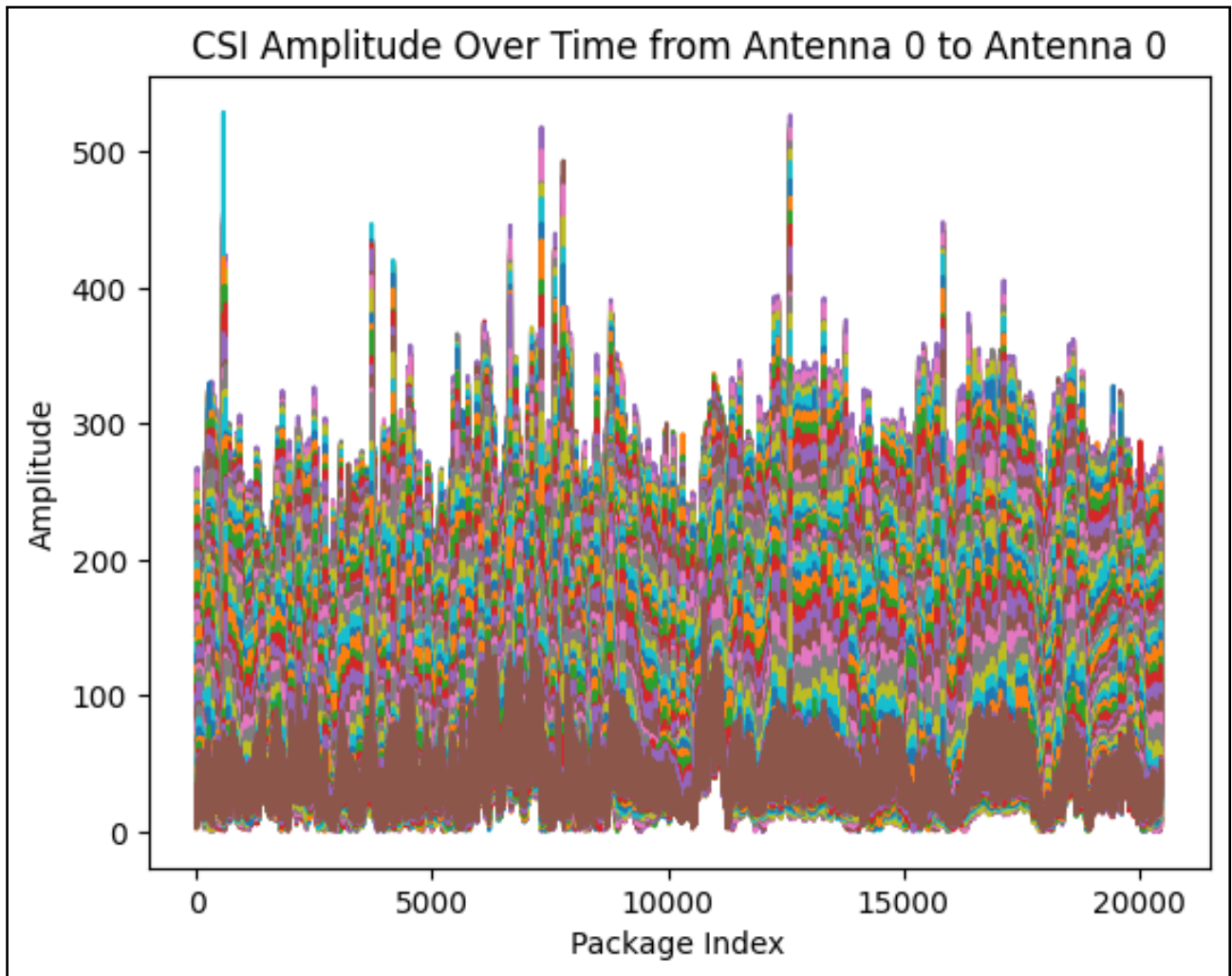


Abbildung 5.4: Die Grafik hat zwei Achsen: die x-Achse zeigt die Paketnummer bzw. den Verlauf über die Zeit, während die y-Achse den gemessenen Amplitudenwert visualisiert. Die verschiedenen Farben beziehen sich dabei auf einen bestimmten Subchannel. Der Router misst dabei pro Paket immer 56 Werte, für jeden Subchannel einen Wert. Bei dieser Grafik befand sich eine Person im Raum.

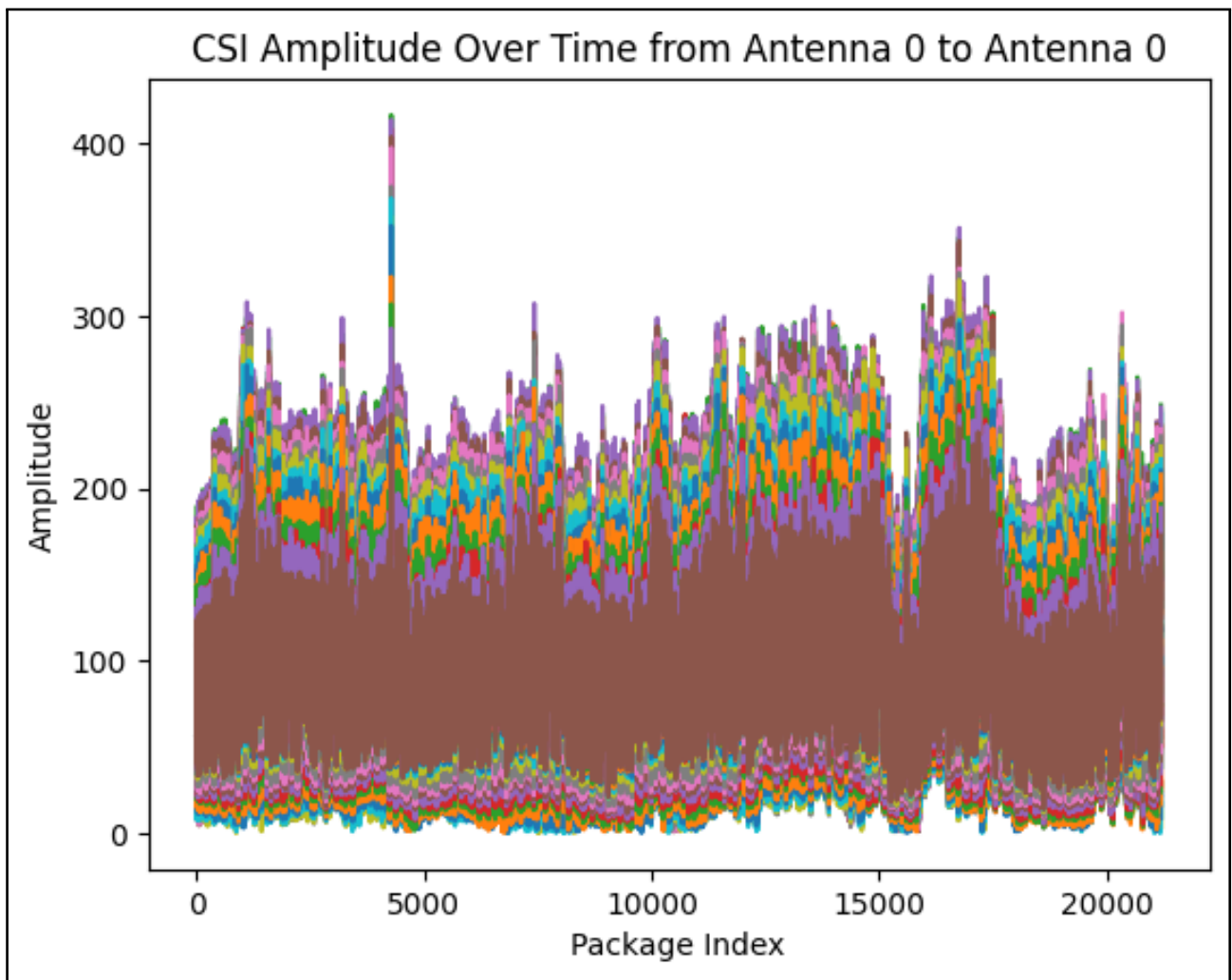


Abbildung 5.5: Amplitude der WiFi-Pakete, wenn sich keine Person im Raum befindet.

Auch statistisch lässt sich der Unterschied beweisen. Die Aufnahmen mit einer Person im Raum besitzen eine grössere Varianz als die Aufnahme im leeren Raum. Es wurde einmal die Varianz aller Personensamples und aller leeren Samples berechnet. Dabei wurden zwei leicht unterschiedliche Arten zur Berechnung angewendet. Beim ersten Wert der Varianz wurden die Subchannel-Werte eines Pakets aggregiert, sodass pro Paket nur ein einziger Wert in die Verteilung aufgenommen wird. Beim zweiten Wert der Varianz wurde jeder einzelne Amplitudenwert aller Subchannels eines Pakets in die Verteilung aufgenommen. Beide Vergleiche zeigen eine stärkere Varianz im Fall, dass sich eine Person im Raum befindet.

Methode	Varianz ohne Person	Varianz mit Person	Verhältnis Varianz
Paketwerte	207.903	236.18	1.13601
Subchannelwerten	3275.27	3486.37	1.06445

Tabelle 5.1: Vergleich der Streuung der CSI-Amplituden mit und ohne Person im Raum. Verhältniswerte grösser als 1 bedeuten, dass die Varianz mit Person höher ist als ohne Person.

Exemplarisch hier eine Verteilung der Amplitudenwerte aus den Aufnahmen 5 und 6. Die

beiden Verteilungen wurden übereinander gelegt und zeigen grafisch, dass die Verteilung mit Person eine grössere Breite besitzt.

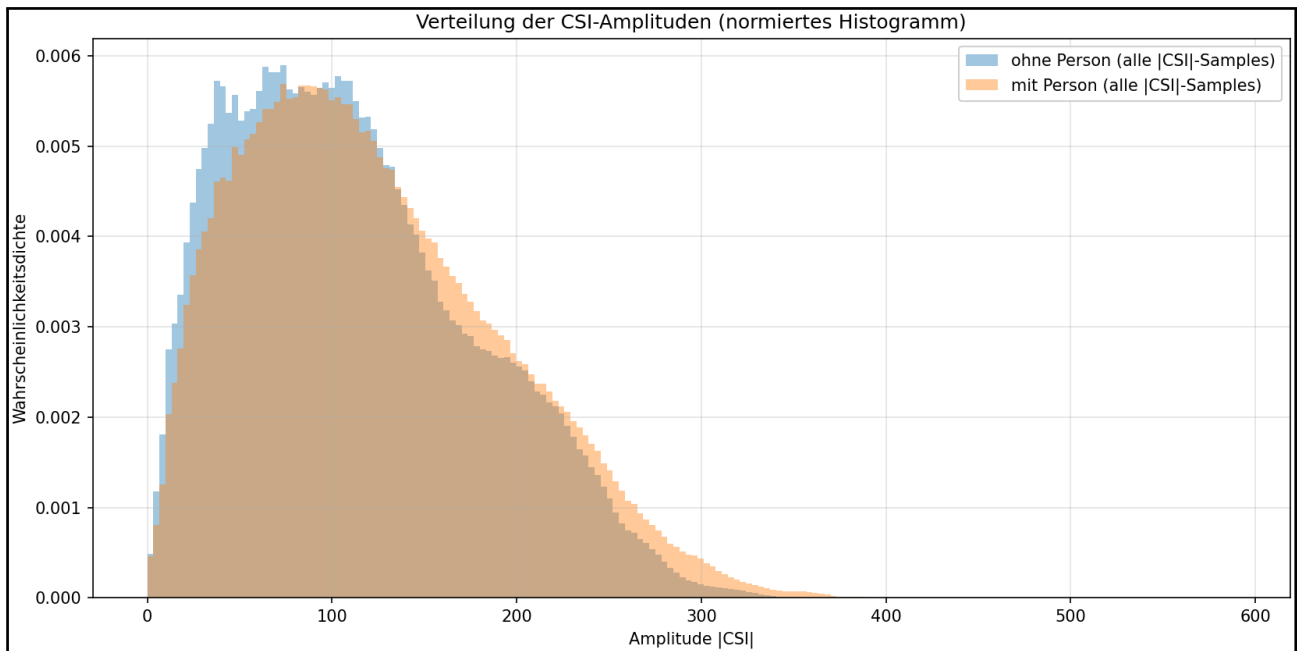


Abbildung 5.6: Die Grafik zeigt zwei Verteilung der Amplitudenwerte aus den Aufnahmen 5 und 6. Ein Datenpunkt der Verteilung ist dabei ein Wert eines Subchannels innerhalb eines Pakets

Meine Auswertungen bestätigen also die Ergebnisse von Xiao et al. und zeigen, dass es charakteristische Unterschiede im Amplitudenwert des CSI Signals gibt, wenn sich eine Person im Raum befindet. Dies motiviert den Einsatz von Machine Learning Methoden und Modellen, um diese Unterschiede zu nutzen und Vorhersagen über die Anwesenheit und Lokalisierung von Personen zu generieren.

5.2.3 Bewegungsmuster

Für die Datenerfassung wurden mehrere Aufnahmen einer einzigen Person in einem Raum gemacht. Um das Datenset möglichst divers zu halten, wurden dabei folgende Bewegungsmuster definiert:

- Bücken
- Sportübungen (Kniebeugen, Bizeps Curls, Ausfallschritte, seitliche Ausfallschritte etc.)
- Arme über den Kopf heben
- Arme noch vorne ausstrecken
- Auf einen Stuhl sitzen und wieder aufstehen
- Stehen im Raum
- Dehnübungen (Dehnen der Arme, Beine, Rücken)

- Laufen im Raum
- Leerläufe bei welchem keine Person im Raum steht

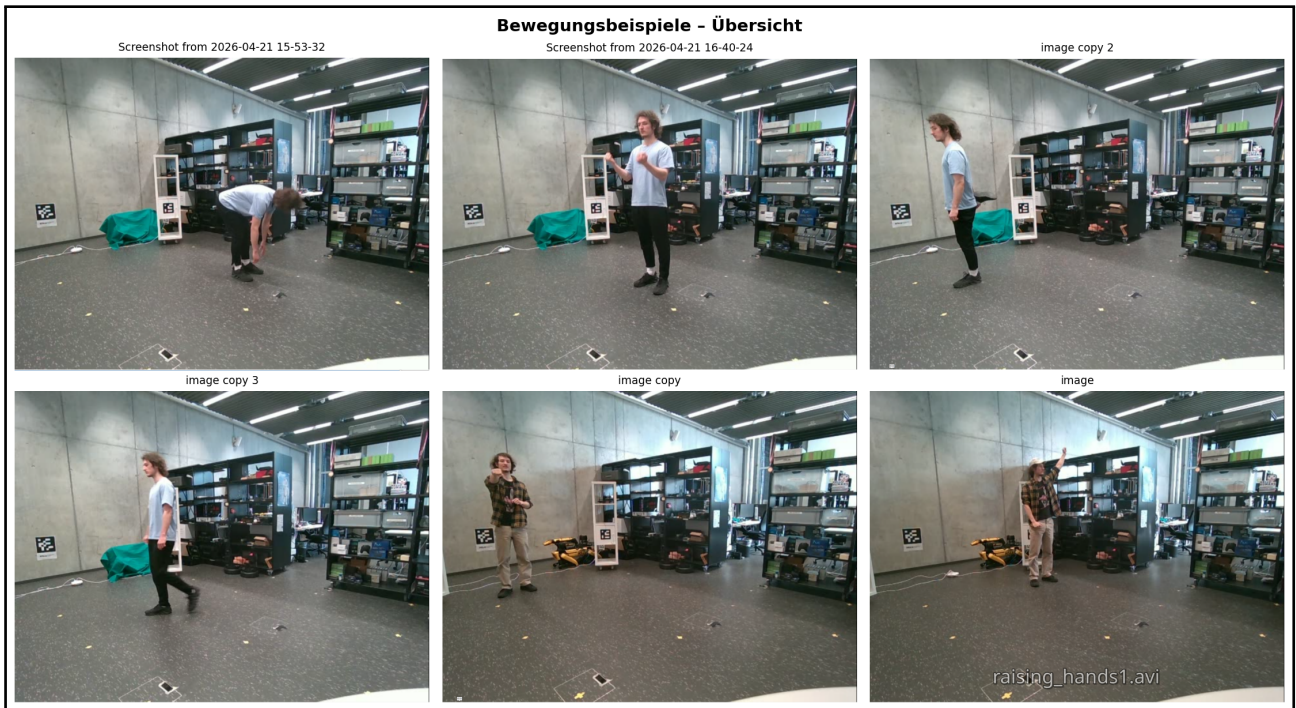


Abbildung 5.7: Beispielbilder aus den aufgenommenen Videos mit verschiedenen Bewegungsmustern nummeriert von oben links nach oben rechts und von unten links nach unten rechts. Bild 1: Bücken, Bild 2: Biceps Curls, Bild 3: Beindehnübungen, Bild 4: Gehen im Aufnahmebereich, Bild 5: Arme nach vorne ausstrecken, Bild 6: Arme über den Kopf heben

5.2.4 Datenset

Ein Sample eines Datenset besitzt immer zwei Dateien. Eine Datei in welcher die 3D Koordinate aufgeführt ist und eine in welcher die 21 CSI Pakete mit allen Werten aufgeführt sind. Die Koordinatenfiles haben den Dateityp .npy und die CSI Daten sind in .mat Dateien angelegt. Die beiden Dateien, welche zusammengehören haben denselben Namen und können so beim Training als ein Sample identifiziert werden. Das Identifizieren als Sample wird pro Ordner mit einer .txt Datei realisiert, welche die einzelnen Samples anhand ihres Dateinamens auflistet.

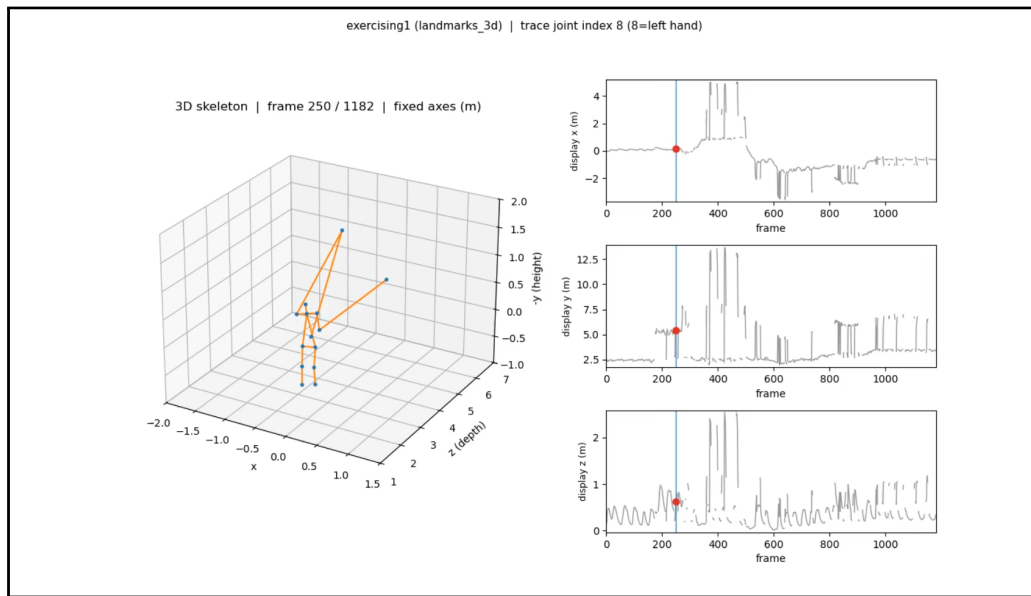
Die Datenstruktur des Datensets ist dabei wie folgt aufgebaut:

```
final_test_data
|-- csi
\-- keypoint
\-- final_test_data_list.txt
test_data
|-- csi
\-- keypoint
\-- test_data_list.txt
train_data
|-- csi
\-- keypoint
\-- train_data_list.txt
```

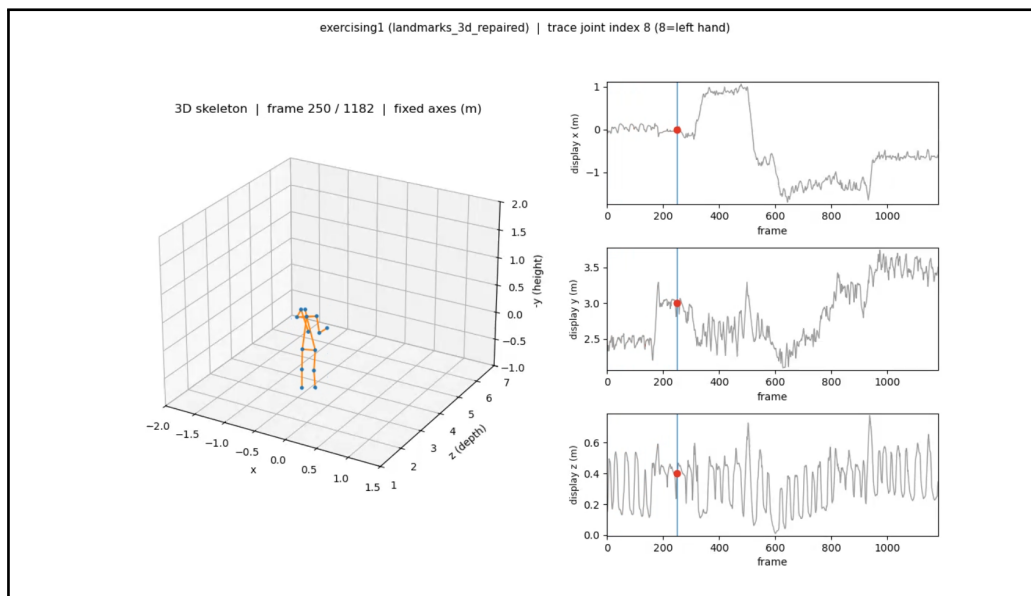
Das Datenset hatte eine Grösse von 94703 Samples und 11.9 GB. Dabei wurde sich am Datenset von Yan orientiert [14], welches ähnlich viele Samples beinhaltet. Ein Clip dauerte dabei 50 Sekunden, wobei vor diesem Aufnahmeintervall jeweils ein 10-Sekunden-Intervall verstrich, in welchem die Position eingenommen werden konnte. Die Aufnahmen wurden an zwei verschiedenen Tagen am Abend realisiert, um die Störungen durch vorbeilaufende Studenten möglichst klein zu halten. Um möglichst ähnliche Aufnahmebedingungen zu erreichen, wurden einige Massnahmen ergriffen. Die Router wurden wie in 3.4 geplant im Raum verteilt. AP 1, AP 2 und der Transmitter wurden dabei in einem länglichen Möbel auf die oberste Ablagefläche gestellt. Um die Positionen der Möbel über die beiden Aufnahmen konstant zu halten wurde auf den Boden ein kleiner Marker gesetzt. Für die Rotation der Möbel wurde dabei eine 45 Grad Rotation gewählt, sodass die Router mit ihren Antennen in die Rechteckmitte orientiert sind. AP 1 wurde auf den in 3.2.2 mit einem Kreis modellierten Tisch gestellt. Hierbei wurde für den Tisch und auch für die Router Position bzw. Kameraposition auf dem Tisch einen Marker gesetzt.

5.3 Generierung der 3D-Koordinaten

Die 3D-Koordinaten wurden wie in Kapitel 3.2.1 mithilfe des mathematischen Verfahrens und der Intel-RealSense-Kamera generiert. Da die rohen 3D-Koordinaten der Körperstellen, vor allem bei Bewegungen, eine niedrigere Qualität aufwiesen, wurden die 3D-Koordinaten noch nachbearbeitet. Nachfolgend ein Beispiel zweier Frames, bei denen einzelne Gelenkpunkte starke Ausreisser aufweisen, welche durch die Nachbearbeitung entfernt werden können.



(a) Vor der Nachbearbeitung



(b) Nach der Nachbearbeitung

Abbildung 5.8: 3D-Koordinatenberechnung desselben Frames vor und nach der Nachbearbeitung. Rechts neben dem Plot wird der Verlauf der x-, y- und z-Koordinaten der linken Hand gezeigt. Dies zeigt die Verschiebung mathematisch.

5.3.1 Nachbearbeitung der Koordinaten

Die Ground-Truth-Posen wurden aus einer computer-vision-basierten Pipeline mit Intel-RealSense-RGB-Daten und MediaPipe abgeleitet. Diese automatisch erzeugten 3D-Koordinaten weisen jedoch typischerweise Tiefenrauschen, kurzfristige Sprünge und anatomisch unplausible Gelenkstellungen auf. Damit das nachgelagerte WiFi-Modell nicht diese Artefakte, sondern die tatsächliche menschliche Kinematik lernt, wurde eine regelbasierte Ground-Truth-Refinement-Pipeline implementiert. Die gewählte Nachbearbeitung folgt dabei etablierten Ansätzen zur Skelett-Normalisierung, Kinematik-Begrenzung und zeitlichen Glättung von Posen [28, 29, 30].

1. Definieren von Knochen:

Die Modelle der Arbeit produzieren alle 14 Gelenke als Output, welche die Körperstellen der Person repräsentieren. Eine Knochenlänge ist definiert als die Distanz zwischen zwei Gelenkpunkten, z.B. die Distanz zwischen Kopf und Nacken oder Nacken und linker Schulter. Nachfolgend sind die 14 Gelenke und die 13 Knochen meines Modells aufgelistet. Da kamerabasierte Posen pro Frame leicht unterschiedliche Skelettlängen erzeugen, wird die Knochenstruktur pro Clip statistisch normalisiert. Die Medianbildung über den gesamten Clip und das anschliessende Clipping auf einen Referenzbereich stabilisieren die Skeletttopologie und reduzieren inkonsistente Längenartefakte [28].

Kopf-Nacken	Kopf
Linke Schulter-Nacken	Nacken
Rechte Schulter-Nacken	Linke Schulter
Linker Ellenbogen-Linke Schulter	Rechte Schulter
Linke Hand-Linker Ellenbogen	Linker Ellenbogen
Rechter Ellenbogen-Rechte Schulter	Rechter Ellenbogen
Rechte Hand-Rechter Ellenbogen	Linke Hand
Linke Hüfte-Nacken	Rechte Hand
Rechte Hüfte-Nacken	Linke Hüfte
Linkes Knie-Linke Hüfte	Rechte Hüfte
Linkes Fussgelenk-Linkes Knie	Linkes Knie
Rechtes Knie-Rechte Hüfte	Rechtes Knie
Rechtes Fussgelenk-Rechtes Knie	Linkes Fussgelenk
	Rechtes Fussgelenk

Knochen

Gelenke

2. Begrenzen der Knochenlängen:

Um die Knochenlängen zu begrenzen, wird der Median pro Knochen über alle Frames des Clips bestimmt. Danach werden die Knochenlängen in jedem Frame auf einen definierten Referenzbereich geclippt. Der Referenzbereich bewegt sich dabei zwischen 0.75x und 1.25x der durchschnittlichen Knochenlänge. Dieses Beschränken entfernt einzelne Ausreisser und verbessert die Qualität der 3D Koordinaten.

3. Beschränken von unmöglichen Winkeln:

Zusätzlich zu den Knochenlängen werden ausgewählte Gelenkwinkel begrenzt, um klar unrealistische Konfigurationen zu vermeiden. Dabei wird jeweils nur das mittlere Gelenk der Kette angepasst, sodass der Innenwinkel am mittleren Gelenk in einen zulässigen Bereich fällt:

- **Ellbogen und Knie:** Innenwinkel an Ellbogen bzw. Knie zwischen Ober-/Unterarm bzw. Oberschenkel und Unterschenkel. Standardbereich ca. 12° – 178° (sehr großzügig, v. a. gegen extreme Überstreckung bzw. vollständiges “Einknicken”).
- **Kopf–Hals–Torso:** Innenwinkel am Nacken zwischen dem Vektor von der Hüftmitte (Mittelwert linker und rechter Hüfte) zum Nacken und dem Vektor vom Nacken zum Kopf. Standardbereich ca. 15° – 142° ; angepasst wird nur der Kopf.

Diese Winkelgrenzen folgen dem Prinzip biomechanisch plausibler Gelenkrestriktionen, wie sie zur Bereinigung optisch geschätzter 3D-Posen eingesetzt werden [31].

4. Beschränken der Referenztiefe des gesamten Körpers:

Um starke, sprungartige Fehler der Tiefenwerte aller 3D-Koordinaten zu dämpfen, wird pro Frame ein skalarer Referenzwert gebildet: der Median der z -Koordinaten von Nacken sowie linker und rechter Hüfte. Ändert sich dieser Median zum vorherigen, bereits reparierten Frame um mehr als einen Schwellenwert (Standard **0,10 m**), wird die gesamte Pose entlang der z -Achse verschoben, sodass die Änderung des Referenzwerts genau diesem Maximum entspricht. Damit wird das typische Tiefen-Jitter von RGB-D-Sensoren reduziert, das bei videobasierter Pose-Erkennung zu abrupten Sprüngen in der Tiefenachse führen kann [32].

5. Beschränkung der Bewegungsgeschwindigkeit aller Gelenke:

Für jedes der 14 Gelenke wird die euklidische Verschiebung zwischen zwei aufeinanderfolgenden Frames begrenzt (Standard: maximal **0,08 m** pro Frame). Liegt die räumliche Distanz zwischen der Position im aktuellen und im vorherigen reparierten Frame über diesem Wert, wird die neue Position auf dem Strahl von der Vorher-Position zur Roh-Position so gesetzt, dass die Distanz genau dem Maximum entspricht. Die Begrenzung wirkt damit gleichermaßen in alle Raumrichtungen (x, y, z). Diese Geschwindigkeitsbegrenzung entspricht einem heuristischen Temporal-Smoothing, wie er zur Stabilisierung kamerabasierter Posen gegen hochfrequentes Zittern eingesetzt wird [33].

5.4 Export in Ground Truth Daten

Um die CSI-Daten und die 3D-Koordinaten der Gelenke für das Training des Modells nutzen zu können, müssen diese Daten in das richtige Format gebracht werden. Dabei wird die Form eines Tokens definiert, welcher als ganze Einheit in den Transformer eingespeist wird. Ein einziger Token hat dabei die Form (3,3,56,21), wobei die ersten zwei Dimensionen die MIMO-Kanalmatrix eines CSI-Pakets darstellen (3 Empfangsantennen $\times$ 3 Sendeantennen), die dritte Dimension die Anzahl der Subchannels und die vierte Dimension die Anzahl der Zeitschritte ist. Die 21 Zeitschritte ergeben sich aus den 3 Receiver-Routern mit je 7 CSI-Paketen pro Token, welche chronologisch zusammengeführt werden. Die Router sind also nicht als eigene Dimension abgebildet, sondern verteilen sich gleichmässig auf die Zeitachse.

Einem Token wird dann ein einzelner Frame bzw. ein Set an 3D-Koordinaten der Gelenke zugeordnet, welche die Ground Truth-Informationen für diesen Token darstellen. Die Ground Truth hat dabei die Form (14,3), wobei die erste Dimension die Anzahl der Gelenke ist und die zweite Dimension die 3D-Koordinaten (x,y,z) der Gelenke darstellt.

Um die erstellten Tokens mit den Ground Truth-Daten zu matchen, müssen zuerst korrekte, vergleichbare Timestamps generiert werden. Die Timestamps der Kameraframes werden direkt auf dem Laptop erstellt und gespeichert. Es existiert dann ein Timestamp pro Frame, welcher die Zeit angibt, wann der Frame auf dem Laptop gespeichert wurde. Pro CSI-Paket, welches vom Receiver-Router empfangen wird, wird ebenfalls ein Timestamp generiert, welcher die Zeit angibt, wann das CSI-Paket auf dem Router empfangen wurde. Um diesen Timestamp korrekt mit dem Kameratimestamp zu vergleichen, wurde ein NTP-Server mit einem Synchronisationskript erstellt. Dieses wird pro Session einmalig gestartet, sodass die Uhren der Router mit der Uhr des Laptops bzw. des Auswertungsgeräts synchronisiert werden. Danach werden die 3D-Koordinaten der Gelenke den Timestamps der CSI-Pakete zugeordnet. Es wird dabei

immer der Kameratimestamp als Referenz hinzugezogen und pro Router geschaut, welche 7 CSI-Pakete am nächsten beim Kameratimestamp liegen.

Es wird dann pro Sample bzw. Koordinaten-Token-Matching eine eigene Datei erstellt. Zudem werden automatisch Trainings-, Validierungs- und Testdaten in einem Verhältnis von 80-10-10 erstellt. So kann Validierung während des Trainingsprozesses und unabhängiges Testen nach dem Training gewährleistet werden.

5.5 Hardware Trainingsinfrastruktur

Yan et al. benötigten für ihren Trainingslauf etwa 450 bis 500 Epochen. Da dies sehr viele Epochen sind und in der zugrunde liegenden Studie ein verteilter Run auf mehreren Geräten durchgeführt wurde, musste das Risiko eines zu langen Trainingsprozesses reduziert werden.

Die Problemstellung bestand darin, das Training mit dem Datensatz von Yan et al. in einer praktikablen Zeit zu ermöglichen, damit mehrere Trainingsruns durchgeführt werden können und die Workstations nicht über längere Zeit blockiert werden.

Option 1 war ein lokales Training auf einer GPU. Zu Beginn der Arbeit wurde das Training auf einer Alienware mit einer NVIDIA RTX 2070 Grafikkarte getestet. Dabei zeigte sich, dass ein Trainingsrun mit 500 Epochen und einer Batch size von 64 etwa 4 Tage und 12 Stunden gedauert hätte.

Option 2 war eine Cloud-Infrastruktur oder verteiltes Training mit zwei GPUs im Robotics Lab der HSLU. Da die Cloud-Lösung zusätzliche Kosten verursacht hätte, wurde die zweite Variante gewählt. Zwei NVIDIA RTX 2070 Grafikkarten wurden miteinander verbunden, um das Training zu beschleunigen. Der anschliessende Trainingsrun mit 500 Epochen und einer Batch size von 64 dauerte etwa 2 Tage und 6 Stunden.

Die von mir gewählte Lösung war schliesslich das Training auf einer einzelnen GPU. Obwohl das verteilte Training funktionierte, fiel die Beschleunigung geringer aus als erwartet, da die Kommunikation und der Austausch der Gradienten zwischen den beiden Grafikkarten einen grossen Teil der zusätzlichen Rechenleistung wieder aufbrauchten. Nach einer kurzen Evaluation der nötigen Optimierungen wurde daher aufgrund des zusätzlichen Material- und Zeitaufwands entschieden, das Training auf einer GPU durchzuführen, um die Entwicklung der Arbeit nicht zu verzögern.

Es wurde dann allerdings eine zu diesem Zeitpunkt verfügbare NVIDIA RTX 4070 Grafikkarte für das Training genutzt, welche eine deutlich höhere Rechenleistung als die RTX 2070 besitzt.

5.6 Live-Framework

Zusätzlich zu den beiden Modellen wurde eine Software-Komponente gebaut, welche Live-Inferenz möglich macht. Mit Live-Inferenz ist es möglich, Daten in Echtzeit zu verarbeiten und Ergebnisse sofort zu erhalten, also die Möglichkeit zu haben, live eine Person zu erkennen. Es müssen lediglich, genau wie beim Akquirieren der Daten, die drei Router über einen Switch an die GPU-Workstation angeschlossen werden. Danach werden die Pakete live gelesen und tokenisiert. Im Anschluss werden sie in das Modell eingespeist und mit einem Forward-Pass verarbeitet und in einem 3D-Koordinatensystem ausgegeben. Das Live-Framework ist momentan nur für das Yan-Modell gebaut, kann in der Zukunft noch erweitert werden. Zudem ist der Output noch nicht zielführend, aufgrund der unterschiedlichen Router Anordnungen und

Positionierung von Objekten im Raum zwischen der Aufnahme der Trainingsdaten und der Live-Verarbeitung. Die Modelle sind also noch nicht in der Lage Personen live zu erkennen, da sie extrem sensitiv auf Verschiebungen im Setup sind. Es wurde auch eine Testmöglichkeit gebaut, in welchem existierende Testdaten eingelesen werden, um zu zeigen, dass das Problem nicht an der Live-Framework Komponente, sondern am Modell selbst liegt. Für diese vorge speicherten Daten funktioniert die Visualisierung reibungslos.

5.7 Überwundene Schwierigkeiten

5.7.1 Infrastruktur Startup

Beim Starten der Router-Infrastruktur kam es öfters zu Verbindungsproblemen. Die Receiver-Router haben sich nicht mit dem WLAN-Netzwerk des Transmitter-Routers verbunden. Dies war verwunderlich, da an den Konfigurationen der Router nichts verändert wurde. Diese Schwierigkeit wurde überwunden, indem zuerst immer der Transmitter-Router gestartet wurde und alle anderen Receiver-Router nacheinander mit dem Netzwerk des Transmitters verbunden wurden. Dabei wurden immer die Logs überprüft, ob der Receiver-Router wirklich im Netzwerk ist. Hier wurde mittels eines Laptops per LAN-Kabel und SSH auf die Konsole des Transmitter-Routers zugegriffen. Die Logs wurden mit dem Befehl `logread` -fäufgerufen. Des Weiteren wurde das Sendeskript auf dem Transmitter, welches die Daten an die verbundenen Geräte sendet, nicht mehr automatisch nach dem Start-up ausgeführt, sondern manuell über die Konsole gestartet. Diese Massnahmen haben die Verbindungsprobleme gelöst.

5.7.2 Interpolation und Wahl der Daten

Beim ersten Versuch der Erstellung des Modells wurde exakt die Architektur von Yan [14] übernommen. Diese Studie hatte allerdings ein leicht anderes Setup. Ihre Hardware nutzte lediglich 30 Subchannels und nicht 56, wie dies bei den in dieser Arbeit verwendeten Routern der Fall war. Zuerst wurde die Entscheidung getroffen, die Amplituden- und Phasenverschiebungswerte auf 30 Werte zu projizieren bzw. eine Interpolation vorzunehmen. Zudem wurden ebenfalls ganze leere Sequenzen als Trainingsdaten aufgenommen, was im Paper von Yan nicht gemacht wurde. Die erste Evaluation des Modells war dann aber nicht erfolgreich. Vor dem zweiten Training musste die Architektur des Modells leicht angepasst werden. Es wurde nun nicht mehr auf 30 Werte interpoliert, sondern direkt 56 Werte in das Modell eingespeist. Zudem wurden die komplett leeren Sequenzen weggelassen. Die Ergebnisse wurden dadurch viel besser.

6. Validation und Evaluation

In diesem Kapitel wird auf die produzierten Resultate eingegangen. Zuerst werden die Resultate beschrieben und nach Leistungsfähigkeit und Qualität evaluiert. Danach wird überprüft, ob die erwarteten Ziele erreicht wurden und ob die Arbeit damit valide ist.

6.1 Resultate

Beide Modelle basieren auf demselben Datensatz und wurden auf denselben Testdaten evaluiert. Der Trainingsdatensatz umfasst 75722 Samples, wobei das Validierungsset aus 9538 und das Testset aus etwa 9443 Samples besteht.

6.1.1 Yan-Modell

Das Yan-Modell wurde über 143 Epochen trainiert. Für die finale Auswertung wurde das Testset in neun Splits unterteilt und die Resultate anschliessend zu einem globalen Wert aggregiert. Der gewichtet berechnete mittlere MPJPE-Wert über alle 9009 Testsamples beträgt 201.84 mm, also rund 20 cm. Die beste Vorhersage eines einzelnen Samples liegt bei 12.99 mm, die schlechteste bei 5355.73 mm. Da der globale Median nicht direkt aus den Split-Medianen rekonstruierbar ist, wird hier bewusst nur der gewichtete Mittelwert sowie die globalen Extremwerte berichtet.

Beim Training wurde Early Stopping angewendet, nachdem der Validation-Loss über längere Zeit nicht mehr stark gefallen war. Die Trainingsdauer betrug insgesamt 30 Stunden. Auch wenn der Validation-Loss nicht mehr stark gefallen war, könnte man den Trainingsprozess für noch bessere Ergebnisse weiterlaufen lassen. Nachfolgend zwei ausgewählte Grafiken, welche den Verlauf des Trainingsruns zeigen. Zum einen den Validierungsverlauf mit dem MPJPE-Wert der Output-Pose des Modells mit dem höchsten Konfidenz-Wert, zum anderen den Verlauf der Loss-Funktion, welche die Differenz zwischen den vorhergesagten 3D-Koordinaten und den Ground-Truth-3D-Koordinaten darstellt. Diese zeigen beide, dass das Modell über die Epochen hinweg die Trainingsdaten gut gelernt hat.

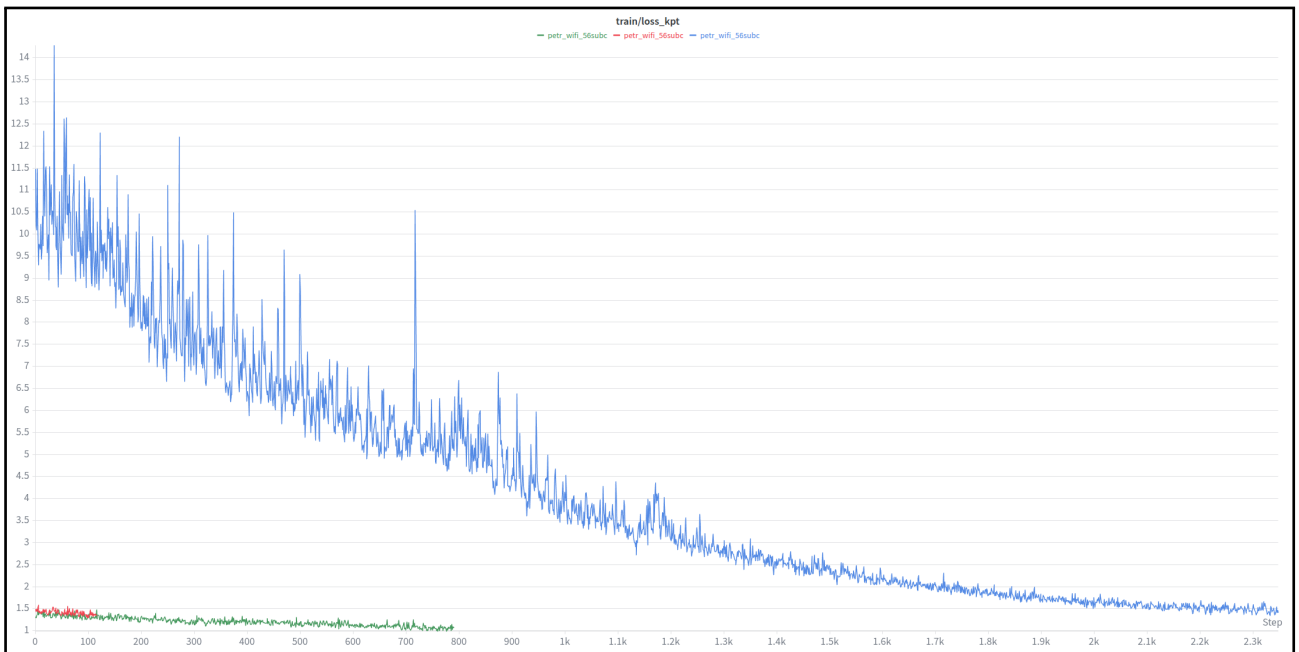


Abbildung 6.1: Training Loss Verlauf über die Epochen bzw. Steps. Die x-Achse zeigt die Anzahl der Epochen bzw. Steps und die y-Achse zeigt den Wert der Loss Funktion, welcher die Differenz zwischen den vorhergesagten 3D Koordinaten und den Ground Truth 3D Koordinaten darstellt. Die drei Farben der Kurven sind das Wiederaufnehmen des runs nach einer Pause bzw. einem Crash des Trainingsprozesses.

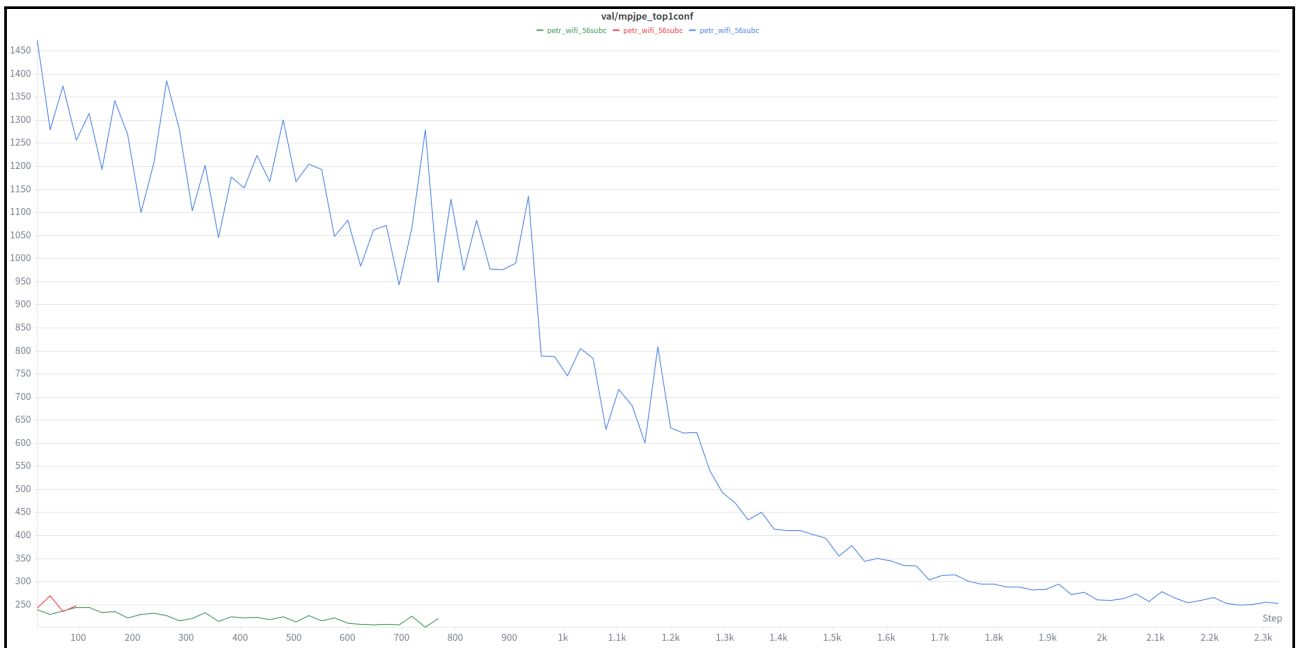


Abbildung 6.2: Validation-Loss-Verlauf über die Steps hinweg. Die x-Achse zeigt die Anzahl der Steps und die y-Achse zeigt den Verlauf des Validation-Losses in mm. Der Validation-Loss bezieht sich dabei auf den MPJPE-Wert der Output-Pose des Modells mit dem höchsten Konfidenz-Wert.

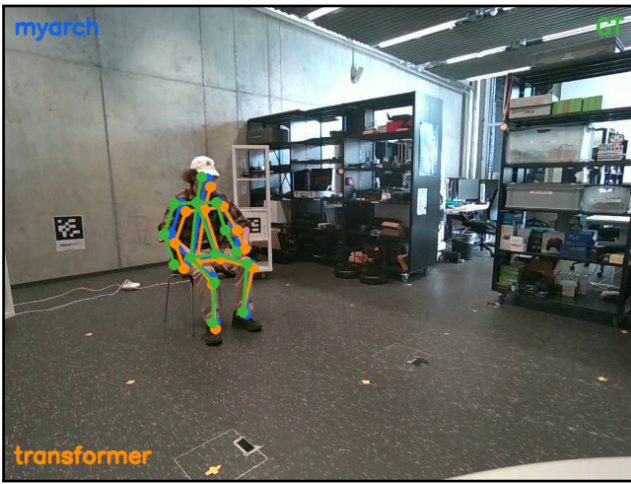
6.1.2 CTD-Modell

Das CTD-Modell wurde über 200 Epochen trainiert und performte auf dem Testdatensatz etwas besser als das Yan-Modell. Das CTD-Modell hat im Vergleich zur Yan-Architektur weniger Parameter, weshalb die Trainingszeit mit ca. 15 Stunden auch kürzer war. Insgesamt besitzt das CTD 3'120'000 Parameter, während das Yan-Modell ca. 13'100'000 Parameter umfasst. Der Evaluationslauf umfasste 9001 Samples und liefert damit eine belastbare Grundlage für den direkten Vergleich der beiden Modellvarianten.

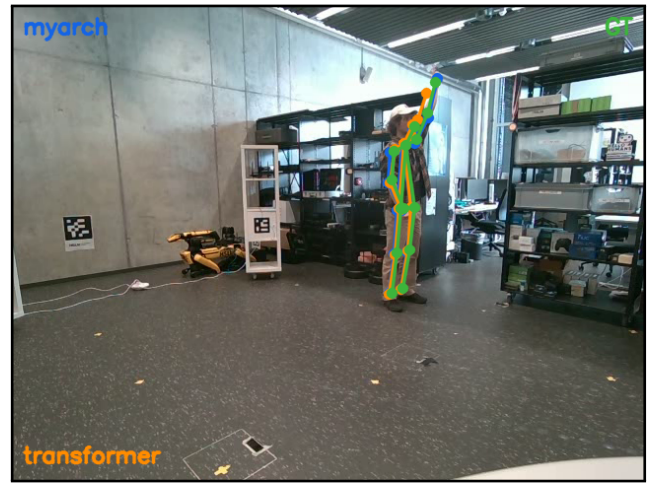
Beim CTD wurden zwei MPJPE-Werte gemessen: Der erste bezieht sich auf die Ausgabe nach dem Attention-Block, der zweite auf die Ausgabe nach dem Diffusionsblock. Auf dem gesamten Testset ergibt sich für die grobe Vorhersage ein mittlerer MPJPE von 112.77 mm, ein Median von 81.69 mm, ein Minimalwert von 13.51 mm sowie ein Maximalwert von 4118.17 mm. Die verfeinerte Vorhersage erreicht einen mittleren MPJPE von 111.87 mm, einen Median von 78.75 mm, einen Minimalwert von 12.87 mm sowie einen Maximalwert von 4114.10 mm.

Damit zeigt sich, dass der Diffusionsblock die bereits vom Attention-Block erzeugten Posen nochmals leicht verbessert. Die Differenz zwischen grober und verfeinerter Vorhersage ist zwar insgesamt moderat, bestätigt aber, dass das iterative Nachkorrekturprinzip einen messbaren Beitrag zur Genauigkeit leistet. Gleichzeitig wird sichtbar, dass trotz der verbesserten Mittelwerte weiterhin einzelne starke Ausreisser vorhanden sind, was auf schwierige Pose-Konstellationen und die verbleibende Varianz im Datensatz hinweist. Im direkten Vergleich mit dem Yan-Modell ist zudem zu berücksichtigen, dass dessen Architektur primär für die Vorhersage mehrerer Personen ausgelegt wurde. Diese Auslegung ist vorteilhaft, wenn gleichzeitig mehrere Instanzen im Raum vorhanden sind und das Modell zwischen verschiedenen Posen unterscheiden muss. Die Architektur des CTD-Modells hingegen ist nativ auf die Vorhersage von lediglich einer Person optimiert.

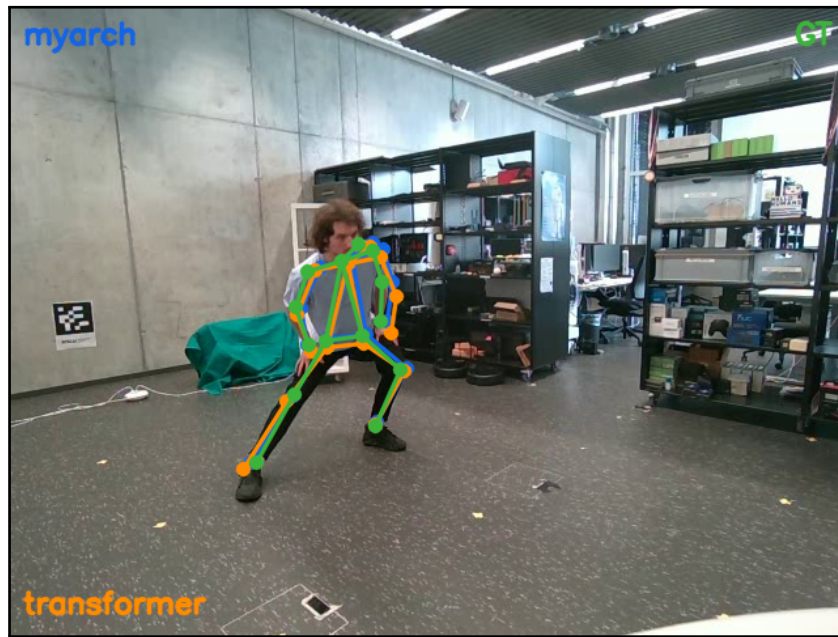
Nachfolgend einige Auszüge von verschiedenen Posen. Dabei wurde pro Bild die GT und die beiden Modellvorhersagen in das Kamerakoordinatensystem projiziert.



Frame 1



Frame 2



Frame 3

Abbildung 6.3: Resultate von drei verschiedenen Frames. Grün ist dabei die Ground Truth, blau die Vorhersage des CNN-Transformer-Diffusionsmodells und Orange die Vorhersage des Yan-Modells. Frame 1 zeigt eine Person auf einem Stuhl, Frame 2 eine Person mit erhobener Hand und Frame 3 eine Dehnübung.

6.2 Validation

Die Validation dieser Arbeit orientiert sich direkt an den in Abschnitt 1.2 formulierten Zielen. Diese Ziele lauten:

1. Die Lokalisierung von einer Person aus WiFi-Signalen.

Das entwickelte Modell ist in der Lage, eine Person anhand von WiFi-Signalen im definierten Raum zu erkennen und ihre Anwesenheit zu bestimmen. Damit wird das grundlegende Ziel einer Personenlokalisierung auf Basis von WiFi-Signalen erfüllt.

2. Die Lokalisierung soll dabei Koordinaten von Körperstellen vorhersagen.

Das Modell gibt nicht nur eine reine Personenpräsenz aus, sondern sagt auch Koordinaten relevanter Körperlandmarks voraus. Dadurch wird nicht nur festgestellt, ob sich eine Person im Raum befindet, sondern auch, wo sich einzelne Körperstellen befinden.

3. Die Lokalisierung soll in 3D erfolgen.

Die Vorhersagen werden in drei Dimensionen ausgegeben und können direkt als 3D-Körperpose interpretiert werden. Damit geht das Modell über eine reine 2D-Detektion hinaus und erfüllt die geforderte räumliche Schätzung.

4. Die Personenlokalisierung soll für einen bestimmten Raum erfolgen.

Das System wurde auf eine konkrete Experimentierumgebung trainiert und validiert. Der verwendete Raum ist dabei das Zimmer Nummer 13 im S12 (Robotics Lab Zimmer).

Es wurde damit gezeigt, dass die Ziele der Arbeit erreicht wurden.

7. Diskussion

In diesem Kapitel werden zentrale Entscheidungen, die im Verlauf der Arbeit getroffen wurden, kritisch reflektiert. Im Fokus stehen dabei die Wahl der Modellarchitekturen, des Router-Setups, der Ground-Truth-Generierung sowie der Trainingsinfrastruktur. Dabei werden sowohl die Gründe für die jeweilige Entscheidung als auch deren Auswirkungen auf das Endergebnis diskutiert.

7.1 Wahl der Machine Learning Modelle

Die Entscheidung für das Yan-Modell als erstes Referenzmodell entstand direkt aus der Literaturrecherche. Yan et al. waren die ersten, die zeigten, dass Transformer-Architekturen auch für CSI-basierte 3D-Pose-Schätzung funktionieren [14]. Da zu Beginn der Arbeit noch nicht abschliessend feststand, ob die optionale Erweiterung auf Mehrpersonen-Detektion umgesetzt werden würde, bot eine Architektur, die nativ mehrere Personen unterstützt, zusätzliche Flexibilität. Im weiteren Verlauf zeigte sich jedoch, dass primär logistische Gründe gegen eine Mehrpersonen-Variante sprachen. Genügend Probandinnen und Probanden zu finden, die über mehrere Stunden hinweg konsistente Bewegungsdaten produzieren, ohne dass weitere, unbeteiligte Personen den Aufnahmerraum betreten und potenziell die Signale stören, war praktisch nicht umsetzbar. Trotzdem lieferte das Yan-Modell auch im Einpersonen-Fall solide Resultate. Diese stellten sich aber erst nach einer Anpassung der Architektur ein. Die ursprünglich geplante Interpolation der 56 Subcarrier auf die 30 Subcarrier des Originalsetups von Yan musste verworfen werden, da sie zu einer signifikanten Verschlechterung der Trainingsresultate führte.

Parallel dazu wurde mit dem CTD-Modell eine eigene Architektur entwickelt, die auf der Kombination von CNN-Encoder, Attention-Block und Diffusions-Refiner basiert. Auch dieses Modell lieferte brauchbare Resultate und unterbot mit einem mittleren MPJPE von 111,87 mm den Wert des Yan-Modells bei deutlich kleinerer Parameterzahl. Bemerkenswert ist jedoch, dass der finale Diffusionsblock die Posen nur marginal um etwa 2 mm verbesserte. Der erwartete Mehrwert eines iterativen Refinements blieb somit hinter den Erwartungen zurück. Eine mögliche Erklärung ist, dass die durch den Attention-Block bereits erzeugten Posen die nutzbare Information aus den CSI-Tokens weitgehend ausschöpfen und der Diffusionsblock in der gewählten Konfiguration keine zusätzliche, signalseitig fundierte Korrektur mehr beitragen kann.

7.2 Wahl des Router-Setups

Das gewählte Setup mit einem Transmitter und drei Receiver-Routern war anspruchsvoller in der Umsetzung als ein einfacheres Setup mit nur einem Sender und einem Empfänger, wie es in der Vorgängerarbeit am Robotics Lab der HSLU verwendet wurde [23]. Insbesondere die Synchronisation und das Matching der CSI-Pakete über drei Receiver hinweg erforderten zusätzlichen Aufwand bei der Implementierung des NTP-basierten Zeitabgleichs und der Token-Konstruktion. Im Gegenzug konnten dadurch deutlich mehr und vielfältigere Daten pro Aufnahme erzeugt werden, was insbesondere für das Training tieferer Modelle mit grösserer

Parameterzahl entscheidend war.

Das Setup orientiert sich zudem an den in Kapitel 2 vorgestellten Studien zur CSI-basierten Landmark-Detektion, die ebenfalls mit drei Receivern arbeiten [14, 18]. Für ein finales Produkt im realen Einsatz wäre allerdings ein deutlich schlankeres Setup mit einem einzelnen Sender und einem Endgerät, zum Beispiel einem Smartphone oder Laptop, wünschenswert. Für die in dieser Arbeit verfolgten Untersuchungen zur grundsätzlichen Machbarkeit war das Drei-Receiver-Setup jedoch die richtige Wahl, da es eine bessere Vergleichbarkeit mit dem Stand der Forschung erlaubt.

7.3 Wahl der Ground-Truth-Generierung

Die Entscheidung für die Intel-RealSense-Kamera mit aktiver Tiefenmessung in Kombination mit MediaPipe hat sich im Nachhinein als solide Wahl erwiesen. Die Tiefeninformation lieferte eine direkt nutzbare z-Komponente und ermöglichte zusammen mit den intrinsischen Kameraparametern eine geschlossene 2D-zu-3D-Koordinatentransformation. Bereits zu Beginn der Arbeit war absehbar, dass eine Nachbearbeitung der Rohkoordinaten notwendig sein würde, da insbesondere die Tiefenmessung der RealSense, Rauschen und kurzfristige Sprünge aufweist. Die implementierte Pipeline mit Skelett-Normalisierung, Gelenkwinkel-Constraints und temporaler Glättung konnte diese Schwächen erfolgreich kompensieren und lieferte konsistente, anatomisch plausible Ground-Truth-Posen.

Eine Alternative für zukünftige Arbeiten wäre der Einsatz von BlazePose [34], das direkt aus reinen RGB-Bildern 3D-Koordinaten von Gelenken schätzt. Damit liesse sich der Aufwand der Tiefenmessung und der nachgelagerten Korrekturpipeline reduzieren. Allerdings ist im Gegenzug zu prüfen, ob die Tiefen-Genauigkeit eines rein RGB-basierten Verfahrens für die Anforderungen der CSI-Pose-Schätzung ausreicht, insbesondere bei Bewegungen senkrecht zur Bildebene.

7.4 Wahl der Trainingsinfrastruktur

Aufgrund der hohen Epochenzahl in der Studie von Yan et al. wurde zu Beginn der Arbeit erwartet, dass leistungsstärkere GPUs oder gar eine Cloud-Infrastruktur notwendig sein würden. Eine erste Hochrechnung auf einer NVIDIA RTX 2070 hatte für 500 Epochen eine Trainingsdauer von rund vier Tagen ergeben. Im Verlauf des Trainings zeigte sich jedoch, dass der Validation-Loss bereits ab etwa Epoche 150 nur noch marginal sank. Damit konnte die geplante Epochenzahl reduziert und auf den Einsatz einer Cloud-basierten GPU vollständig verzichtet werden. Stattdessen wurde das finale Training auf einer lokal verfügbaren NVIDIA RTX 4070 durchgeführt, wodurch Kosten als auch Setup-Aufwand gespart werden konnten. Diese Erfahrung zeigt, dass eine sorgfältige Beobachtung des Trainingsverhaltens oft wichtiger ist als eine pauschale Übernahme der Epochenangaben aus Referenzstudien.

8. Zusammenfassung und Ausblick

Diese Arbeit untersucht, ob sich die 3D-Lokalisierung einer Person und die Vorhersage von Körperlandmarks allein aus WiFi-Signalen realisieren lassen. Ausgangspunkt ist die Channel State Information (CSI), also die Amplitude und Phase der empfangenen WLAN-Signale auf einzelnen Subcarriern. Da sich bereits kleine Bewegungen im Raum auf diese Signale auswirken, bietet CSI eine datenschutzfreundliche Alternative zu Kamerasystemen, die in sensiblen Umgebungen oft nicht einsetzbar sind.

Ziel der Arbeit war es, für einen definierten Raum die 3D-Koordinaten von 14 Körperlandmarks einer einzelnen Person aus CSI-Daten vorherzusagen. Dafür wurden zwei Modellarchitekturen implementiert und verglichen. Als Referenz diente die Transformer-Architektur von Yan et al., die ursprünglich für die Mehrpersonen-Pose-Schätzung entwickelt wurde und über Encoder, Pose Decoder und Refine Decoder iterativ 3D-Posen verfeinert. Ergänzend wurde eine eigene CNN-Attention-Diffusion-Architektur (CTD) entwickelt, die CSI-Daten zuerst mit einem CNN-Encoder tokenisiert, anschliessend zeitliche Abhängigkeiten in einem Attention-Block modelliert und die vorhergesagten Gelenkpositionen in einem Diffusions-Refiner über zehn Schritte nachkorrigiert.

Für Datenerfassung und Evaluation wurde eine eigene Messinfrastruktur aufgebaut. Vier TP-Link Archer C7 Router mit Atheros-Chipsatz bildeten ein Setup aus einem Transmitter und drei Receivern. Nach dem Flashen von OpenWrt und der Konfiguration der Netzwerkverbindungen wurden CSI-Daten über UDP an eine Workstation übertragen. Als Ground Truth diente eine Intel RealSense Kamera in Kombination mit MediaPipe: Aus RGB- und Tiefenbildern wurden 2D-Landmarks extrahiert und in ein 3D-Koordinatensystem transformiert. Eine regelbasierte Nachbearbeitung reduzierte dabei Tiefenrauschen, anatomisch unplausible Gelenkwinkel und kurzfristige Sprünge in den Referenzdaten. Die Synchronisation zwischen Kamera und Routern erfolgte über NTP; pro Sample wurden 21 CSI-Zeitschritte den zugehörigen 3D-Posen zugeordnet.

Das erzeugte Datenset umfasst 94'703 Samples mit insgesamt 11,9 GB und orientiert sich am Umfang der Studie von Yan et al. Die Aufnahmen decken vielfältige Bewegungsmuster ab, darunter Bücken, Dehnübungen, Sportübungen, Gehen und Sitz-Stand-Wechsel. Eine statistische Voranalyse bestätigte, dass sich die Amplitudenvarianz der CSI-Signale unterscheidet, wenn sich eine Person im Raum befindet oder nicht. Dies rechtfertigte den Einsatz von Machine-Learning-Modellen für die Poseschätzung.

Beim Training zeigten sich praktische Herausforderungen. Beim Starten der Router Infrastruktur traten wiederholt Verbindungsprobleme auf, die durch ein festes Startprotokoll, manuelles Ausführen des Sendeskripts und Log-Überwachung über SSH behoben werden konnten. Ein erster Trainingsversuch mit Interpolation der 56 Subcarrier auf 30 Werte sowie der Aufnahme leerer Sequenzen lieferte keine brauchbaren Resultate. Erst nach Anpassung der Architektur auf 56 Subcarrier und dem Entfernen leerer Sequenzen verbesserte sich die Modellgüte deutlich.

Beide Modelle wurden auf demselben Datensatz evaluiert. Das Yan-Modell erreichte einen mittleren MPJPE von 201,84 mm bei rund 13,1 Millionen Parametern und einer Trainingsdauer von 30 Stunden. Das CTD-Modell mit etwa 3,1 Millionen Parametern erzielte einen mittleren MPJPE von 111,87 mm nach dem Diffusionsblock und trainierte in rund 15 Stunden. Der Diffusions-Refiner verbesserte die Attention-Vorhersage messbar, blieb aber insgesamt moderat. Trotz der besseren Mittelwerte traten weiterhin einzelne starke Ausreisser auf. Zusätzlich wurde

ein Live-Framework implementiert, das CSI-Daten in Echtzeit tokenisiert und Vorhersagen visualisiert. Für vorbereitete Testdaten funktioniert dies zuverlässig; bei verändertem Router-Setup oder geänderter Raumkonfiguration bricht die Live-Inferenz jedoch ein, was auf eine geringe Generalisierungsfähigkeit über das trainierte Setup hinaus hinweist.

Insgesamt wurden die formulierten Ziele erreicht: Es wurde gezeigt, dass aus WiFi-Signalen die 3D-Position einzelner Körperlandmarks einer Person in einem konkreten Raum vorhergesagt werden kann. Das CTD-Modell lieferte dabei die besseren Ergebnisse bei deutlich weniger Parametern. Gleichzeitig zeigt die Arbeit Grenzen auf, insbesondere bei Setup-Wechseln, Live-Betrieb und der Übertragbarkeit auf neue Umgebungen.

Für weiterführende Arbeiten bieten sich mehrere Richtungen an. Ein zentraler Punkt ist die Verbesserung der Generalisierung auf neue Räume und veränderte Router-Layouts. Neuere Ansätze wie PerceptAlign zeigen, dass explizite Einbindung der Gerätegeometrie in ein gemeinsames Koordinatensystem die Cross-Layout-Performance deutlich verbessern kann [18]. Eine Integration solcher Geometrie-Informationen wäre auch für das hier entwickelte Setup vielversprechend.

Als weiterer Ansatz für Domain-Shift über mehrere Räume hinweg bietet sich *Nested Learning* an [35]. Das im Language-Model-Bereich vorgestellte Paradigma modelliert ein Netzwerk als verschachtelte Optimierungsprobleme, bei denen sich Modellparameter abhängig vom eingehenden Kontext anpassen können. Übertragen auf eine CSI-basierte Pose-Schätzung könnten die eingehenden CSI-Tokens dazu genutzt werden, die Gewichte des Modells situationsabhängig zu modifizieren, anstatt ein fixes, raumspezifisches Mapping zu memorieren. Damit liesse sich potenziell eine schnellere Anpassung an neue Umgebungen erreichen, ohne das gesamte Modell neu trainieren zu müssen.

Schliesslich wäre eine Erweiterung von der Einzelpersonen- auf die Mehrpersonen-Schätzung interessant, wofür die Yan-Architektur grundsätzlich ausgelegt ist. Dafür bräuchte es jedoch zusätzliche Trainingsdaten und angepasste Evaluationsprotokolle. Insgesamt bleibt WiFi-basierte 3D-Pose-Schätzung ein aktives Forschungsfeld mit hohem Potenzial für datenschutzfreundliche Anwendungen, sofern Robustheit, Setup-Unabhängigkeit und praktische Deploybarkeit weiter verbessert werden.

Literaturverzeichnis

- [1] K. He, G. Gkioxari, P. Dollár, and R. Girshick, “Mask R-CNN.” Comment: open source; appendix on more results.
- [2] S. Singh, A. Yadav, J. Jain, H. Shi, J. Johnson, and K. Desai, “Benchmarking Object Detectors with COCO: A New Path Forward.” Comment: Technical report. Dataset website: <https://cocorem.xyz> and code: <https://github.com/kdexd/coco-rem>.
- [3] F. Pasti and N. Bellotto, *Evaluation of Computer Vision-Based Person Detection on Low-Cost Embedded Systems*.
- [4] M. Valverde, A. Moutinho, and J.-V. Zacchi, “A Survey of Deep Learning-Based 3D Object Detection Methods for Autonomous Driving Across Different Sensor Modalities,” vol. 25, no. 17.
- [5] “HESAI JT128 3D LiDAR AVG AMR Map Recognition Navigation.”
- [6] “TP-Link TL-Wr841n - kaufen bei Digitec.”
- [7] M. Youssef, M. Mah, and A. Agrawala, “Challenges: Device-free passive localization for wireless environments,” in *Proceedings of the 13th Annual ACM International Conference on Mobile Computing and Networking*, MobiCom ’07, pp. 222–229, Association for Computing Machinery.
- [8] F. Adib and D. Katabi, “See through walls with WiFi!,” in *Proceedings of the ACM SIGCOMM 2013 Conference on SIGCOMM*, SIGCOMM ’13, pp. 75–86, Association for Computing Machinery.
- [9] abiresearch.com, “Whitepaper — The Commercial Market Value for Wi-Fi Sensing.”
- [10] IEEE Computer Society, “Ieee standard for information technology–telecommunications and information exchange between systems local and metropolitan area networks–specific requirements - part 11: Wireless lan medium access control (mac) and physical layer (phy) specifications,” *IEEE Std 802.11-2016 (Revision of IEEE Std 802.11-2012)*, pp. 1–3534, 2016.
- [11] D. Halperin, W. Hu, A. Sheth, and D. Wetherall, “Tool release: Gathering 802.11n channel state information,” *ACM SIGCOMM Computer Communication Review*, vol. 41, no. 1, pp. 53–53, 2011.
- [12] D. Tse and P. Viswanath, *Fundamentals of Wireless Communication*. Cambridge: Cambridge University Press, 2005.
- [13] P. Kocheta, N. S. Bhatia, and K. Obraczka, “PulseFi: A Low Cost Robust Machine Learning System for Accurate Cardiopulmonary and Apnea Monitoring Using Channel State Information.” Comment: 12 pages, 10 figures.

- [14] K. Yan, F. Wang, B. Qian, H. Ding, J. Han, and X. Wei, “Person-in-WiFi 3D: End-to-End Multi-Person 3D Pose Estimation with Wi-Fi,” pp. 969–978.
- [15] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, “End-to-end object detection with transformers,” in *Computer Vision – ECCV 2020*, pp. 213–229.
- [16] F. Wang, S. Zhou, S. Panev, J. Han, and D. Huang, “Person-in-WiFi: Fine-Grained Person Perception Using WiFi,” pp. 5452–5461.
- [17] W. Jiang, H. Xue, C. Miao, S. Wang, S. Lin, C. Tian, S. Murali, H. Hu, Z. Sun, and L. Su, “Towards 3D human pose construction using wifi,” pp. 1–14. [TLDR] WiPose is the first 3D human pose construction framework using commercial WiFi devices that addresses a series of technical challenges and can localize each joint on the human skeleton with an average error, achieving a 35% improvement in accuracy over the state-of-the-art posture construction model designed for dedicated radar sensors.
- [18] S. Jia, Y. Lu, B. Liu, X. Zhang, P. Zhao, X. Tang, Y. Wei, J. Huang, H. Yan, and Z. Liu, “Breaking coordinate overfitting: Geometry-aware wifi sensing for cross-layout 3d pose estimation.”
- [19] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need.”
- [20] X. Zhu, W. Su, L. Lu, B. Li, X. Wang, and J. Dai, “Deformable DETR: Deformable transformers for end-to-end object detection.”
- [21] E. Ramesh and et al., “Latentcsi: Real-time reconstruction of physical scenes from wifi csi via latent diffusion,” *arXiv preprint arXiv:2506.10605 / Technical Report*, 2025.
- [22] baeldung, “Focal Length and Intrinsic Camera Parameters — Baeldung on Computer Science.”
- [23] S. Siegler, “Untersuchung zur wifi-basierten personenerkennung,” 2023.
- [24] The OpenWrt Project, “Openwrt wireless freedom.” <https://openwrt.org>, 2026. Accessed: 2026-05-19.
- [25] H. A. H. Ali and S. Seytnazarov, “Human walking direction detection using wireless signals, machine and deep learning algorithms,” *Sensors*, vol. 23, no. 24, p. 9726, 2023. Details the absolute requirement of custom OpenWrt images on TP-Link hardware to extract CSI data.
- [26] Y. Xie, Z. Li, and M. Li, “Precise power delay profiling with commodity wi-fi,” *IEEE Transactions on Mobile Computing*, vol. 18, no. 6, pp. 1343–1355, 2015. M introduction of the Atheros CSI Tool utilizing modified ath9k drivers on OpenWrt.
- [27] J. Xiao, K. Wu, Y. Yi, and L. Wang, “Fimd: Fine-grained device-free motion detection.” Placeholder entry for the cited Xiao et al. work used in the thesis draft.
- [28] T. Chen *et al.*, “Anatomy-aware 3d human pose estimation in video,” *IEEE Transactions on Multimedia*, vol. 23, pp. 3412–3423, 2021.

- [29] F. Bogo, J. Romero, G. Pons-Moll, and M. J. Black, “Keep it SMPL: Automatic estimation of 3d human pose and shape from a single image,” in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2016.
- [30] A. Zeng *et al.*, “SmoothNet: A plug-and-play network for refining human poses in videos,” in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2022.
- [31] I. Akhter and M. J. Black, “Pose-conditioned joint angle limits for 3d human pose reconstruction,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 1446–1455, 2015.
- [32] R. A. Clark, A. L. Bryant, Y.-H. Pua, P. McCrory, K. Bennell, and M. Regan, “Validity of the microsoft kinect for assessment of postural control and sagittal plane kinematics,” *Gait & Posture*, vol. 36, no. 3, pp. 421–425, 2012.
- [33] G. Casiez, N. Roussel, and D. Vogel, “1€ filter: A simple speed-based low-pass filter for noisy input in interactive systems,” in *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI)*, pp. 2527–2530, 2012.
- [34] V. Bazarevsky, I. Grishchenko, K. Raveendran, T. Zhu, F. Zhang, and M. Grundmann, “Blazepose: On-device real-time body pose tracking,” *arXiv preprint arXiv:2006.10204*, 2020.
- [35] A. Behrouz, M. Razaviyayn, P. Zhong, and V. Mirrokni, “Nested learning: The illusion of deep learning architecture.”

9. Anhänge und KI-Deklaration

9.1 KI-Deklaration

In der Arbeit wurde KI bei der Literaturrecherche, beim Erstellen der Software und beim Schreiben der schriftlichen Arbeit verwendet. Bei der Literaturrecherche wurde KI vor allem dazu verwendet, einen Überblick über das CSI-Forschungsfeld zu erhalten und komplexere Themen mit Fragen an KI-Chatbots inhaltlich zu durchdringen. Beim Erstellen der Software wurde Cursor benutzt, welcher direkte KI-Integration im Code-Editor zur Verfügung stellt. Auch beim Schreiben der Arbeit wurde KI benutzt, um geschriebene Texte zu kontrollieren und Nachverbesserungen zu erstellen.

9.2 Anhänge

9.2.1 Aufgabenstellung

Hier lässt sich das Dokument der Aufgabenstellung finden, welche zu Beginn der Arbeit erstellt wurde. Das Dokument lässt sich auch im Repository der schriftlichen Arbeit finden, welches unten in Projektmanagement verlinkt ist.

Aufgabenstellung

Modul:	Dept I BAA FS26
Titel:	WiFi CSI People Detection
Ausgangslage und Problemstellung:	Die Erkennung von Menschen in der Umgebung ist eine wichtige Grundlage für viele Anwendungen im Bereich Mensch-Maschine-Interaktion. Das Mittel der Wahl sind hierbei Kameras, allerdings ist deren Einsatz von der Lichtsituation abhängig und zum Schutz der Privatsphäre nicht überall möglich. Neue Entwicklungen ermöglichen People Detection aus WiFi-Signalen und bieten damit komplementäre Möglichkeiten. In einer ersten Arbeit wurde die prinzipielle Machbarkeit gezeigt und ein System entwickelt, das Anwesenheit erkennt.
Ziel der Arbeit und erwartete Resultate:	Das Ziel der Arbeit ist das Erkennen von Personen aus WiFi-Signalen. Idealerweise in 2D zum Vergleich mit Bilddaten, möglicherweise im 3D-Raum mit Vergleich zu Tiefendaten. Im Fokus steht die Detektion einer Person mit der Option mehrere Personen zu erkennen. Im Fokus liegt die Datenakquisition für einen bestimmten Raum, möglicherweise werden in mehreren Räumen Experimente durchgeführt.
Gewünschte Methoden, Vorgehen:	Übersicht über den State-of-art, Datenerfassung und -aufbereitung, Auswahl von Merkmalen und Modellen, Vorbereitung von Trainingsdaten, Modelltraining und -optimierung, Validierung und Tests. Es soll die korrekte Datenakquise, Datenaufbereitung und Synchronisierung der Kameradaten mit den CSI-Daten, mit dem Trainieren eines Modells der Vorgängerarbeit bewiesen werden. Danach soll ein Modell erstellt werden, welches eine Personendetektion in einem 2D-Raum ermöglicht. Das Modell beschränkt sich auf einen Raum mit einer Person die sich darin befindet bzw. detektiert werden soll. Die Person soll mittels einer 2D Maske erkannt werden. Optionale Erweiterungen sind dabei die Erkennung von Körper-Landmarks der Person, welche die wichtigsten Gelenke eines Menschen wie z.B. Handgelenke, Ellenbogen oder Hüftgelenke und eine Erweiterung auf den 3D-Raum. Zudem soll eine Validation des Modells erstellt werden, welche die Kameradaten mit dem Output des Modells vergleicht. Es soll dabei auch die Transformer-Architektur und CNN-Architektur für Personenerkennung mittels WiFi miteinander verglichen werden.
Kreativität, Methoden, Innovation:	Optional könnten die CSI-Daten für das Modell vorgängig mit Aufbereitungsmethoden wie «Conjugate Multiplication» und «PCA» verarbeitet werden. Diese Methoden vergrössern zum einen den dynamischen Anteil des Signals und komprimieren den CSI-Datensatz.
Sonstige Bemerkungen:	ADML vorteilhaft.

Projektteam

Student:in 1:	Fabian Stettler
Betreuer:in:	Jensen

Abbildung 9.1: Aufgabenstellung der Arbeit 1

Auftraggeber

Firma:	HSLU AI Robotics Research Lab
Ansprechperson:	Björn Jensen
Funktion:	
Strasse:	
PLZ/Ort:	
Telefon:	+41 41 349 35 76
E-Mail:	bjoern.jensen@hslu.ch
Website:	

Version 13.06.2023 / bcl

Abbildung 9.2: Aufgabenstellung der Arbeit 2

9.2.2 Projektmanagement

Hier lassen sich die Links zu den Repositories finden, welche bei Zugriff auf die GitLab-Switch-Umgebung abgerufen werden können. Zudem lassen sich die Planung des Projekts, Sitzungsprotokolle und die Risiko-Analyse im Code-Repository der schriftlichen Arbeit unter /admin finden.

Datenverwaltung

Repository	Link
Code Repository dieser Arbeit	https://gitlab.switch.ch/rair/fs26_baa_stettler_fabian_wifi_detection
Repository des Textdokuments	https://gitlab.switch.ch/rair/fs26_baa_stettler_fabian_wifi_detection_report
Code Repository von Vorgängerarbeit am Robotics Lab der HSLU	https://gitlab.switch.ch/rair/fs26_baa_stettler_fabian_wifi_detection_prev_version_cnn
Code Repository von Yan et al.	https://aiotgroup.github.io/Person-in-WiFi-3D/

Tabelle 9.1: Verwendete Repositories

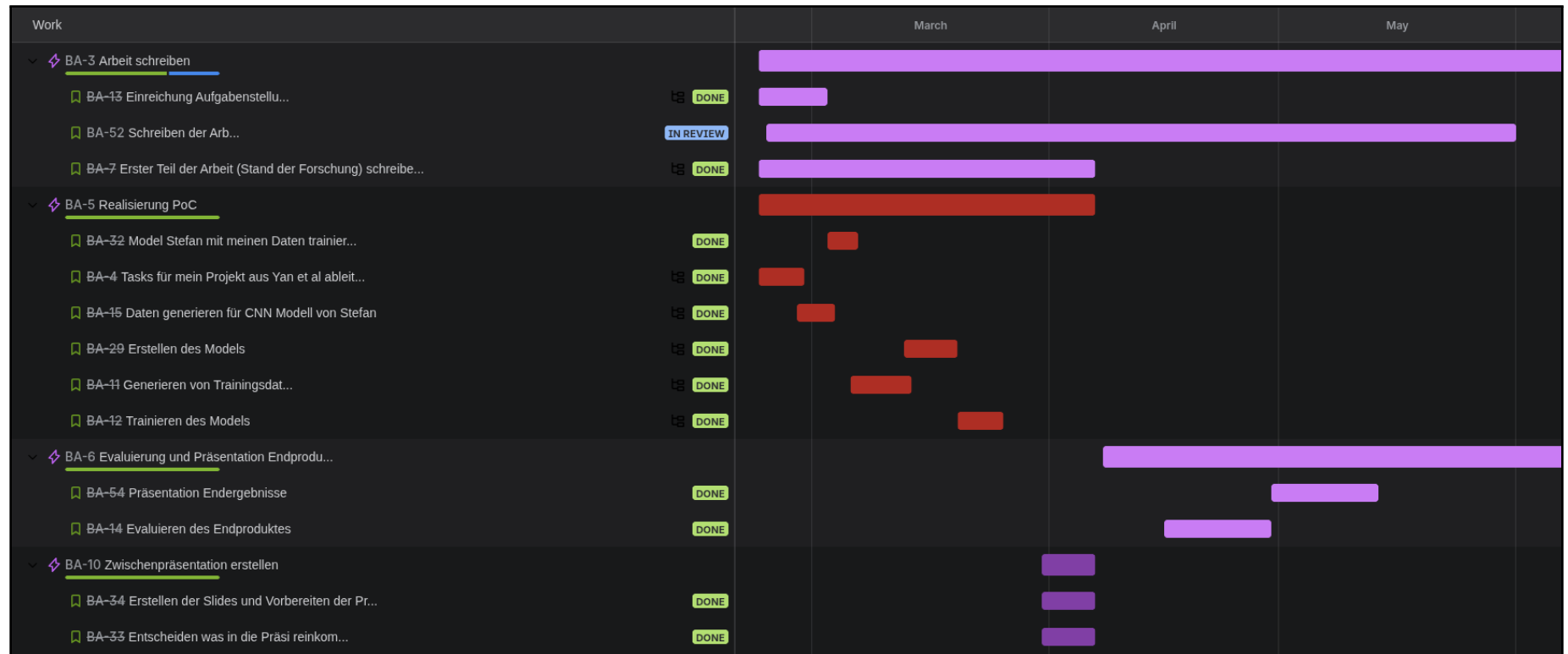


Abbildung 9.3: Planung der Arbeit

Risk Identification					
ID	Description	Potential Impact	Severity	Likelihood	Possible Mitigation Strategies
1	I'm unable to generate coordinates for the landmarks during acquisition of the training data	I can not train the model	Critical		3 buy azure kinect
2	Matching of camera and CSI Data does not work properly	I can not train the model	Critical		2 accept slight deviation between frames and CSI Packages
3	Transformer model does not converge	model does not work properly	Medium		1 use different model architecture
4					
5					
6					
7					
8					
9					
Scores for Severity and Likelihood Severity values = (Low, Medium, High, Critical) Likelihood = (1, 2, 3, 4, 5)					

Abbildung 9.4: Risiko-Analyse der Arbeit

9.2.3 Daten

Datensatz	Link
Daten und Auswertungen (HSLU OneDrive)	https://hsluzern-my.sharepoint.com/:f:/r/personal/fabian_stettler_stud_hslu_ch/Documents/BAA/data?csf=1&web=1&e=jMno9h
Rohdaten (kDrive)	https://kdrive.infomaniak.com/app/share/1778937/5a0f3cc5-3b7e-41e8-b872-311522de0a93

Tabelle 9.2: Verwendete Datensätze